LAPORAN TUGAS BESAR 1 DASAR INTELEGENSI ARTIFISIAL

IF3070 Dasar Intelegensi Artifisial

Kelompok 33

Disusun Oleh

Faizal Ali	(18224007)
Muhammad Fariz Difa’urrahman	(18224028)
Arkandhiya Ibrahim Dewantara	(18224052)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

DAFTAR ISI..2
DESKRIPSI PERSOALAN...3
PEMBAHASAN...3
KESIMPULAN DAN SARAN.. 3
PEMBAGIAN TUGAS... 3
REFERENSI.. 3
LAMPIRAN..3
    LAMPIRAN FORM PENGGUNAAN GENERATIVE AI..3
    LAMPIRAN LAIN.. 3

1. DESKRIPSI PERSOALAN

Deskripsi

Gambar 1. 1 (Sumber: Gambar Kapal)

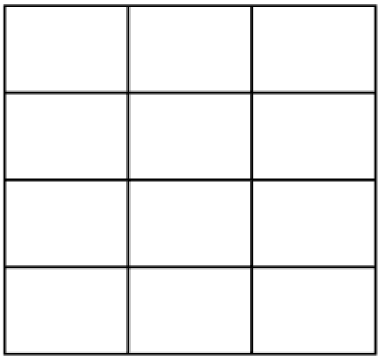
Anda adalah seorang *Forward Deployed Engineer* (FDE) di PT ~~Owen~~ Intim Line yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda.

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik dan keputusan pemilihan letak kendaraan belum tentu menghasilkan keuntungan yang optimal.

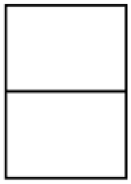
Tim operasional meminta Anda membangun sistem pendukung keputusan untuk menentukan kendaraan yang akan diangkut serta posisi dan orientasi setiap kendaraan pada geladak. Sistem tersebut harus dapat mengevaluasi berbagai kemungkinan konfigurasi, mempertimbangkan fungsi-fungsi objektif yang ditentukan, dan menghasilkan solusi penataan yang optimal.

Spesifikasi

Tugas ini bertujuan untuk membuat konfigurasi penempatan kendaraan paling optimal pada sebuah geladak kapal kargo. Setiap kendaraan dan geladak direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang. Solusi dapat memilih kendaraan mana yang diangkut dan menentukan posisi serta orientasinya di dalam geladak, dengan tetap memenuhi seluruh batasan.



Kapal



Kendaraan

Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Vehicle	id	“Owen”	*Identifier*
	Dimensions	{w:3, l:4}	Ukuran sebuah kendaraan, terdiri atas *width* dan *length* (satuan grid)
	Orientation	Horizontal	*Horizontal* atau *Vertical*
	ShippingFee	60	Tarif/keuntungan pengiriman kendaraan (satuan juta rupiah)
	Weight	5	Berat kendaraan (satuan unit)
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan kendaraan (dalam sisa

			hari)
Ship	Dimensions	{w:10, l:20}	Ukuran geladak pada kapal, terdiri atas *width* dan *length* (satuan grid)
	MaxCapacity	500	Kapasitas angkut maksimum suatu kapal (satuan unit)

2. PEMBAHASAN

2.1. Pra-implementasi Algoritma

2.2.1. Pemilihan Objective Function

1. **Objective function**

   Memaksimalkan total perolehan biaya pengiriman (*shipping fee*) dari seluruh kendaraan yang berhasil dimuat ke atas kapal sebagai tolok ukur utama performa algoritma pencarian.

2. **Kendaraan tidak boleh saling bertumpuk (*no overlapping*)**
   Memastikan setiap sel koordinat grid pada geladak kapal hanya ditempati oleh maksimal satu kendaraan pada waktu yang sama guna menjamin validitas posisi spasial dua dimensi.

3. **Kendaraan harus berada di dalam batas geladak.**

   Mewajibkan seluruh dimensi kendaraan, baik secara horizontal maupun vertikal, berada sepenuhnya di dalam batas fisik panjang dan lebar geladak kapal (*out of bounds prevention*).

4. **Kapal tidak boleh mengangkut melebihi kapasitasnya.**

   Menetapkan total akumulasi berat seluruh kendaraan agar tidak melampaui batas daya angkut maksimum kapal sebagai *hard constraint* demi keselamatan struktural pelayaran.

5. ***Weight***

   Mempertimbangkan bobot setiap kendaraan sebagai parameter kelayakan muatan guna mengoptimalkan pemanfaatan ruang dan mencegah pelanggaran batas tonase.

6. **Estimasi Waktu Kedatangan (*ETA*)**

   Mengatur aksesibilitas penataan muatan berdasarkan urutan pembongkaran di pelabuhan tujuan, sehingga kendaraan dengan jadwal bongkar awal tidak terhalang oleh kendaraan lain (mencegah *cargo shifting*).

2.2.2. Implementasi Objective Function

objective.py

```
from objects.state import State

def objective(state: State) -> int:
  return state.get_objective()
```

Fungsi	`def objective(state: State) -> int:`
Deskripsi	Berfungsi untuk menerima parameter berupa objek state (dari kelas State), lalu memanggil method get_objective() dari objek tersebut untuk menghitung dan mengevaluasi nilai kualitas/skor dari suatu solusi (individu) dalam algoritma genetika.
Output	Mengembalikan sebuah nilai berupa bilangan bulat (int) yang merepresentasikan hasil perhitungan nilai objektif (objective value / fitness value) dari state tersebut.

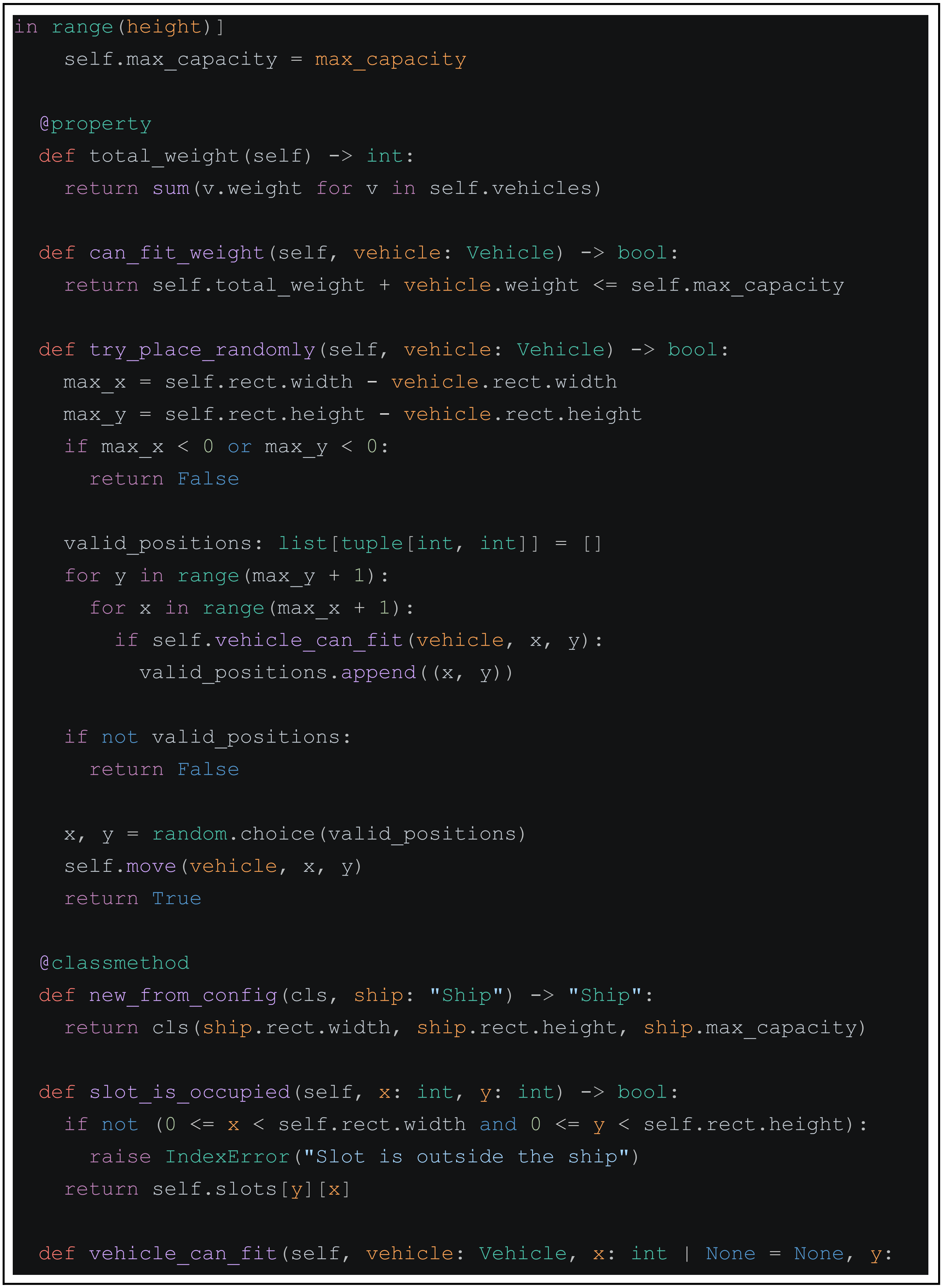
Source Code

```
def objective(state: State) -> int:
  return state.get_objective()
```

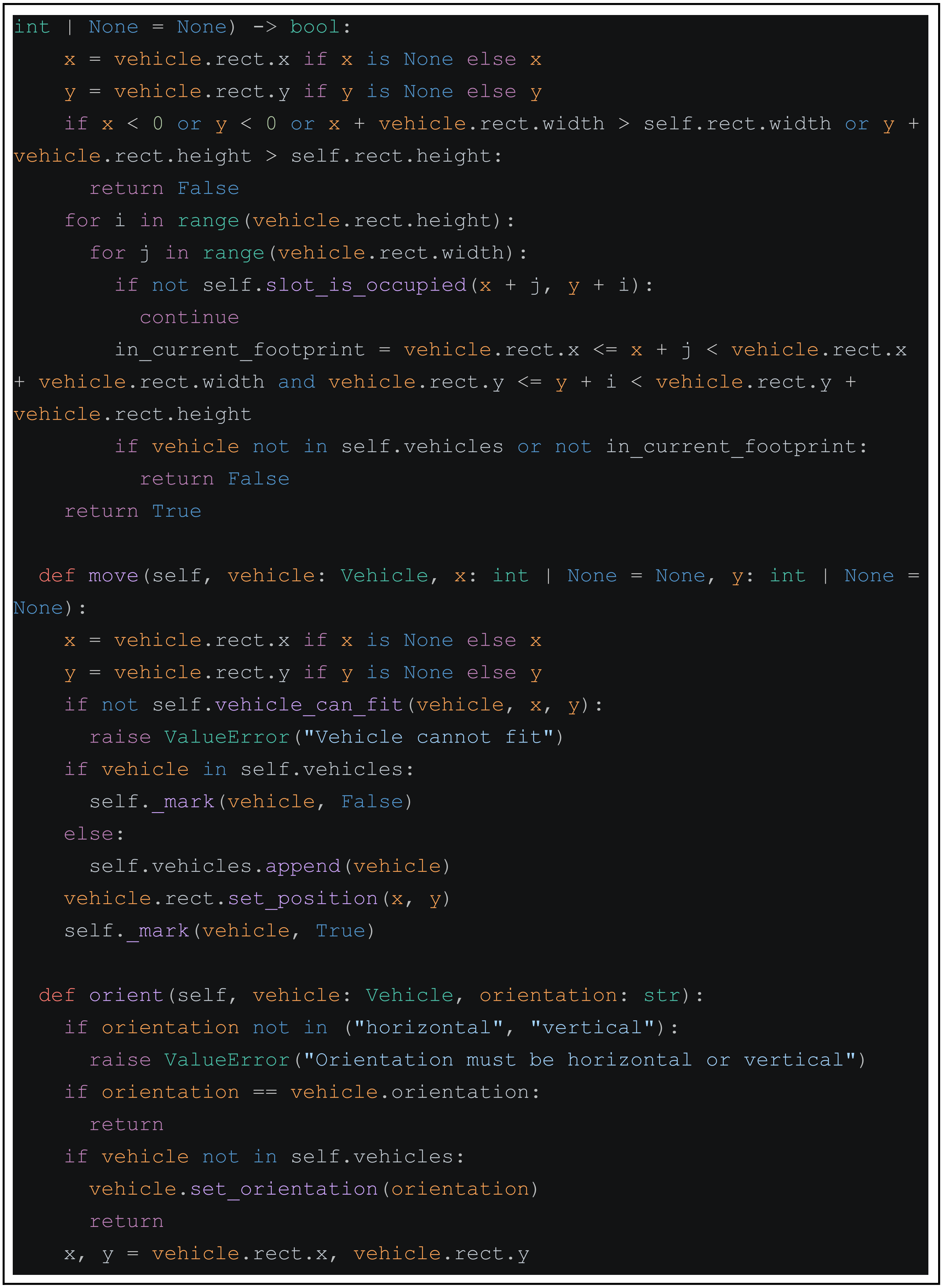
2.2.3. Implementasi Ship

ship.py

```
import random

from objects.rect import Rect
from objects.vehicle import Vehicle

class Ship:
  def __init__(self, width: int, height: int, max_capacity: int):
    self.rect = Rect(width, height)
    self.vehicles: list[Vehicle] = []
    self.slots: list[list[bool]] = [[False for _ in range(width)] for _
```



```
in range(height)]
    self.max_capacity = max_capacity

  @property
  def total_weight(self) -> int:
    return sum(v.weight for v in self.vehicles)

  def can_fit_weight(self, vehicle: Vehicle) -> bool:
    return self.total_weight + vehicle.weight <= self.max_capacity

  def try_place_randomly(self, vehicle: Vehicle) -> bool:
    max_x = self.rect.width - vehicle.rect.width
    max_y = self.rect.height - vehicle.rect.height
    if max_x < 0 or max_y < 0:
      return False

    valid_positions: list[tuple[int, int]] = []
    for y in range(max_y + 1):
      for x in range(max_x + 1):
        if self.vehicle_can_fit(vehicle, x, y):
          valid_positions.append((x, y))

    if not valid_positions:
      return False

    x, y = random.choice(valid_positions)
    self.move(vehicle, x, y)
    return True

  @classmethod
  def new_from_config(cls, ship: "Ship") -> "Ship":
    return cls(ship.rect.width, ship.rect.height, ship.max_capacity)

  def slot_is_occupied(self, x: int, y: int) -> bool:
    if not (0 <= x < self.rect.width and 0 <= y < self.rect.height):
      raise IndexError("Slot is outside the ship")
    return self.slots[y][x]

  def vehicle_can_fit(self, vehicle: Vehicle, x: int | None = None, y:
```



```
int | None = None) -> bool:
    x = vehicle.rect.x if x is None else x
    y = vehicle.rect.y if y is None else y
    if x < 0 or y < 0 or x + vehicle.rect.width > self.rect.width or y +
vehicle.rect.height > self.rect.height:
      return False
    for i in range(vehicle.rect.height):
      for j in range(vehicle.rect.width):
        if not self.slot_is_occupied(x + j, y + i):
          continue
        in_current_footprint = vehicle.rect.x <= x + j < vehicle.rect.x
+ vehicle.rect.width and vehicle.rect.y <= y + i < vehicle.rect.y +
vehicle.rect.height
        if vehicle not in self.vehicles or not in_current_footprint:
          return False
    return True

  def move(self, vehicle: Vehicle, x: int | None = None, y: int | None =
None):
    x = vehicle.rect.x if x is None else x
    y = vehicle.rect.y if y is None else y
    if not self.vehicle_can_fit(vehicle, x, y):
      raise ValueError("Vehicle cannot fit")
    if vehicle in self.vehicles:
      self._mark(vehicle, False)
    else:
      self.vehicles.append(vehicle)
    vehicle.rect.set_position(x, y)
    self._mark(vehicle, True)

  def orient(self, vehicle: Vehicle, orientation: str):
    if orientation not in ("horizontal", "vertical"):
      raise ValueError("Orientation must be horizontal or vertical")
    if orientation == vehicle.orientation:
      return
    if vehicle not in self.vehicles:
      vehicle.set_orientation(orientation)
      return
    x, y = vehicle.rect.x, vehicle.rect.y
    old_width, old_height = vehicle.rect.width, vehicle.rect.height
    if x + old_height > self.rect.width or y + old_width >
self.rect.height:
      raise ValueError("Vehicle cannot fit")
    for row in range(y, y + old_width):
      for col in range(x, x + old_height):
        in_old_footprint = x <= col < x + old_width and y <= row < y +
old_height
        if self.slot_is_occupied(col, row) and not in_old_footprint:
          raise ValueError("Vehicle cannot fit")
    self._mark(vehicle, False)
    vehicle.set_orientation(orientation)
    self._mark(vehicle, True)

  def _mark(self, vehicle: Vehicle, occupied: bool):
    for y in range(vehicle.rect.y, vehicle.rect.y +
vehicle.rect.height):
      for x in range(vehicle.rect.x, vehicle.rect.x +
vehicle.rect.width):
        self.slots[y][x] = occupied

  def remove(self, vehicle: Vehicle):
    if vehicle not in self.vehicles:
      raise ValueError("Vehicle not in ship")
    self._mark(vehicle, False)
    self.vehicles.remove(vehicle)
    vehicle.remove_position()

  def replace(self, old_vehicle: Vehicle, new_vehicle: Vehicle):
    if old_vehicle not in self.vehicles:
      raise ValueError("Vehicle not in ship")
    if new_vehicle in self.vehicles:
      raise ValueError("Replacement vehicle is already in ship")
    x, y = old_vehicle.rect.x, old_vehicle.rect.y
    if x + new_vehicle.rect.width > self.rect.width or y +
new_vehicle.rect.height > self.rect.height:
      raise ValueError("Vehicle cannot fit")
    for row in range(y, y + new_vehicle.rect.height):
      for col in range(x, x + new_vehicle.rect.width):
        in_old_footprint = col < x + old_vehicle.rect.width and row < y + old_vehicle.rect.height
        if self.slot_is_occupied(col, row) and not in_old_footprint:
          raise ValueError("Vehicle cannot fit")
    index = self.vehicles.index(old_vehicle)
    self._mark(old_vehicle, False)
    old_vehicle.remove_position()
    new_vehicle.rect.set_position(x, y)
    self.vehicles[index] = new_vehicle
    self._mark(new_vehicle, True)
```

Fungsi	`def __init__(self, width: int, height: int, max_capacity: int):`
Deskripsi	Menginisialisasi objek Ship dengan parameter lebar (width), tinggi (height), dan kapasitas maksimum (max_capacity), serta menyiapkan daftar kendaraan (vehicles) kosong dan matriks grid slot (slots) kapal.
Output	-

Source Code

```
def __init__(self, width: int, height: int, max_capacity: int):
    self.rect = Rect(width, height)
    self.vehicles: list[Vehicle] = []
    self.slots: list[list[bool]] = [[False for _ in range(width)] for _ in range(height)]
    self.max_capacity = max_capacity
```

Fungsi	`def total_weight(self) -> int:`
Deskripsi	Menghitung total berat keseluruhan dari seluruh kendaraan yang saat ini berada di dalam kapal dengan menjumlahkan atribut berat (v.weight)

	masing-masing kendaraan.
Output	Bilangan bulat (int) yang merepresentasikan total berat muatan.

Source Code

```
def total_weight(self) -> int:
    return sum(v.weight for v in self.vehicles)
```

Fungsi	`def can_fit_weight(self, vehicle: Vehicle) -> bool:`
Deskripsi	Memeriksa apakah kapasitas berat kapal masih mencukupi jika kendaraan baru ditambahkan (total berat saat ini ditambah berat kendaraan baru tidak melebihi max_capacity).
Output	True jika muatan masih mencukupi, False jika melebihi kapasitas.

Source Code

```
def can_fit_weight(self, vehicle: Vehicle) -> bool:
    return self.total_weight + vehicle.weight <=
self.max_capacity
```

Fungsi

```
def try_place_randomly(self, vehicle: Vehicle) ->
bool:
```

Deskripsi	Mencoba menempatkan suatu kendaraan ke dalam kapal pada posisi koordinat yang valid secara acak.
Output	True jika berhasil menempatkan kendaraan secara acak, False jika tidak ada posisi yang valid atau ukuran tidak muat.

Source Code

```
def try_place_randomly(self, vehicle: Vehicle) ->
bool:
    max_x = self.rect.width - vehicle.rect.width
    max_y = self.rect.height - vehicle.rect.height
    if max_x < 0 or max_y < 0:
      return False

    valid_positions: list[tuple[int, int]] = []
    for y in range(max_y + 1):
      for x in range(max_x + 1):
        if self.vehicle_can_fit(vehicle, x, y):
          valid_positions.append((x, y))

    if not valid_positions:
      return False

    x, y = random.choice(valid_positions)
    self.move(vehicle, x, y)
    return True
```

Fungsi	`def new_from_config(cls, ship: "Ship") -> "Ship":`
Deskripsi	Class method untuk membuat dan mengembalikan objek Ship baru berdasarkan konfigurasi ukuran dan kapasitas dari objek kapal referensi yang diberikan.
Output	Objek instansiasi baru dari kelas Ship.
Source Code	

```
def new_from_config(cls, ship: "Ship") -> "Ship":
    return cls(ship.rect.width, ship.rect.height,
ship.max_capacity)
```

Fungsi	`def slot_is_occupied(self, x: int, y: int) -> bool:`
Deskripsi	Memeriksa apakah suatu slot pada koordinat grid tertentu (x, y) sudah ditempati oleh kendaraan. Melempar IndexError jika koordinat berada di luar

	batas area kapal.
Output	True jika slot sudah terisi, False jika kosong.

Source Code

```
def slot_is_occupied(self, x: int, y: int) -> bool:
    if not (0 <= x < self.rect.width and 0 <= y < self.rect.height):
      raise IndexError("Slot is outside the ship")
    return self.slots[y][x]
```

Fungsi

```
def vehicle_can_fit(self, vehicle: Vehicle, x: int | None = None, y: int | None = None) -> bool:
```

Deskripsi	Memeriksa apakah suatu kendaraan dapat dimuat pada posisi koordinat (x, y) tertentu tanpa keluar dari batas ukuran kapal atau bertabrakan dengan kendaraan lain.
Output	True jika posisi valid dan muat, False jika sebaliknya.

Source Code

```
def vehicle_can_fit(self, vehicle: Vehicle, x: int | None = None, y: int | None = None) -> bool:
    x = vehicle.rect.x if x is None else x
    y = vehicle.rect.y if y is None else y
    if x < 0 or y < 0 or x + vehicle.rect.width > self.rect.width or y + vehicle.rect.height > self.rect.height:
      return False
    for i in range(vehicle.rect.height):
      for j in range(vehicle.rect.width):
        if not self.slot_is_occupied(x + j, y + i):
          continue
        in_current_footprint = vehicle.rect.x <= x + j < vehicle.rect.x + vehicle.rect.width and vehicle.rect.y <= y + i < vehicle.rect.y + vehicle.rect.height
        if vehicle not in self.vehicles or not
```

```
in_current_footprint:
          return False
    return True
```

Fungsi	`def move(self, vehicle: Vehicle, x: int | None = None, y: int | None = None):`
Deskripsi	Menempatkan atau memindahkan kendaraan ke posisi koordinat baru (x, y) di dalam kapal. Jika kendaraan belum ada di dalam kapal, maka kendaraan akan ditambahkan ke daftar muatan. Melempar ValueError jika posisi tidak valid.
Output	-

Source Code

```
def move(self, vehicle: Vehicle, x: int | None = None, y: int | None = None):
    x = vehicle.rect.x if x is None else x
    y = vehicle.rect.y if y is None else y
    if not self.vehicle_can_fit(vehicle, x, y):
      raise ValueError("Vehicle cannot fit")
    if vehicle in self.vehicles:
      self._mark(vehicle, False)
    else:
      self.vehicles.append(vehicle)
    vehicle.rect.set_position(x, y)
    self._mark(vehicle, True)
```

Fungsi	`def orient(self, vehicle: Vehicle, orientation: str):`
Deskripsi	Mengubah orientasi kendaraan dan memperbarui area okupansi pada grid kapal, dengan memastikan

	kendaraan tetap muat setelah dirotasi. Melempar ValueError jika orientasi tidak valid atau ukuran melebihi batas.
Output	**-**
Source Code	

```
def orient(self, vehicle: Vehicle, orientation: str):
    if orientation not in ("horizontal", "vertical"):
      raise ValueError("Orientation must be horizontal or vertical")
    if orientation == vehicle.orientation:
      return
    if vehicle not in self.vehicles:
      vehicle.set_orientation(orientation)
      return
    x, y = vehicle.rect.x, vehicle.rect.y
    old_width, old_height = vehicle.rect.width, vehicle.rect.height
    if x + old_height > self.rect.width or y + old_width > self.rect.height:
      raise ValueError("Vehicle cannot fit")
    for row in range(y, y + old_width):
      for col in range(x, x + old_height):
        in_old_footprint = x <= col < x + old_width and y <= row < y + old_height
        if self.slot_is_occupied(col, row) and not in_old_footprint:
          raise ValueError("Vehicle cannot fit")
    self._mark(vehicle, False)
    vehicle.set_orientation(orientation)
    self._mark(vehicle, True)
```

Fungsi	`def _mark(self, vehicle: Vehicle, occupied: bool):`
Deskripsi	Fungsi internal (helper method) untuk menandai

	atau mengubah status okupansi slot pada grid slots di seluruh jangkauan area kendaraan menjadi terisi (True) atau kosong (False).
Output	-

Source Code

```
def _mark(self, vehicle: Vehicle, occupied: bool):
    for y in range(vehicle.rect.y, vehicle.rect.y + vehicle.rect.height):
      for x in range(vehicle.rect.x, vehicle.rect.x + vehicle.rect.width):
        self.slots[y][x] = occupied
```

Fungsi	`def remove(self, vehicle: Vehicle):`
Deskripsi	Menghapus kendaraan dari daftar muatan kapal, mengosongkan kembali slot grid yang sebelumnya ditempati, serta mereset posisi kendaraan tersebut. Melempar ValueError jika kendaraan tidak ditemukan di dalam kapal.
Output	-

Source Code

```
def remove(self, vehicle: Vehicle):
    if vehicle not in self.vehicles:
      raise ValueError("Vehicle not in ship")
    self._mark(vehicle, False)
    self.vehicles.remove(vehicle)
    vehicle.remove_position()
```

Fungsi	`def replace(self, old_vehicle: Vehicle, new_vehicle: Vehicle):`
Deskripsi	Mengganti posisi kendaraan lama (old_vehicle)

	dengan kendaraan baru (new_vehicle) pada titik koordinat yang sama di dalam kapal, dengan memastikan kendaraan pengganti muat dan tidak tumpang tindih. Melempar ValueError jika validasi gagal.
Output	**-**
Source Code	

```
  def replace(self, old_vehicle: Vehicle, new_vehicle: Vehicle):
    if old_vehicle not in self.vehicles:
      raise ValueError("Vehicle not in ship")
    if new_vehicle in self.vehicles:
      raise ValueError("Replacement vehicle is already in ship")
    x, y = old_vehicle.rect.x, old_vehicle.rect.y
    if x + new_vehicle.rect.width > self.rect.width or y + new_vehicle.rect.height > self.rect.height:
      raise ValueError("Vehicle cannot fit")
    for row in range(y, y + new_vehicle.rect.height):
      for col in range(x, x + new_vehicle.rect.width):
        in_old_footprint = col < x + old_vehicle.rect.width and row < y + old_vehicle.rect.height
        if self.slot_is_occupied(col, row) and not in_old_footprint:
          raise ValueError("Vehicle cannot fit")
    index = self.vehicles.index(old_vehicle)
    self._mark(old_vehicle, False)
    old_vehicle.remove_position()
    new_vehicle.rect.set_position(x, y)
    self.vehicles[index] = new_vehicle
    self._mark(new_vehicle, True)
```

2.2.4. Implementasi Vehicle

vehicle.py

```python
from objects.rect import Rect

TRUCK_ID = "truck"
MOTORCYCLE_ID = "motorcycle"
CAR_ID = "car"
QUADCOPTER_ID = "quadcopter"

class Vehicle:
  def __init__(self, id: str, width: int, height: int, weight: int, shipping_fee: int, eta: int):
    self.id = id
    self.rect = Rect(width, height)
    self.orientation = "horizontal" if width >= height else "vertical"
    self.remove_position()
    self.weight = weight
    self.shipping_fee = shipping_fee
    self.eta = eta

  def remove_position(self):
    self.rect.set_position(-1, -1)

  def set_orientation(self, orientation: str):
    if orientation not in ("horizontal", "vertical"):
      raise ValueError("Orientation must be horizontal or vertical")
    if orientation != self.orientation:
      self.rect.width, self.rect.height = self.rect.height, self.rect.width
      self.orientation = orientation

class VehicleBuilder:
  counter: dict[str, int] = {}

  def truck(self) -> Vehicle:
    self.counter[TRUCK_ID] = self.counter.get(TRUCK_ID, 0) + 1
    return Vehicle(id=f"{TRUCK_ID}-{self.counter[TRUCK_ID]}", width=1, height=3, weight=1000, shipping_fee=500, eta=2)
  def motorcycle(self) -> Vehicle:
    self.counter[MOTORCYCLE_ID] = self.counter.get(MOTORCYCLE_ID, 0) + 1
    return Vehicle(id=f"{MOTORCYCLE_ID}-{self.counter[MOTORCYCLE_ID]}",
width=1, height=1, weight=20, shipping_fee=50, eta=2)

  def car(self) -> Vehicle:
    self.counter[CAR_ID] = self.counter.get(CAR_ID, 0) + 1
    return Vehicle(id=f"{CAR_ID}-{self.counter[CAR_ID]}", width=1,
height=2, weight=500, shipping_fee=100, eta=3)

  def quadcopter(self) -> Vehicle:
    self.counter[QUADCOPTER_ID] = self.counter.get(QUADCOPTER_ID, 0) + 1
    return Vehicle(id=f"{QUADCOPTER_ID}-{self.counter[QUADCOPTER_ID]}",
width=2, height=2, weight=5, shipping_fee=200, eta=1)
```

Fungsi	`def __init__(self, id: str, width: int, height: int, weight: int, shipping_fee: int, eta: int):`
Deskripsi	Menginisialisasi objek Vehicle baru dengan parameter ID, lebar (width), tinggi (height), berat (weight), biaya pengiriman (shipping_fee), dan estimasi waktu tiba (eta), serta mengatur orientasi awal dan mereset posisinya.
Output	**-**
Source Code	

```
def __init__(self, id: str, width: int, height: int,
weight: int, shipping_fee: int, eta: int):
    self.id = id
    self.rect = Rect(width, height)
    self.orientation = "horizontal" if width >=
height else "vertical"
    self.remove_position()
    self.weight = weight
    self.shipping_fee = shipping_fee
    self.eta = eta
```

Fungsi	`def remove_position(self):`
Deskripsi	Mengatur posisi koordinat objek kendaraan pada grid rectangle ke nilai di luar area valid yaitu (-1, -1).
Output	-

Source Code

```
def remove_position(self):
    self.rect.set_position(-1, -1)
```

Fungsi	`def set_orientation(self, orientation: str):`
Deskripsi	Mengubah orientasi kendaraan menjadi "horizontal" atau "vertical". Jika orientasi baru berbeda dengan orientasi sebelumnya, ukuran lebar dan tinggi rectangle kendaraan akan ditukar (swap). Melempar ValueError jika nilai orientasi tidak valid.
Output	-

Source Code

```
def set_orientation(self, orientation: str):
    if orientation not in ("horizontal", "vertical"):
      raise ValueError("Orientation must be horizontal or vertical")
    if orientation != self.orientation:
      self.rect.width, self.rect.height = self.rect.height, self.rect.width
      self.orientation = orientation
```

Fungsi	`def truck(self) -> Vehicle:`
Deskripsi	Menambah penghitung ID truk, lalu membuat dan mengembalikan objek Vehicle baru bertipe truk dengan spesifikasi ukuran, berat, biaya, dan ETA tertentu.
Output	Objek instance dari kelas Vehicle.

Source Code

```python
def truck(self) -> Vehicle:
    self.counter[TRUCK_ID] = self.counter.get(TRUCK_ID, 0) + 1
    return Vehicle(id=f"{TRUCK_ID}-{self.counter[TRUCK_ID]}", width=1, height=3, weight=1000, shipping_fee=500, eta=2)
```

Fungsi	`def motorcycle(self) -> Vehicle:`
Deskripsi	Menambah penghitung ID sepeda motor, lalu membuat dan mengembalikan objek Vehicle baru bertipe motor dengan spesifikasi ukuran, berat, biaya, dan ETA tertentu.
Output	Objek instance dari kelas Vehicle.

Source Code

```python
def motorcycle(self) -> Vehicle:
    self.counter[MOTORCYCLE_ID] = self.counter.get(MOTORCYCLE_ID, 0) + 1
    return Vehicle(id=f"{MOTORCYCLE_ID}-{self.counter[MOTORCYCLE_ID]}", width=1, height=1, weight=20, shipping_fee=50, eta=2)
```

Fungsi	`def car(self) -> Vehicle:`
Deskripsi	Menambah penghitung ID mobil, lalu membuat dan mengembalikan objek Vehicle baru bertipe mobil dengan spesifikasi ukuran, berat, biaya, dan ETA tertentu.
Output	Objek instance dari kelas Vehicle.
Source Code	

```python
def car(self) -> Vehicle:
    self.counter[CAR_ID] = self.counter.get(CAR_ID, 0) + 1
    return Vehicle(id=f"{CAR_ID}-{self.counter[CAR_ID]}", width=1, height=2, weight=500, shipping_fee=100, eta=3)
```

Fungsi	`def quadcopter(self) -> Vehicle:`
Deskripsi	Menambah penghitung ID quadcopter, lalu membuat dan mengembalikan objek Vehicle baru bertipe quadcopter dengan spesifikasi ukuran, berat, biaya, dan ETA tertentu.
Output	Objek instance dari kelas Vehicle.
Source Code	

```python
  def quadcopter(self) -> Vehicle:
    self.counter[QUADCOPTER_ID] = self.counter.get(QUADCOPTER_ID, 0) + 1
    return Vehicle(id=f"{QUADCOPTER_ID}-{self.counter[QUADCOPTER_ID]}", width=2, height=2, weight=5, shipping_fee=200, eta=1)
```

2.2.5. Implementasi Rectangle

rect.py

```python
class Rect:
  def __init__(self, width: int, height: int):
    if width <= 0 or height <= 0:
      raise ValueError()
    self.width = width
    self.height = height
    self.x = -1
    self.y = -1

  def set_position(self, x: int, y: int):
    self.x = x
    self.y = y
```

Fungsi	`def __init__(self, width: int, height: int):`
Deskripsi	Menginisialisasi objek Rect dengan parameter lebar (width) dan tinggi (height). Melakukan validasi apabila nilai lebar atau tinggi kurang dari atau sama dengan 0, maka akan melempar ValueError. Jika valid, atribut ukuran akan disetel dan posisi koordinat awal (self.x, self.y) diatur ke -1.
Output	-
Source Code	

```python
  def __init__(self, width: int, height: int):
    if width <= 0 or height <= 0:
      raise ValueError()
    self.width = width
    self.height = height
    self.x = -1
    self.y = -1
```

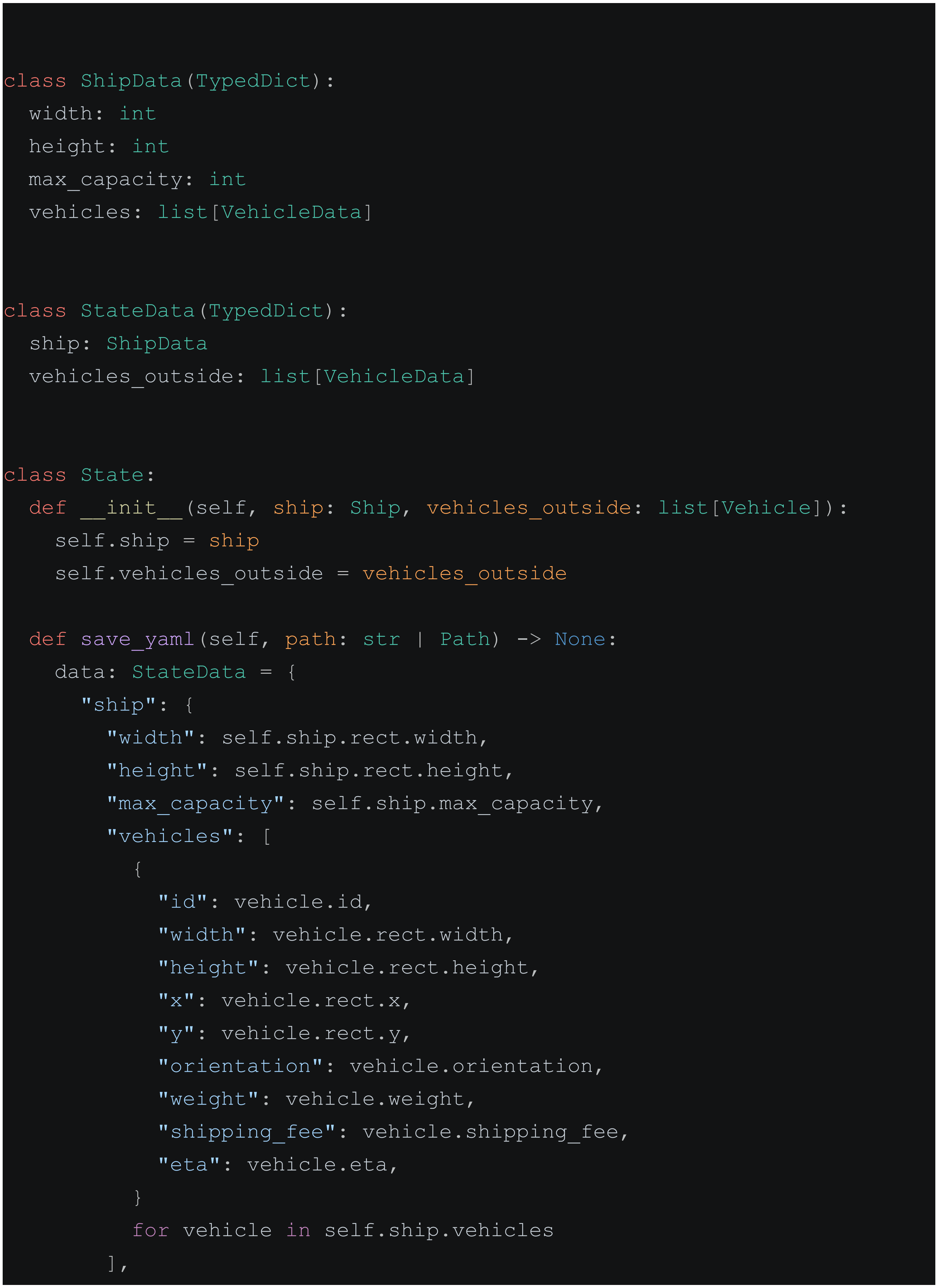
Fungsi	`def set_position(self, x: int, y: int):`
Deskripsi	Mengatur atau memperbarui posisi koordinat rectangle pada grid berdasarkan nilai parameter koordinat horizontal (x) dan vertikal (y) yang diberikan.
Output	-
Source Code	

```python
def set_position(self, x: int, y: int):
    self.x = x
    self.y = y
```

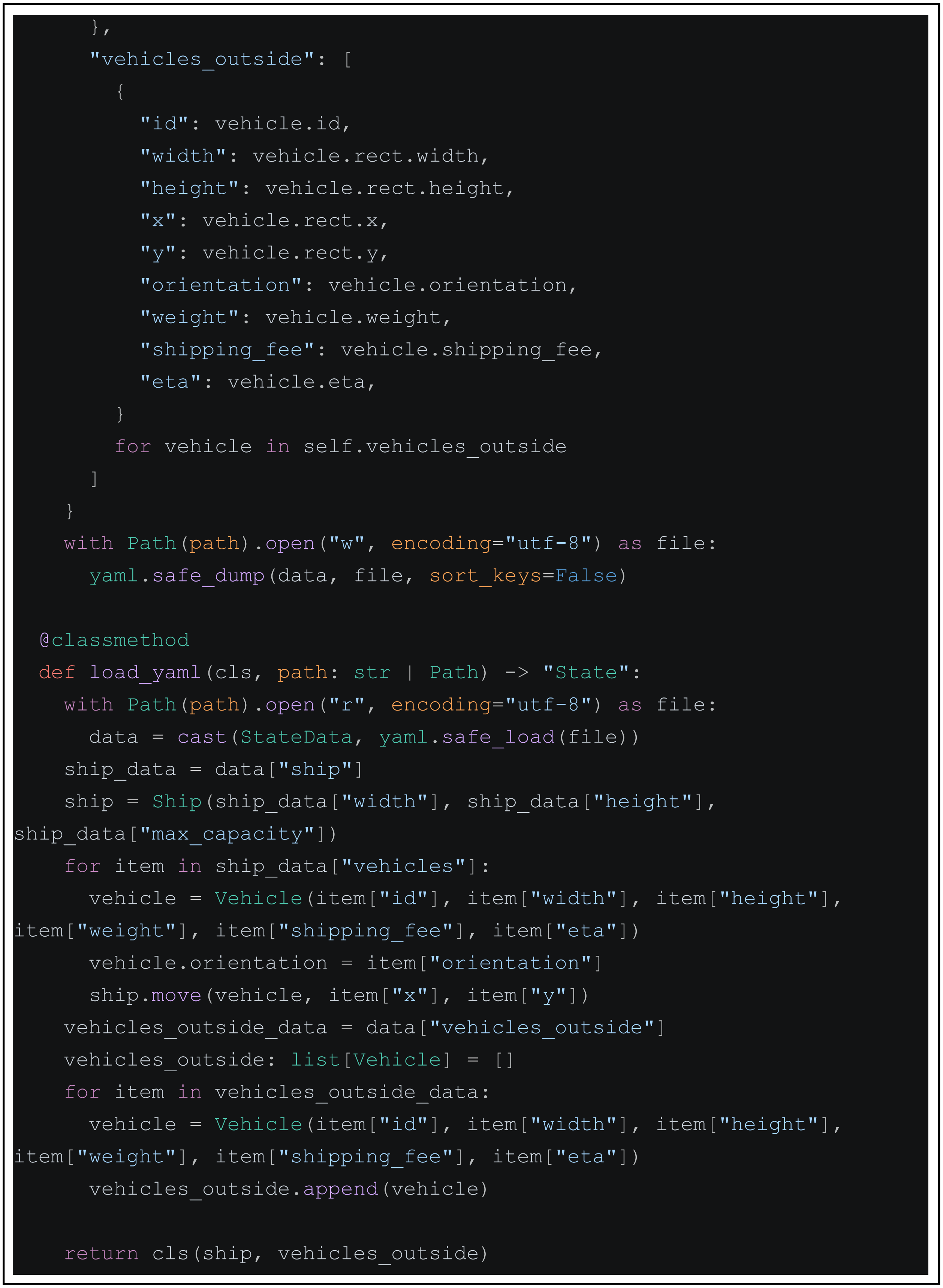
2.2.6. Implementasi State

state.py

```python
import random
from copy import deepcopy
from pathlib import Path
from typing import TypedDict, cast

import yaml

from objects.ship import Ship
from objects.vehicle import Vehicle


class VehicleData(TypedDict):
  id: str
  width: int
  height: int
  x: int
  y: int
  orientation: str
  weight: int
  shipping_fee: int
  eta: int
```



```
class ShipData(TypedDict):
  width: int
  height: int
  max_capacity: int
  vehicles: list[VehicleData]


class StateData(TypedDict):
  ship: ShipData
  vehicles_outside: list[VehicleData]


class State:
  def __init__(self, ship: Ship, vehicles_outside: list[Vehicle]):
    self.ship = ship
    self.vehicles_outside = vehicles_outside

  def save_yaml(self, path: str | Path) -> None:
    data: StateData = {
      "ship": {
        "width": self.ship.rect.width,
        "height": self.ship.rect.height,
        "max_capacity": self.ship.max_capacity,
        "vehicles": [
          {
            "id": vehicle.id,
            "width": vehicle.rect.width,
            "height": vehicle.rect.height,
            "x": vehicle.rect.x,
            "y": vehicle.rect.y,
            "orientation": vehicle.orientation,
            "weight": vehicle.weight,
            "shipping_fee": vehicle.shipping_fee,
            "eta": vehicle.eta,
          }
          for vehicle in self.ship.vehicles
        ],
```



```
      },
      "vehicles_outside": [
        {
          "id": vehicle.id,
          "width": vehicle.rect.width,
          "height": vehicle.rect.height,
          "x": vehicle.rect.x,
          "y": vehicle.rect.y,
          "orientation": vehicle.orientation,
          "weight": vehicle.weight,
          "shipping_fee": vehicle.shipping_fee,
          "eta": vehicle.eta,
        }
        for vehicle in self.vehicles_outside
      ]
    }
    with Path(path).open("w", encoding="utf-8") as file:
      yaml.safe_dump(data, file, sort_keys=False)

  @classmethod
  def load_yaml(cls, path: str | Path) -> "State":
    with Path(path).open("r", encoding="utf-8") as file:
      data = cast(StateData, yaml.safe_load(file))
    ship_data = data["ship"]
    ship = Ship(ship_data["width"], ship_data["height"],
ship_data["max_capacity"])
    for item in ship_data["vehicles"]:
      vehicle = Vehicle(item["id"], item["width"], item["height"],
item["weight"], item["shipping_fee"], item["eta"])
      vehicle.orientation = item["orientation"]
      ship.move(vehicle, item["x"], item["y"])
    vehicles_outside_data = data["vehicles_outside"]
    vehicles_outside: list[Vehicle] = []
    for item in vehicles_outside_data:
      vehicle = Vehicle(item["id"], item["width"], item["height"],
item["weight"], item["shipping_fee"], item["eta"])
      vehicles_outside.append(vehicle)

    return cls(ship, vehicles_outside)
  def get_objective(self) -> int:
    return sum(vehicle.shipping_fee for vehicle in self.ship.vehicles)

class StateFactory:
  def __init__(self, ship: Ship, vehicles: list[Vehicle]) -> None:
    self.ship = Ship.new_from_config(ship)
    self.vehicles = deepcopy(vehicles)

  def create_state(self) -> State:
    return State(Ship.new_from_config(self.ship),
deepcopy(self.vehicles))

  def create_random_state(self) -> State:
      state = self.create_state()
      candidates = state.vehicles_outside[:]
      random.shuffle(candidates)
      grid_width = state.ship.rect.width
      grid_height = state.ship.rect.height

      for vehicle in candidates:
          if random.random() > 0.5:
              continue
          orientation = random.choice(["horizontal", "vertical"])
          rx = random.randint(0, grid_width - 1)
          ry = random.randint(0, grid_height - 1)
          vehicle.set_orientation(orientation)
          loaded_weight = sum(v.weight for v in state.ship.vehicles)
          if loaded_weight + vehicle.weight <= state.ship.max_capacity:
              if state.ship.vehicle_can_fit(vehicle, rx, ry):
                  state.ship.move(vehicle, rx, ry)
                  state.vehicles_outside.remove(vehicle)
      return state
```

Fungsi

```
def __init__(self, ship: Ship, vehicles_outside:
list[Vehicle]):
```

Deskripsi	Menginisialisasi objek State dengan parameter objek kapal (ship) dan daftar kendaraan yang berada di luar kapal (vehicles_outside).
Output	Menginisialisasi instance objek State.

Source Code

```python
def __init__(self, ship: Ship, vehicles_outside: list[Vehicle]):
    self.ship = ship
    self.vehicles_outside = vehicles_outside
```

Fungsi	`def save_yaml(self, path: str | Path) -> None:`
Deskripsi	Menyimpan data kondisi saat ini (konfigurasi kapal, daftar kendaraan di dalam kapal beserta posisinya, dan kendaraan di luar kapal) ke dalam sebuah file berformat YAML pada jalur (path) yang ditentukan.
Output	-

Source Code

```python
def save_yaml(self, path: str | Path) -> None:
    data: StateData = {
      "ship": {
        "width": self.ship.rect.width,
        "height": self.ship.rect.height,
        "max_capacity": self.ship.max_capacity,
        "vehicles": [
          {
            "id": vehicle.id,
            "width": vehicle.rect.width,
            "height": vehicle.rect.height,
            "x": vehicle.rect.x,
            "y": vehicle.rect.y,
            "orientation": vehicle.orientation,
            "weight": vehicle.weight,
            "shipping_fee": vehicle.shipping_fee,
            "eta": vehicle.eta,
          }
          for vehicle in self.ship.vehicles
        ],
      },
      "vehicles_outside": [
        {
          "id": vehicle.id,
          "width": vehicle.rect.width,
          "height": vehicle.rect.height,
          "x": vehicle.rect.x,
          "y": vehicle.rect.y,
          "orientation": vehicle.orientation,
          "weight": vehicle.weight,
          "shipping_fee": vehicle.shipping_fee,
          "eta": vehicle.eta,
        }
        for vehicle in self.vehicles_outside
      ]
    }
    with Path(path).open("w", encoding="utf-8") as
file:
      yaml.safe_dump(data, file, sort_keys=False)
```

Fungsi	`def load_yaml(cls, path: str | Path) -> "State":`
Deskripsi	Membaca data dari file YAML yang diberikan, lalu memuat kembali dan mengembalikan objek State beserta konfigurasi kapal dan penempatan kendaraan di dalamnya.
Output	Objek instansiasi dari kelas State.

Source Code

```
def load_yaml(cls, path: str | Path) -> "State":
    with Path(path).open("r", encoding="utf-8") as
file:
      data = cast(StateData, yaml.safe_load(file))
    ship_data = data["ship"]
    ship = Ship(ship_data["width"],
ship_data["height"], ship_data["max_capacity"])
    for item in ship_data["vehicles"]:
      vehicle = Vehicle(item["id"], item["width"],
item["height"], item["weight"],
item["shipping_fee"], item["eta"])
      vehicle.orientation = item["orientation"]
      ship.move(vehicle, item["x"], item["y"])
    vehicles_outside_data = data["vehicles_outside"]
    vehicles_outside: list[Vehicle] = []
    for item in vehicles_outside_data:
      vehicle = Vehicle(item["id"], item["width"],
item["height"], item["weight"],
item["shipping_fee"], item["eta"])
      vehicles_outside.append(vehicle)

    return cls(ship, vehicles_outside)
```

Fungsi	`def get_objective(self) -> int:`
Deskripsi	Menghitung total nilai objektif (fitness value) dari state saat ini dengan menjumlahkan biaya pengiriman (shipping_fee) dari seluruh kendaraan yang berhasil dimuat ke dalam kapal.
Output	Bilangan bulat (int) yang merepresentasikan total nilai objektif/biaya muatan.

Source Code

```
def get_objective(self) -> int:
    return sum(vehicle.shipping_fee for vehicle in
self.ship.vehicles)
```

Fungsi

```
def __init__(self, ship: Ship, vehicles:
list[Vehicle]) -> None:
```

Deskripsi	Menginisialisasi factory dengan menyalin konfigurasi dasar kapal (ship) menggunakan metode new_from_config dan menduplikasi daftar kendaraan (vehicles) secara mendalam (deepcopy).
Output	Menginisialisasi instance objek StateFactory.
Source Code	

```python
def __init__(self, ship: Ship, vehicles: list[Vehicle]) -> None:
    self.ship = Ship.new_from_config(ship)
    self.vehicles = deepcopy(vehicles)
```

Fungsi	`def create_state(self) -> State:`
Deskripsi	Membuat dan mengembalikan objek State baru dengan menyalin ulang kondisi awal kapal dan daftar kendaraan tanpa mengubah posisinya.
Output	Objek instance baru dari kelas State.
Source Code	

```python
def create_state(self) -> State:
    return State(Ship.new_from_config(self.ship), deepcopy(self.vehicles))
```

Fungsi	`def create_random_state(self) -> State:`
Deskripsi	Membuat objek State secara acak (random state), di mana kendaraan di luar diacak urutannya, lalu secara probabilistik ditempatkan ke dalam grid kapal dengan orientasi, koordinat, serta pengecekan batas kapasitas berat dan ukuran yang valid.
Output	Objek instance dari kelas State dengan konfigurasi penempatan kendaraan acak.

Source Code

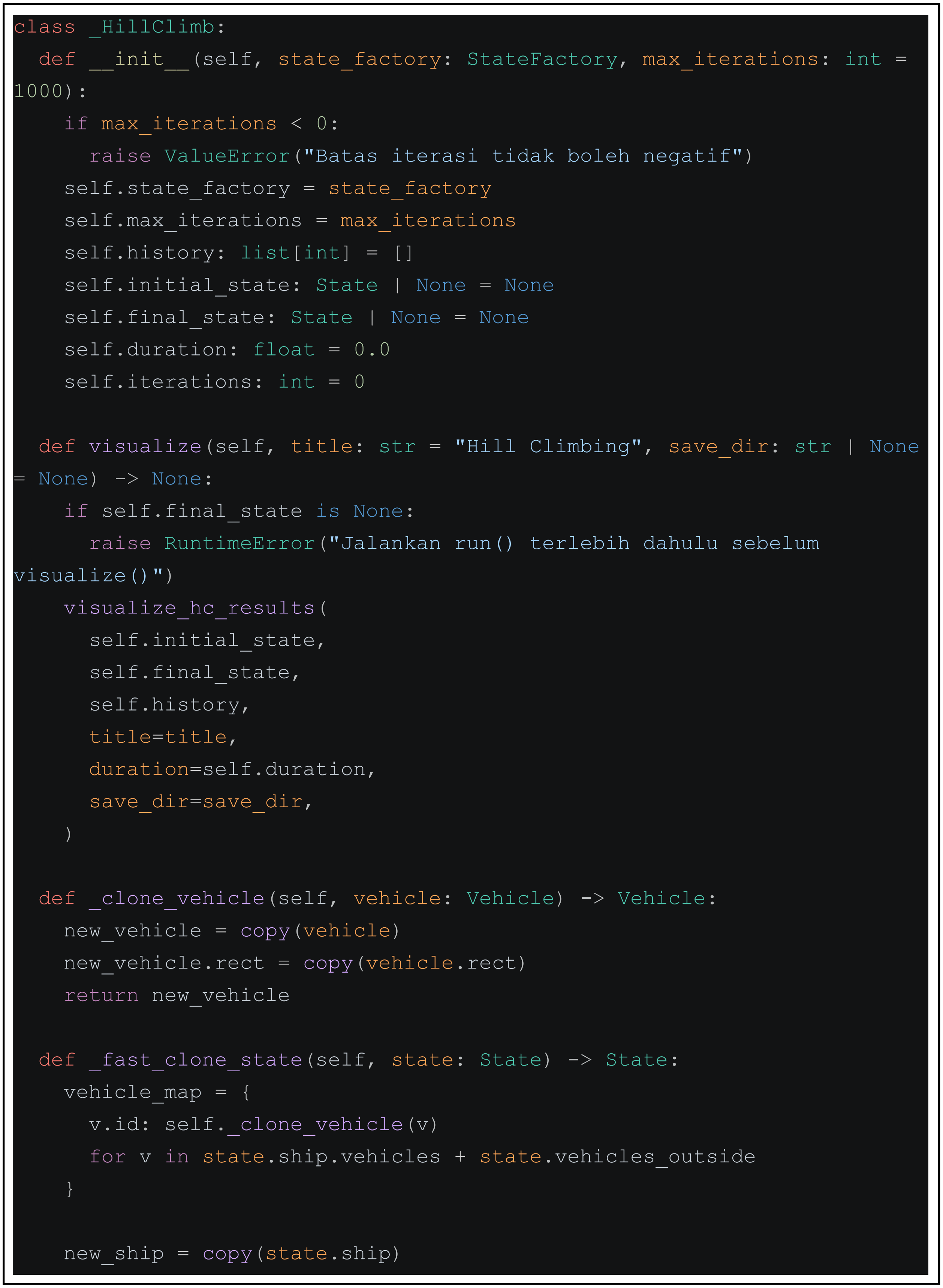
```
def create_random_state(self) -> State:
    state = self.create_state()
    candidates = state.vehicles_outside[:]
    random.shuffle(candidates)
    grid_width = state.ship.rect.width
    grid_height = state.ship.rect.height

    for vehicle in candidates:
        if random.random() > 0.5:
            continue
        orientation = random.choice(["horizontal", "vertical"])
        rx = random.randint(0, grid_width - 1)
        ry = random.randint(0, grid_height - 1)
        vehicle.set_orientation(orientation)
        loaded_weight = sum(v.weight for v in state.ship.vehicles)
        if loaded_weight + vehicle.weight <= state.ship.max_capacity:
            if state.ship.vehicle_can_fit(vehicle, rx, ry):
                state.ship.move(vehicle, rx, ry)
                state.vehicles_outside.remove(vehicle)
    return state
```

2.2. Implementasi Algoritma Local Search

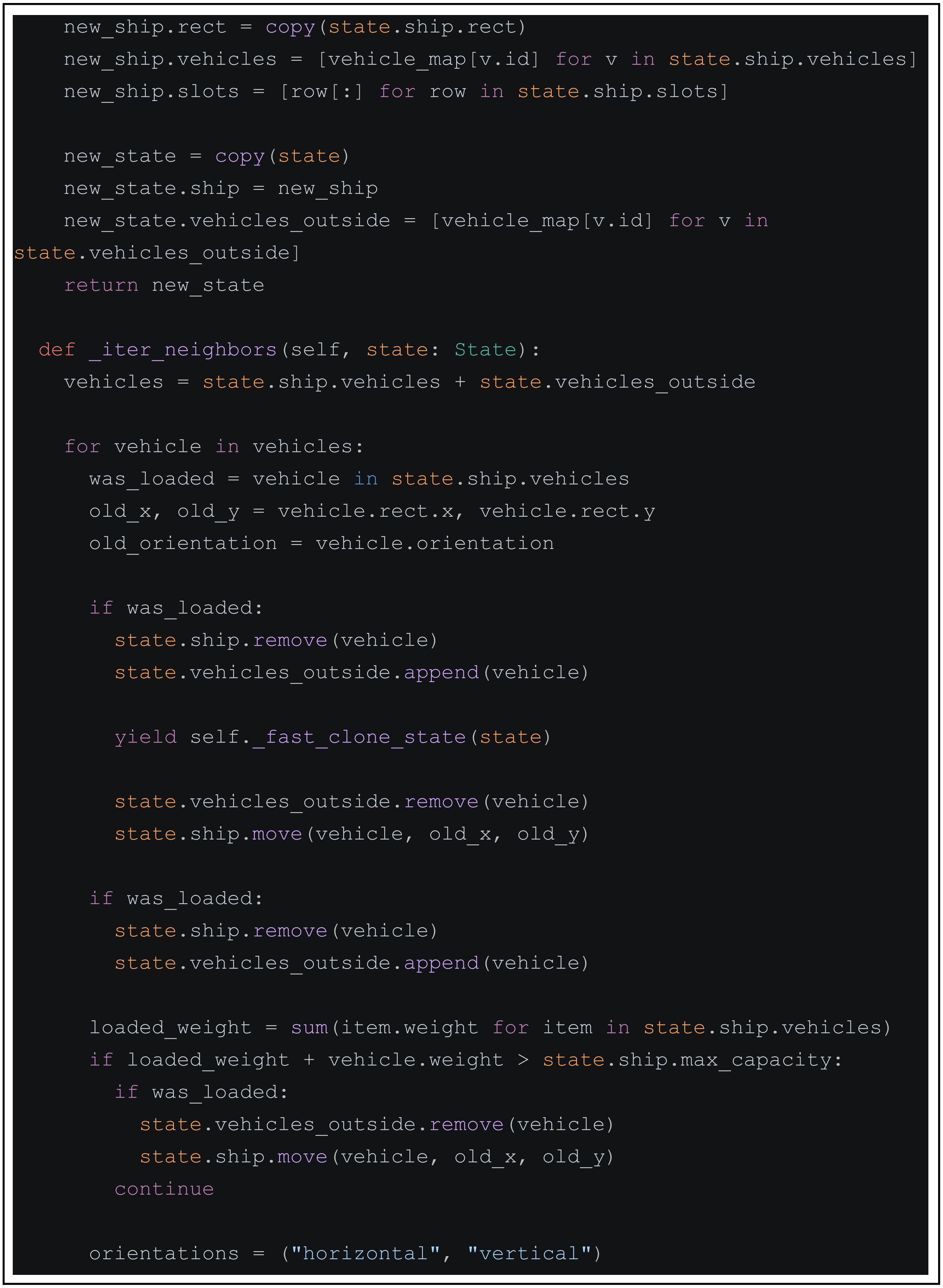
2.2.1. Algoritma *Steepest Ascent Hill-Climbing*

Steepest_Ascend_HC.py

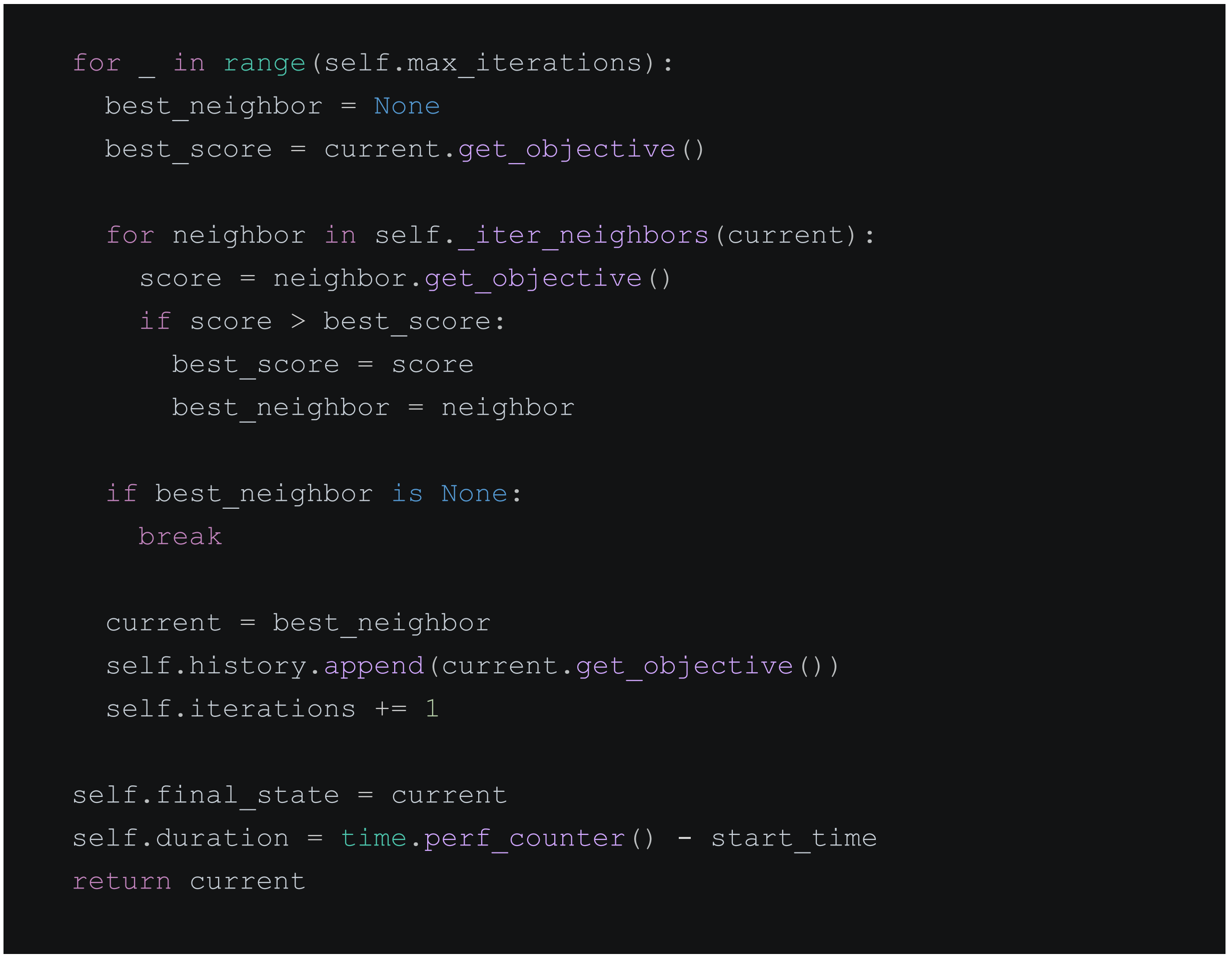
```
import time
from copy import copy
from objects.state import State, StateFactory
from objects.vehicle import Vehicle
from visualizations.ship import visualize_hc_results
```



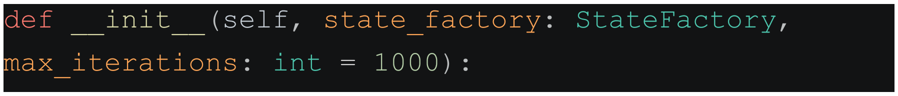
```python
class _HillClimb:
  def __init__(self, state_factory: StateFactory, max_iterations: int = 1000):
    if max_iterations < 0:
      raise ValueError("Batas iterasi tidak boleh negatif")
    self.state_factory = state_factory
    self.max_iterations = max_iterations
    self.history: list[int] = []
    self.initial_state: State | None = None
    self.final_state: State | None = None
    self.duration: float = 0.0
    self.iterations: int = 0

  def visualize(self, title: str = "Hill Climbing", save_dir: str | None = None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih dahulu sebelum visualize()")
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
    )

  def _clone_vehicle(self, vehicle: Vehicle) -> Vehicle:
    new_vehicle = copy(vehicle)
    new_vehicle.rect = copy(vehicle.rect)
    return new_vehicle

  def _fast_clone_state(self, state: State) -> State:
    vehicle_map = {
      v.id: self._clone_vehicle(v)
      for v in state.ship.vehicles + state.vehicles_outside
    }

    new_ship = copy(state.ship)
```



```
    new_ship.rect = copy(state.ship.rect)
    new_ship.vehicles = [vehicle_map[v.id] for v in state.ship.vehicles]
    new_ship.slots = [row[:] for row in state.ship.slots]

    new_state = copy(state)
    new_state.ship = new_ship
    new_state.vehicles_outside = [vehicle_map[v.id] for v in
state.vehicles_outside]
    return new_state

  def _iter_neighbors(self, state: State):
    vehicles = state.ship.vehicles + state.vehicles_outside

    for vehicle in vehicles:
      was_loaded = vehicle in state.ship.vehicles
      old_x, old_y = vehicle.rect.x, vehicle.rect.y
      old_orientation = vehicle.orientation

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

        yield self._fast_clone_state(state)

        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

      loaded_weight = sum(item.weight for item in state.ship.vehicles)
      if loaded_weight + vehicle.weight > state.ship.max_capacity:
        if was_loaded:
          state.vehicles_outside.remove(vehicle)
          state.ship.move(vehicle, old_x, old_y)
        continue

      orientations = ("horizontal", "vertical")
      if vehicle.rect.width == vehicle.rect.height:
        orientations = (vehicle.orientation,)

      for orientation in orientations:
        vehicle.set_orientation(orientation)
        for y in range(state.ship.rect.height):
          for x in range(state.ship.rect.width):
            if was_loaded and (x, y) == (old_x, old_y) and orientation
== old_orientation:
              continue
            if not state.ship.vehicle_can_fit(vehicle, x, y):
              continue

            state.vehicles_outside.remove(vehicle)
            state.ship.move(vehicle, x, y)

            yield self._fast_clone_state(state)

            state.ship.remove(vehicle)
            state.vehicles_outside.append(vehicle)

      vehicle.set_orientation(old_orientation)
      if was_loaded:
        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)


class SteepestAscentHillClimb(_HillClimb):
  def visualize(self, title: str = "Steepest Ascent Hill Climbing",
save_dir: str | None = None) -> None:
    super().visualize(title=title, save_dir=save_dir)

  def run(self, initial_state: State | None = None) -> State:
    start_time = time.perf_counter()
    current = initial_state if initial_state is not None else
self.state_factory.create_random_state()
    self.initial_state = current
    self.history = [current.get_objective()]
    self.iterations = 0
```



```
for _ in range(self.max_iterations):
  best_neighbor = None
  best_score = current.get_objective()

  for neighbor in self._iter_neighbors(current):
    score = neighbor.get_objective()
    if score > best_score:
      best_score = score
      best_neighbor = neighbor

  if best_neighbor is None:
    break

  current = best_neighbor
  self.history.append(current.get_objective())
  self.iterations += 1

self.final_state = current
self.duration = time.perf_counter() - start_time
return current
```

Fungsi	 `def __init__(self, state_factory: StateFactory,` `max_iterations: int = 1000):`
Deskripsi	Menginisialisasi objek pencarian Hill Climbing dengan parameter state factory dan batas maksimum iterasi (max_iterations). Melakukan validasi apabila jumlah iterasi bernilai negatif, maka akan melempar ValueError. Sekaligus menyiapkan atribut riwayat, status awal, status akhir, durasi, dan counter iterasi.
Output	**-**

Source Code

```python
def __init__(self, state_factory: StateFactory,
max_iterations: int = 1000):
    if max_iterations < 0:
      raise ValueError("Batas iterasi tidak boleh
negatif")
    self.state_factory = state_factory
    self.max_iterations = max_iterations
    self.history: list[int] = []
    self.initial_state: State | None = None
    self.final_state: State | None = None
    self.duration: float = 0.0
    self.iterations: int = 0
```

Fungsi

```python
def visualize(self, title: str = "Hill Climbing",
save_dir: str | None = None) -> None:
```

Deskripsi	Memeriksa apakah proses pencarian telah dijalankan (final_state tidak boleh None). Jika belum, akan melempar RuntimeError. Jika sudah, memanggil fungsi visualisasi hasil pencarian dengan menyertakan status awal, akhir, riwayat skor, judul, durasi, dan direktori penyimpanan.
Output	Menampilkan atau menyimpan visualisasi grafik dan hasil penataan kapal.

Source Code

```python
def visualize(self, title: str = "Hill Climbing",
save_dir: str | None = None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih
dahulu sebelum visualize()")
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
    )
```

Fungsi	`def _clone_vehicle(self, vehicle: Vehicle) -> Vehicle:`
Deskripsi	Membuat duplikat salinan dari suatu objek Vehicle beserta atribut rectangle di dalamnya secara mandiri untuk menghindari mutasi referensi asli.
Output	Objek instance Vehicle yang baru (cloned vehicle).

Source Code

```
def _clone_vehicle(self, vehicle: Vehicle) ->
Vehicle:
    new_vehicle = copy(vehicle)
    new_vehicle.rect = copy(vehicle.rect)
    return new_vehicle
```

Fungsi	`def _fast_clone_state(self, state: State) -> State:`
Deskripsi	Melakukan kloning atau penyalinan objek State secara cepat dengan menduplikasi kapal, matriks grid slot, serta seluruh kendaraan baik yang ada di dalam maupun di luar kapal menggunakan peta ID kendaraan.
Output	Objek instance State baru yang merupakan salinan mendalam (deep copy).

Source Code

```
def _fast_clone_state(self, state: State) -> State:
    vehicle_map = {
      v.id: self._clone_vehicle(v)
      for v in state.ship.vehicles +
state.vehicles_outside
```

```
    }

    new_ship = copy(state.ship)
    new_ship.rect = copy(state.ship.rect)
    new_ship.vehicles = [vehicle_map[v.id] for v in state.ship.vehicles]
    new_ship.slots = [row[:] for row in state.ship.slots]

    new_state = copy(state)
    new_state.ship = new_ship
    new_state.vehicles_outside = [vehicle_map[v.id] for v in state.vehicles_outside]
    return new_state
```

Fungsi	`def _iter_neighbors(self, state: State):`
Deskripsi	Menghasilkan (yield) tetangga-tetangga (neighbors) yang valid secara iteratif dari suatu state dengan mencoba mengeluarkan kendaraan, mengubah orientasi, atau memindahkan posisi kendaraan ke koordinat grid lain yang memenuhi syarat kapasitas dan batas ukuran kapal.
Output	Menghasilkan aliran objek State tetangga satu per satu.

Source Code

```
def _iter_neighbors(self, state: State):
    vehicles = state.ship.vehicles + state.vehicles_outside

    for vehicle in vehicles:
      was_loaded = vehicle in state.ship.vehicles
      old_x, old_y = vehicle.rect.x, vehicle.rect.y
      old_orientation = vehicle.orientation
      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

        yield self._fast_clone_state(state)

        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

      loaded_weight = sum(item.weight for item in state.ship.vehicles)
      if loaded_weight + vehicle.weight > state.ship.max_capacity:
        if was_loaded:
          state.vehicles_outside.remove(vehicle)
          state.ship.move(vehicle, old_x, old_y)
        continue

      orientations = ("horizontal", "vertical")
      if vehicle.rect.width == vehicle.rect.height:
        orientations = (vehicle.orientation,)

      for orientation in orientations:
        vehicle.set_orientation(orientation)
        for y in range(state.ship.rect.height):
          for x in range(state.ship.rect.width):
            if was_loaded and (x, y) == (old_x, old_y) and orientation == old_orientation:
              continue
            if not state.ship.vehicle_can_fit(vehicle, x, y):
              continue

            state.vehicles_outside.remove(vehicle)
            state.ship.move(vehicle, x, y)
```

```python
            yield self._fast_clone_state(state)

            state.ship.remove(vehicle)
            state.vehicles_outside.append(vehicle)

      vehicle.set_orientation(old_orientation)
      if was_loaded:
        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)
```

Fungsi

```python
def visualize(self, title: str = "Steepest Ascent Hill Climbing", save_dir: str | None = None) -> None:
```

Deskripsi

Mengambil alih (override) metode visualisasi dari kelas induk dengan menetapkan judul default khusus yaitu "Steepest Ascent Hill Climbing".

Output

Menampilkan atau menyimpan visualisasi hasil Steepest Ascent Hill Climbing.

Source Code

```python
def visualize(self, title: str = "Steepest Ascent Hill Climbing", save_dir: str | None = None) -> None:
    super().visualize(title=title, save_dir=save_dir)
```

Fungsi

```python
def run(self, initial_state: State | None = None) -> State:
```

Deskripsi

Menjalankan algoritma Steepest Ascent Hill Climbing mulai dari state awal (menggunakan status yang diberikan atau membangkitkan random state baru),

	mengevaluasi seluruh tetangga pada setiap iterasi untuk mencari tetangga dengan skor obyektif tertinggi (best score), hingga mencapai kondisi optimal atau batas maksimum iterasi.
Output	Objek instance State terakhir (final state) yang merupakan hasil optimasi terbaik.

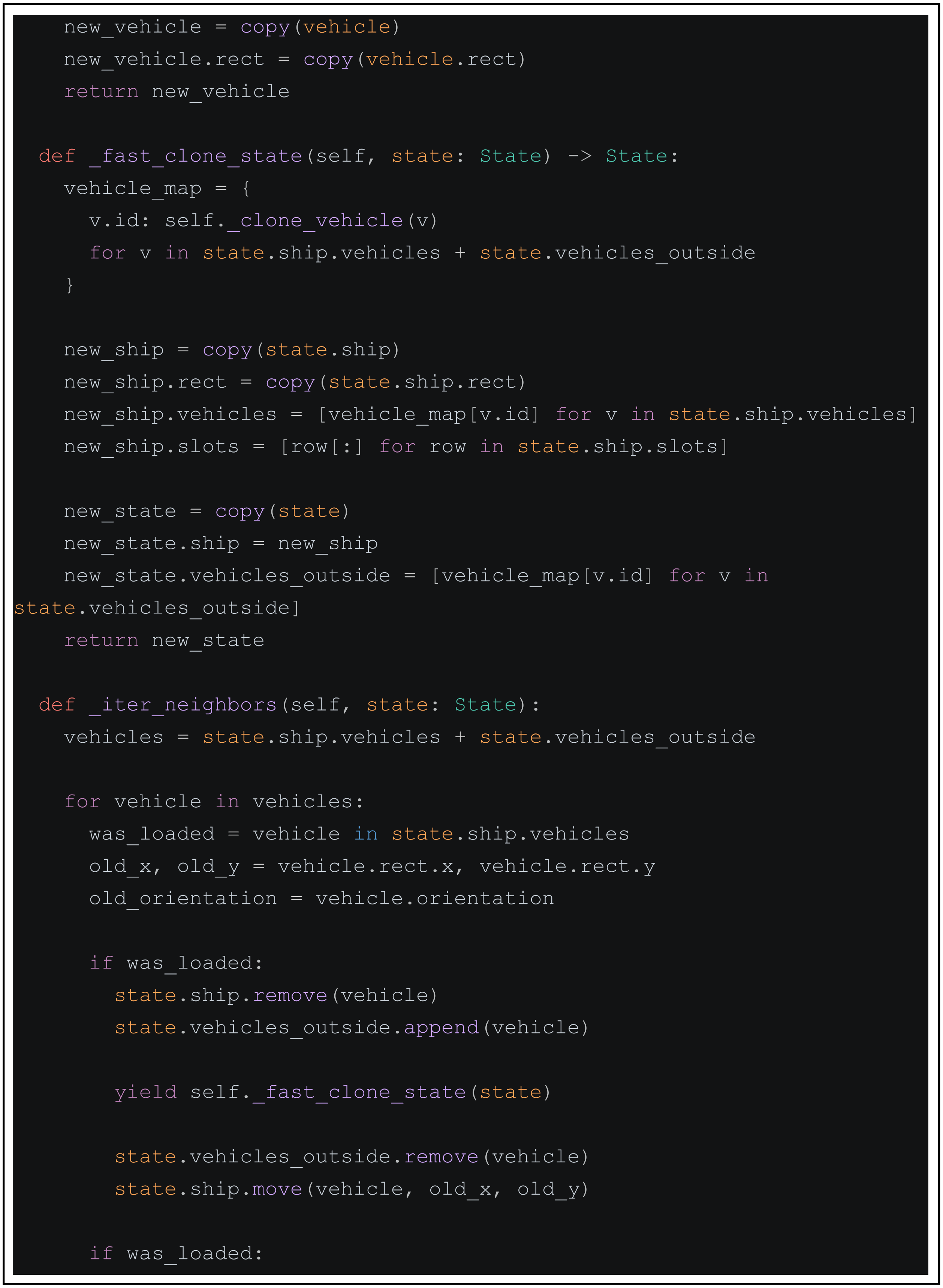
Source Code

```
def run(self, initial_state: State | None = None) -> State:
    start_time = time.perf_counter()
    current = initial_state if initial_state is not None else self.state_factory.create_random_state()
    self.initial_state = current
    self.history = [current.get_objective()]
    self.iterations = 0

    for _ in range(self.max_iterations):
      best_neighbor = None
      best_score = current.get_objective()

      for neighbor in self._iter_neighbors(current):
        score = neighbor.get_objective()
        if score > best_score:
          best_score = score
          best_neighbor = neighbor

      if best_neighbor is None:
        break

      current = best_neighbor
      self.history.append(current.get_objective())
      self.iterations += 1

    self.final_state = current
    self.duration = time.perf_counter() - start_time
    return current
```

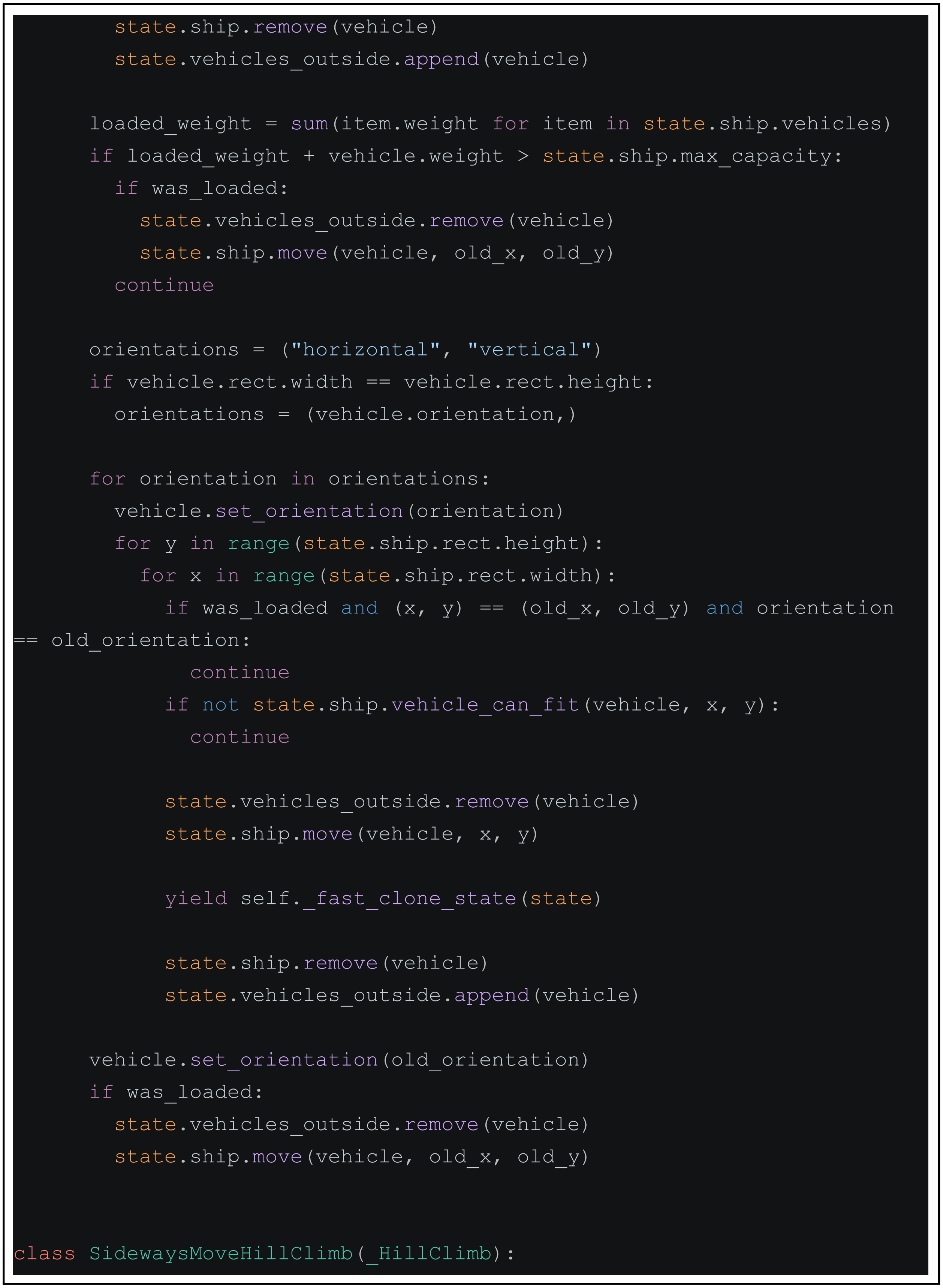
2.2.2. Algoritma *Hill-Climbing with Sideways Move*

Sideways_Move_HC.py

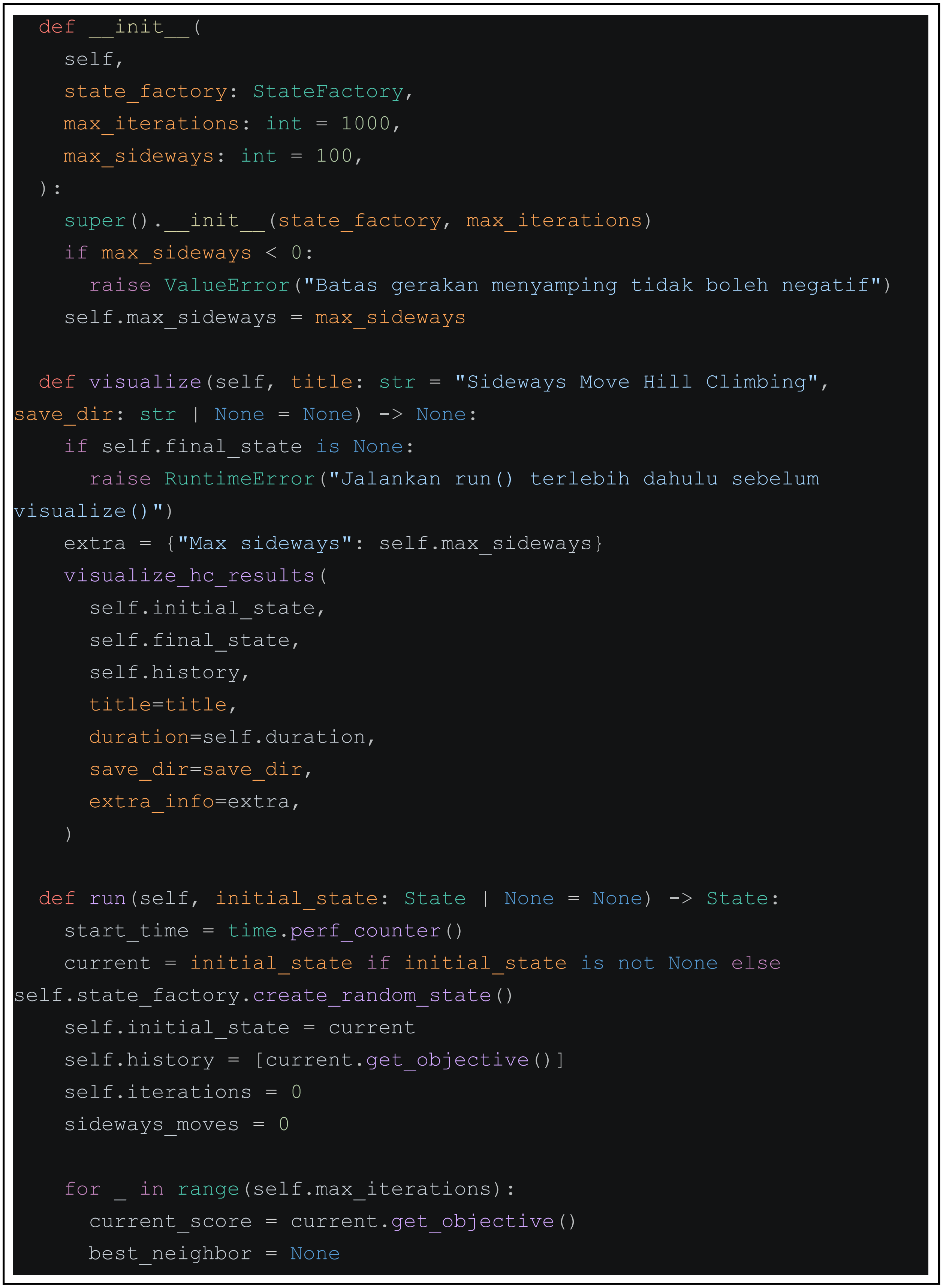
```
import time
from copy import copy
from objects.state import State, StateFactory
from objects.vehicle import Vehicle
from visualizations.ship import visualize_hc_results


class _HillClimb:
  def __init__(self, state_factory: StateFactory, max_iterations: int = 1000):
    if max_iterations < 0:
      raise ValueError("Batas iterasi tidak boleh negatif")
    self.state_factory = state_factory
    self.max_iterations = max_iterations
    self.history: list[int] = []
    self.initial_state: State | None = None
    self.final_state: State | None = None
    self.duration: float = 0.0
    self.iterations: int = 0

  def visualize(self, title: str = "Hill Climbing", save_dir: str | None = None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih dahulu sebelum visualize()")
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
    )

  def _clone_vehicle(self, vehicle: Vehicle) -> Vehicle:
```



```
    new_vehicle = copy(vehicle)
    new_vehicle.rect = copy(vehicle.rect)
    return new_vehicle

  def _fast_clone_state(self, state: State) -> State:
    vehicle_map = {
      v.id: self._clone_vehicle(v)
      for v in state.ship.vehicles + state.vehicles_outside
    }

    new_ship = copy(state.ship)
    new_ship.rect = copy(state.ship.rect)
    new_ship.vehicles = [vehicle_map[v.id] for v in state.ship.vehicles]
    new_ship.slots = [row[:] for row in state.ship.slots]

    new_state = copy(state)
    new_state.ship = new_ship
    new_state.vehicles_outside = [vehicle_map[v.id] for v in
state.vehicles_outside]
    return new_state

  def _iter_neighbors(self, state: State):
    vehicles = state.ship.vehicles + state.vehicles_outside

    for vehicle in vehicles:
      was_loaded = vehicle in state.ship.vehicles
      old_x, old_y = vehicle.rect.x, vehicle.rect.y
      old_orientation = vehicle.orientation

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

        yield self._fast_clone_state(state)

        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)

      if was_loaded:
```



```
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

      loaded_weight = sum(item.weight for item in state.ship.vehicles)
      if loaded_weight + vehicle.weight > state.ship.max_capacity:
        if was_loaded:
          state.vehicles_outside.remove(vehicle)
          state.ship.move(vehicle, old_x, old_y)
        continue

      orientations = ("horizontal", "vertical")
      if vehicle.rect.width == vehicle.rect.height:
        orientations = (vehicle.orientation,)

      for orientation in orientations:
        vehicle.set_orientation(orientation)
        for y in range(state.ship.rect.height):
          for x in range(state.ship.rect.width):
            if was_loaded and (x, y) == (old_x, old_y) and orientation
== old_orientation:
              continue
            if not state.ship.vehicle_can_fit(vehicle, x, y):
              continue

            state.vehicles_outside.remove(vehicle)
            state.ship.move(vehicle, x, y)

            yield self._fast_clone_state(state)

            state.ship.remove(vehicle)
            state.vehicles_outside.append(vehicle)

      vehicle.set_orientation(old_orientation)
      if was_loaded:
        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)


class SidewaysMoveHillClimb(_HillClimb):
```



```
  def __init__(
    self,
    state_factory: StateFactory,
    max_iterations: int = 1000,
    max_sideways: int = 100,
  ):
    super().__init__(state_factory, max_iterations)
    if max_sideways < 0:
      raise ValueError("Batas gerakan menyamping tidak boleh negatif")
    self.max_sideways = max_sideways

  def visualize(self, title: str = "Sideways Move Hill Climbing",
save_dir: str | None = None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih dahulu sebelum
visualize()")
    extra = {"Max sideways": self.max_sideways}
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
      extra_info=extra,
    )

  def run(self, initial_state: State | None = None) -> State:
    start_time = time.perf_counter()
    current = initial_state if initial_state is not None else
self.state_factory.create_random_state()
    self.initial_state = current
    self.history = [current.get_objective()]
    self.iterations = 0
    sideways_moves = 0

    for _ in range(self.max_iterations):
      current_score = current.get_objective()
      best_neighbor = None
      best_score = current_score

      for neighbor in self._iter_neighbors(current):
        score = neighbor.get_objective()
        if score >= best_score:
          best_score = score
          best_neighbor = neighbor

      if best_neighbor is None:
        break

      if best_score == current_score:
        if sideways_moves == self.max_sideways:
          break
        sideways_moves += 1
      else:
        sideways_moves = 0

      current = best_neighbor
      self.history.append(current.get_objective())
      self.iterations += 1

    self.final_state = current
    self.duration = time.perf_counter() - start_time
    return current
```

Fungsi	`def __init__(self, state_factory: StateFactory, max_iterations: int = 1000):`
Deskripsi	Menginisialisasi objek pencarian Hill Climbing dengan parameter state factory dan batas maksimum iterasi (max_iterations). Melakukan validasi apabila jumlah iterasi bernilai negatif, maka akan melempar ValueError. Sekaligus menyiapkan

	atribut riwayat, status awal, status akhir, durasi, dan counter iterasi.
Output	Menginisialisasi instance objek HillClimb.
Source Code	

```
def __init__(self, state_factory: StateFactory,
max_iterations: int = 1000):
    if max_iterations < 0:
      raise ValueError("Batas iterasi tidak boleh
negatif")
    self.state_factory = state_factory
    self.max_iterations = max_iterations
    self.history: list[int] = []
    self.initial_state: State | None = None
    self.final_state: State | None = None
    self.duration: float = 0.0
    self.iterations: int = 0
```

Fungsi	`def visualize(self, title: str = "Hill Climbing", save_dir: str | None = None) -> None:`
Deskripsi	Memeriksa apakah proses pencarian telah dijalankan (final_state tidak boleh None). Jika belum, akan melempar RuntimeError. Jika sudah, memanggil fungsi visualisasi hasil pencarian dengan menyertakan status awal, akhir, riwayat skor, judul, durasi, dan direktori penyimpanan.
Output	Menampilkan atau menyimpan visualisasi grafik dan hasil penataan kapal.
Source Code	

```
def visualize(self, title: str = "Hill Climbing",
save_dir: str | None = None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih
dahulu sebelum visualize()")
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
    )
```

Fungsi	`def _clone_vehicle(self, vehicle: Vehicle) -> Vehicle:`
Deskripsi	Membuat duplikat salinan dari suatu objek `Vehicle` beserta atribut rektangle di dalamnya secara mandiri untuk menghindari mutasi referensi objek asli.
Output	Objek instance Vehicle yang baru.

Source Code

```
  def _clone_vehicle(self, vehicle: Vehicle) ->
Vehicle:
    new_vehicle = copy(vehicle)
    new_vehicle.rect = copy(vehicle.rect)
    return new_vehicle
```

Fungsi	`def _fast_clone_state(self, state: State) -> State:`
Deskripsi	Melakukan kloning atau penyalinan objek State secara cepat dengan menduplikasi kapal, matriks grid slot, serta seluruh kendaraan baik yang ada di dalam maupun di luar kapal menggunakan peta ID kendaraan.
Output	Objek instance State baru yang merupakan salinan mendalam (deep copy).

Source Code

```
def _fast_clone_state(self, state: State) -> State:
    vehicle_map = {
      v.id: self._clone_vehicle(v)
      for v in state.ship.vehicles +
state.vehicles_outside
    }

    new_ship = copy(state.ship)
    new_ship.rect = copy(state.ship.rect)
    new_ship.vehicles = [vehicle_map[v.id] for v in
state.ship.vehicles]
    new_ship.slots = [row[:] for row in
state.ship.slots]

    new_state = copy(state)
    new_state.ship = new_ship
    new_state.vehicles_outside = [vehicle_map[v.id]
for v in state.vehicles_outside]
    return new_state
```

Fungsi	` def _iter_neighbors(self, state: State):`
Deskripsi	Menghasilkan (yield) tetangga-tetangga (neighbors) yang valid secara iteratif dari suatu state dengan mencoba mengeluarkan kendaraan, mengubah orientasi, atau memindahkan posisi kendaraan ke koordinat grid lain yang memenuhi syarat kapasitas dan batas ukuran kapal.
Output	Menghasilkan aliran objek State tetangga satu per satu.

Source Code

```
  def _iter_neighbors(self, state: State):
    vehicles = state.ship.vehicles +
state.vehicles_outside
```

```
    for vehicle in vehicles:
      was_loaded = vehicle in state.ship.vehicles
      old_x, old_y = vehicle.rect.x, vehicle.rect.y
      old_orientation = vehicle.orientation

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

        yield self._fast_clone_state(state)

        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

      loaded_weight = sum(item.weight for item in
state.ship.vehicles)
      if loaded_weight + vehicle.weight >
state.ship.max_capacity:
        if was_loaded:
          state.vehicles_outside.remove(vehicle)
          state.ship.move(vehicle, old_x, old_y)
        continue

      orientations = ("horizontal", "vertical")
      if vehicle.rect.width == vehicle.rect.height:
        orientations = (vehicle.orientation,)

      for orientation in orientations:
        vehicle.set_orientation(orientation)
        for y in range(state.ship.rect.height):
          for x in range(state.ship.rect.width):
            if was_loaded and (x, y) == (old_x,
old_y) and orientation == old_orientation:
              continue
            if not
```

```
state.ship.vehicle_can_fit(vehicle, x, y):
              continue

            state.vehicles_outside.remove(vehicle)
            state.ship.move(vehicle, x, y)

            yield self._fast_clone_state(state)

            state.ship.remove(vehicle)
            state.vehicles_outside.append(vehicle)

      vehicle.set_orientation(old_orientation)
      if was_loaded:
        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)
```

Fungsi

```
  def __init__(
    self,
    state_factory: StateFactory,
    max_iterations: int = 1000,
    max_sideways: int = 100,
  ):
    super().__init__(state_factory, max_iterations)
    if max_sideways < 0:
```

Deskripsi	Menginisialisasi objek pencarian Sideways Move Hill Climbing dengan memanggil konstruktor induk (super().__init__) serta menambahkan parameter batas maksimum gerakan menyamping (max_sideways). Melakukan validasi apabila nilai max_sideways bernilai negatif, maka akan melempar ValueError.
Output	Menginisialisasi instance objek SidewaysMoveHillClimb.

Source Code

```
  def __init__(
    self,
    state_factory: StateFactory,
    max_iterations: int = 1000,
    max_sideways: int = 100,
  ):
    super().__init__(state_factory, max_iterations)
    if max_sideways < 0:
      raise ValueError("Batas gerakan menyamping
tidak boleh negatif")
    self.max_sideways = max_sideways
```

Fungsi

```
  def visualize(self, title: str = "Sideways Move
Hill Climbing", save_dir: str | None = None) ->
None:
```

Deskripsi	Mengambil alih (override) metode visualisasi dari kelas induk dengan menetapkan judul default "Sideways Move Hill Climbing" serta menambahkan informasi tambahan berupa kamus extra yang berisi batas parameter Max sideways untuk ditampilkan pada visualisasi hasil.
Output	Menampilkan atau menyimpan grafik visualisasi hasil *Sideways Move Hill Climbing*.

Source Code

```
  def visualize(self, title: str = "Sideways Move
Hill Climbing", save_dir: str | None = None) ->
None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih
dahulu sebelum visualize()")
    extra = {"Max sideways": self.max_sideways}
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
      extra_info=extra,
    )
```

Fungsi

```
  def run(self, initial_state: State | None = None)
-> State:
```

Deskripsi	Menjalankan algoritma Sideways Move Hill Climbing mulai dari state awal, mengevaluasi seluruh tetangga pada setiap iterasi untuk mencari tetangga dengan skor yang lebih tinggi atau sama (score >= best_score), memantau dan menghitung batasan gerakan menyamping (sideways moves) ketika nilai skor tetangga sama dengan skor saat ini, hingga mencapai kondisi berhenti atau batas maksimum iterasi.
Output	Objek instance State terakhir (final state) yang merupakan hasil optimasi terbaik setelah pencarian selesai.

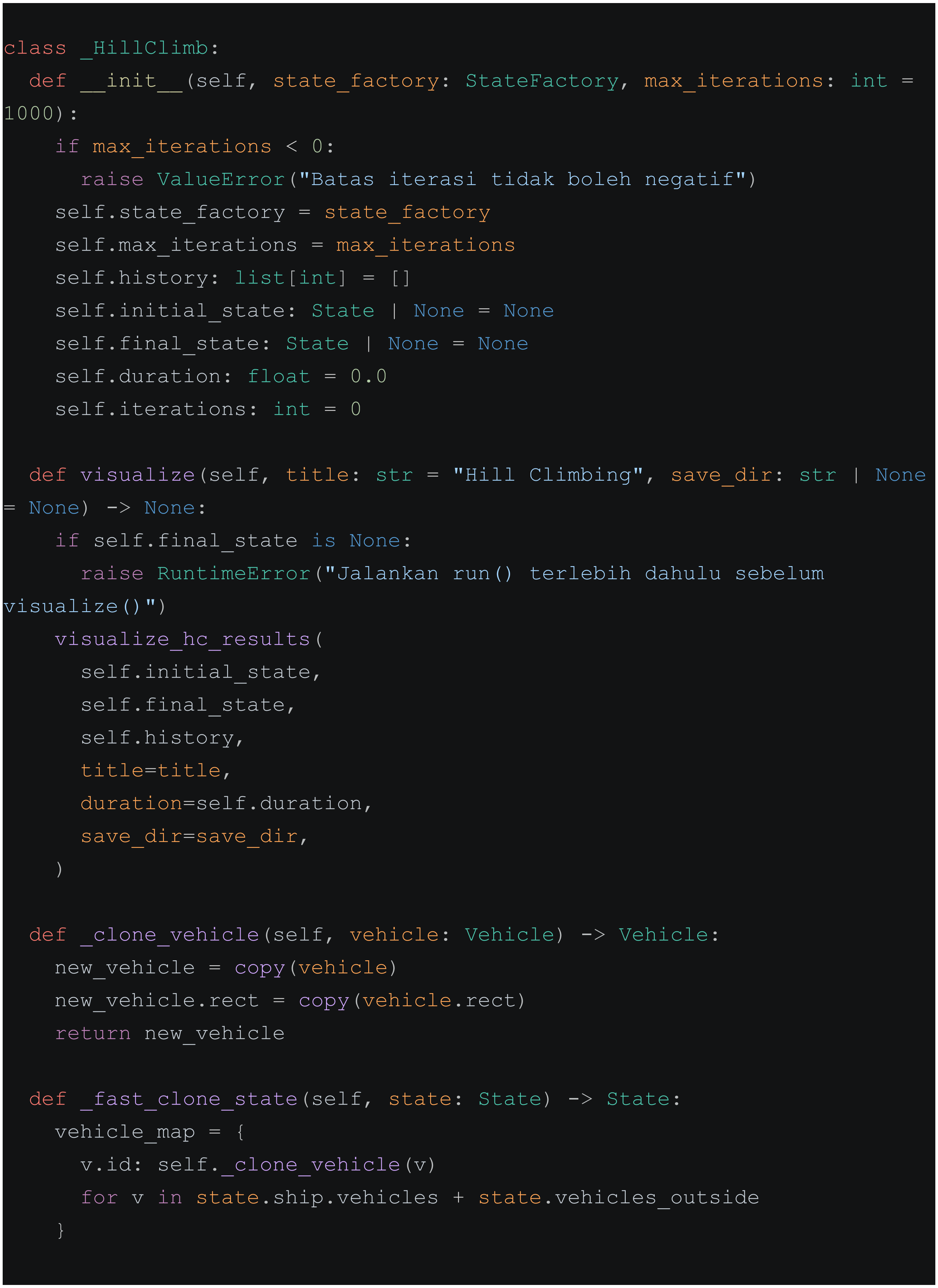
Source Code

```
def run(self, initial_state: State | None = None) ->
State:
    start_time = time.perf_counter()
    current = initial_state if initial_state is not
None else self.state_factory.create_random_state()
    self.initial_state = current
    self.history = [current.get_objective()]
    self.iterations = 0
    sideways_moves = 0

    for _ in range(self.max_iterations):
      current_score = current.get_objective()
      best_neighbor = None
      best_score = current_score

      for neighbor in self._iter_neighbors(current):
        score = neighbor.get_objective()
        if score >= best_score:
          best_score = score
          best_neighbor = neighbor

      if best_neighbor is None:
        break

      if best_score == current_score:
        if sideways_moves == self.max_sideways:
          break
        sideways_moves += 1
      else:
        sideways_moves = 0

      current = best_neighbor
      self.history.append(current.get_objective())
      self.iterations += 1

    self.final_state = current
    self.duration = time.perf_counter() - start_time
    return current
```

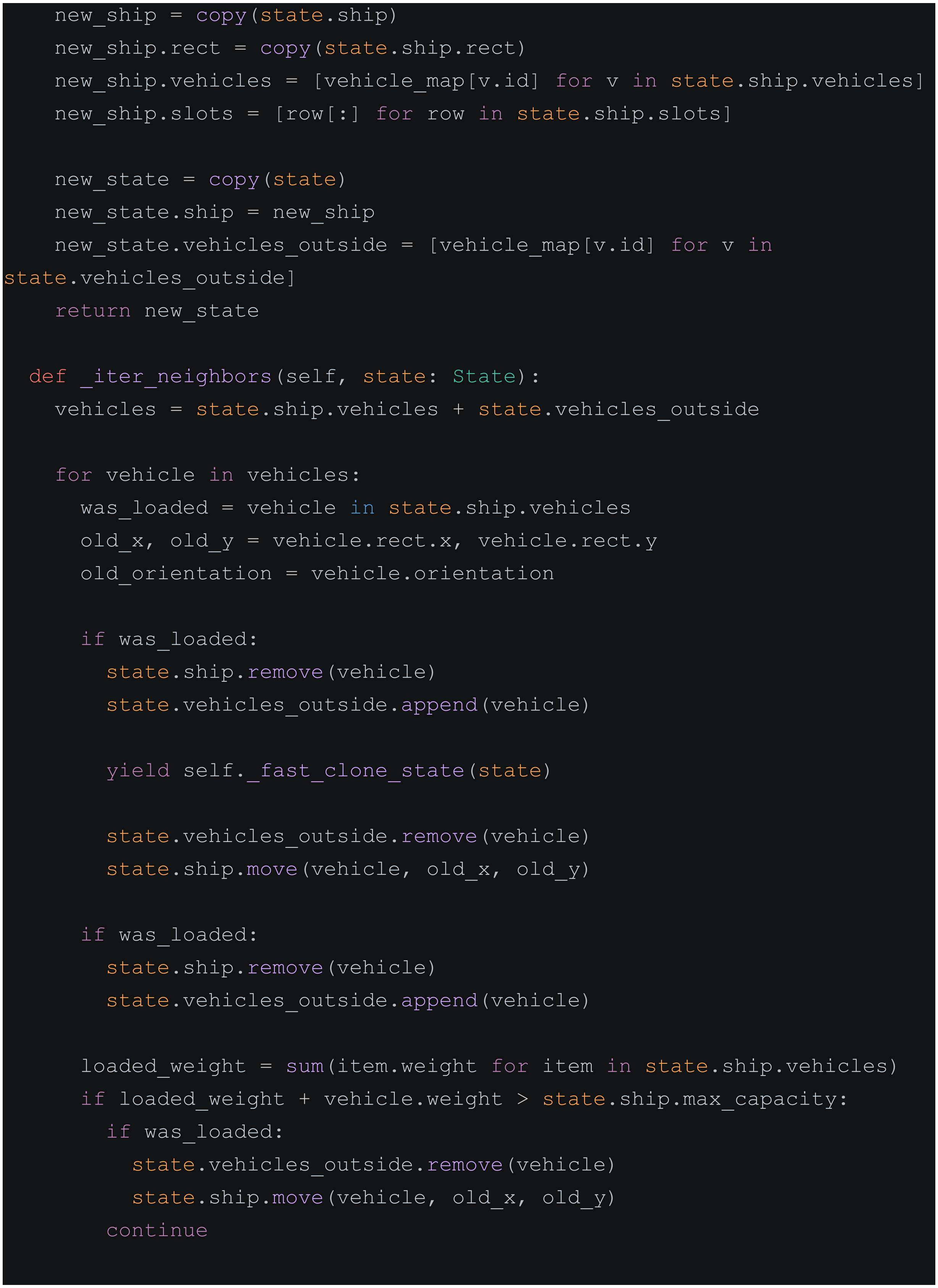
2.2.3. Algoritma *Random Restart Hill-Climbing*

Random_Restart_HC.py

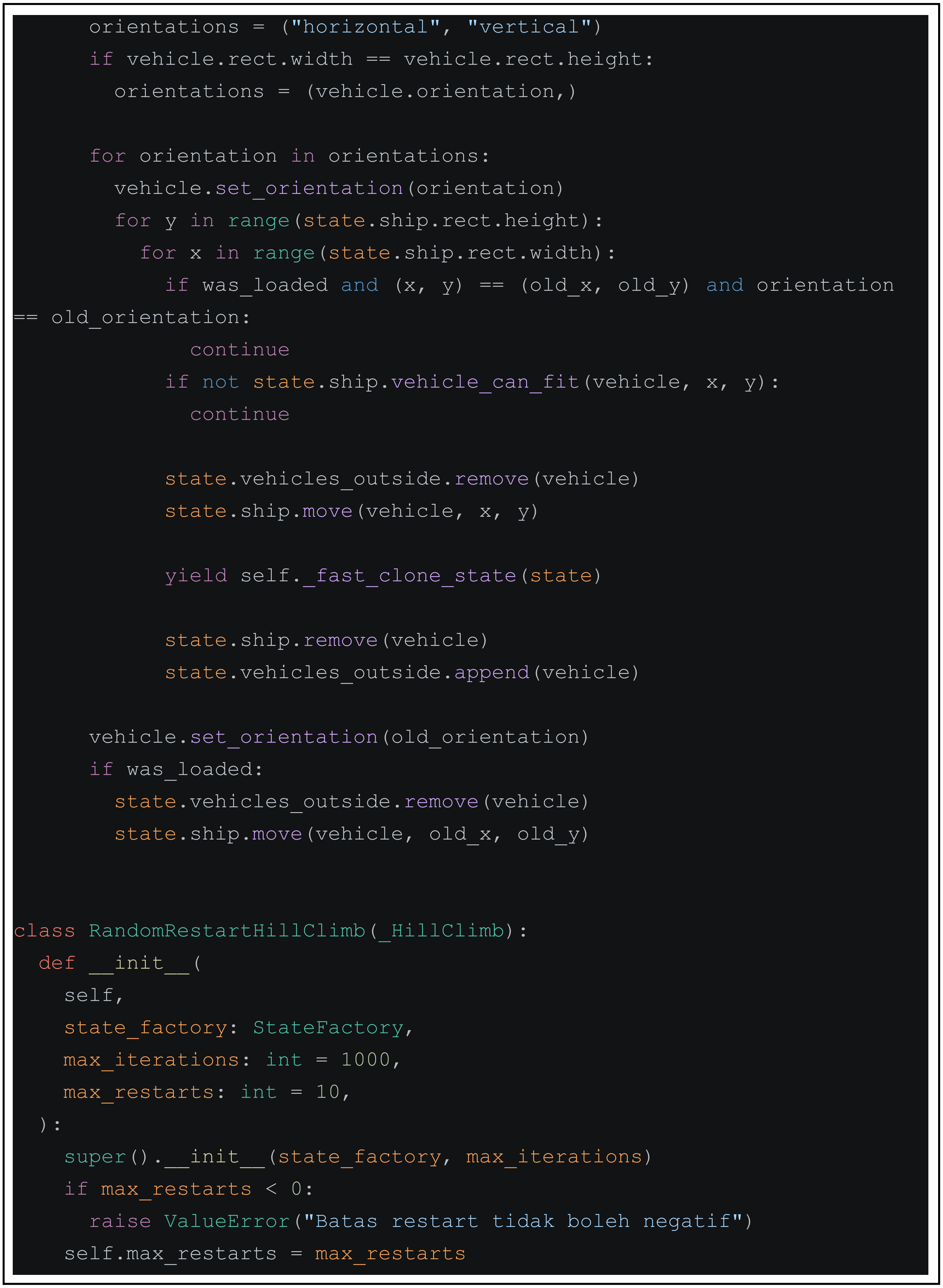
```
import time
from copy import copy
from objects.state import State, StateFactory
from objects.vehicle import Vehicle
from algo.Steepest_Ascend_HC import SteepestAscentHillClimb
from visualizations.ship import visualize_hc_results
```



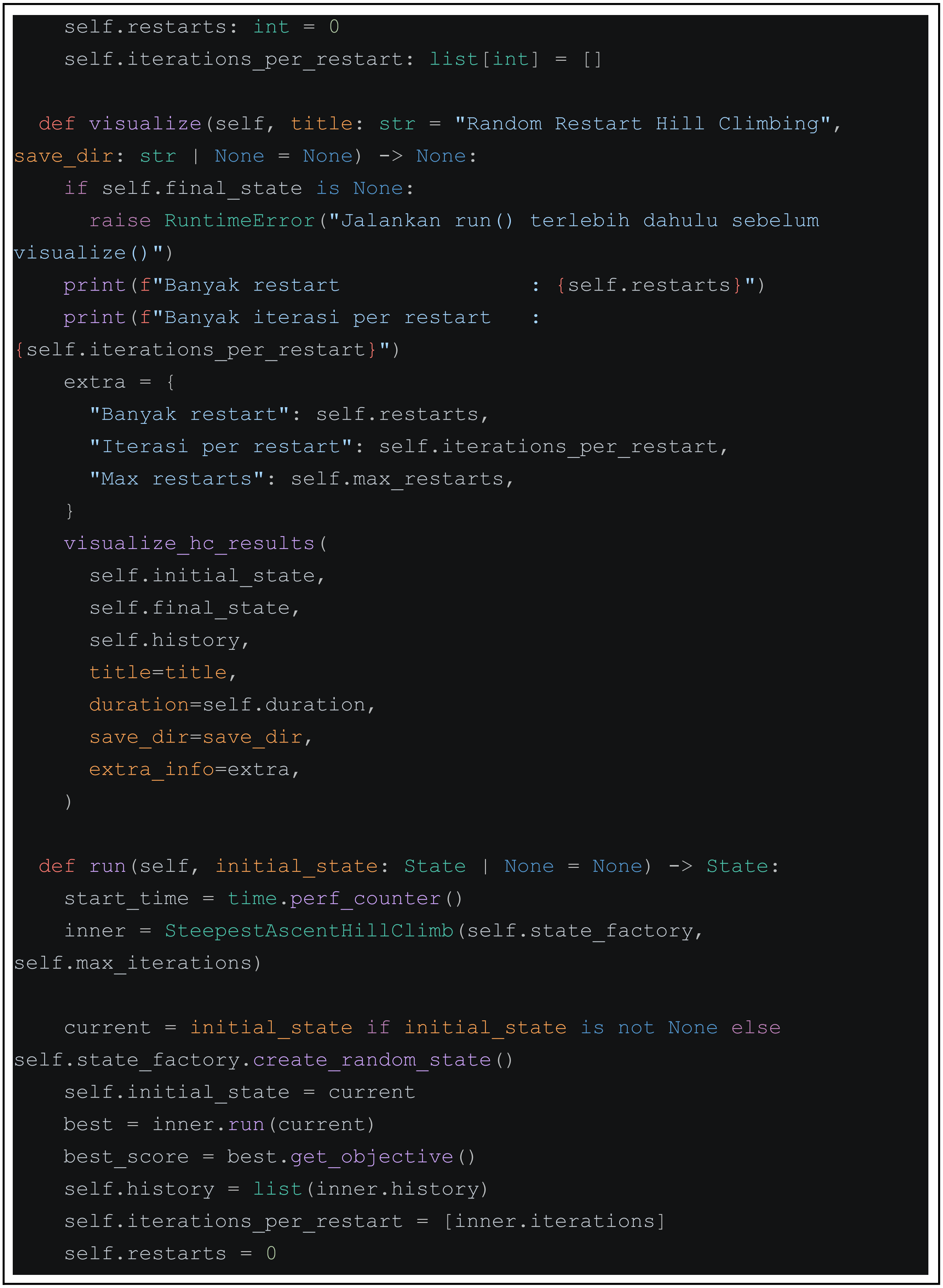
```
class _HillClimb:
  def __init__(self, state_factory: StateFactory, max_iterations: int =
1000):
    if max_iterations < 0:
      raise ValueError("Batas iterasi tidak boleh negatif")
    self.state_factory = state_factory
    self.max_iterations = max_iterations
    self.history: list[int] = []
    self.initial_state: State | None = None
    self.final_state: State | None = None
    self.duration: float = 0.0
    self.iterations: int = 0

  def visualize(self, title: str = "Hill Climbing", save_dir: str | None
= None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih dahulu sebelum
visualize()")
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
    )

  def _clone_vehicle(self, vehicle: Vehicle) -> Vehicle:
    new_vehicle = copy(vehicle)
    new_vehicle.rect = copy(vehicle.rect)
    return new_vehicle

  def _fast_clone_state(self, state: State) -> State:
    vehicle_map = {
      v.id: self._clone_vehicle(v)
      for v in state.ship.vehicles + state.vehicles_outside
    }
```



```
    new_ship = copy(state.ship)
    new_ship.rect = copy(state.ship.rect)
    new_ship.vehicles = [vehicle_map[v.id] for v in state.ship.vehicles]
    new_ship.slots = [row[:] for row in state.ship.slots]

    new_state = copy(state)
    new_state.ship = new_ship
    new_state.vehicles_outside = [vehicle_map[v.id] for v in
state.vehicles_outside]
    return new_state

  def _iter_neighbors(self, state: State):
    vehicles = state.ship.vehicles + state.vehicles_outside

    for vehicle in vehicles:
      was_loaded = vehicle in state.ship.vehicles
      old_x, old_y = vehicle.rect.x, vehicle.rect.y
      old_orientation = vehicle.orientation

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

        yield self._fast_clone_state(state)

        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

      loaded_weight = sum(item.weight for item in state.ship.vehicles)
      if loaded_weight + vehicle.weight > state.ship.max_capacity:
        if was_loaded:
          state.vehicles_outside.remove(vehicle)
          state.ship.move(vehicle, old_x, old_y)
        continue
```



```
      orientations = ("horizontal", "vertical")
      if vehicle.rect.width == vehicle.rect.height:
        orientations = (vehicle.orientation,)

      for orientation in orientations:
        vehicle.set_orientation(orientation)
        for y in range(state.ship.rect.height):
          for x in range(state.ship.rect.width):
            if was_loaded and (x, y) == (old_x, old_y) and orientation
== old_orientation:
              continue
            if not state.ship.vehicle_can_fit(vehicle, x, y):
              continue

            state.vehicles_outside.remove(vehicle)
            state.ship.move(vehicle, x, y)

            yield self._fast_clone_state(state)

            state.ship.remove(vehicle)
            state.vehicles_outside.append(vehicle)

      vehicle.set_orientation(old_orientation)
      if was_loaded:
        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)


class RandomRestartHillClimb(_HillClimb):
  def __init__(
    self,
    state_factory: StateFactory,
    max_iterations: int = 1000,
    max_restarts: int = 10,
  ):
    super().__init__(state_factory, max_iterations)
    if max_restarts < 0:
      raise ValueError("Batas restart tidak boleh negatif")
    self.max_restarts = max_restarts
```



```
    self.restarts: int = 0
    self.iterations_per_restart: list[int] = []

  def visualize(self, title: str = "Random Restart Hill Climbing",
save_dir: str | None = None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih dahulu sebelum
visualize()")
    print(f"Banyak restart               : {self.restarts}")
    print(f"Banyak iterasi per restart   :
{self.iterations_per_restart}")
    extra = {
      "Banyak restart": self.restarts,
      "Iterasi per restart": self.iterations_per_restart,
      "Max restarts": self.max_restarts,
    }
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
      extra_info=extra,
    )

  def run(self, initial_state: State | None = None) -> State:
    start_time = time.perf_counter()
    inner = SteepestAscentHillClimb(self.state_factory,
self.max_iterations)

    current = initial_state if initial_state is not None else
self.state_factory.create_random_state()
    self.initial_state = current
    best = inner.run(current)
    best_score = best.get_objective()
    self.history = list(inner.history)
    self.iterations_per_restart = [inner.iterations]
    self.restarts = 0
    for _ in range(self.max_restarts):
      self.restarts += 1
      start = self.state_factory.create_random_state()
      result = inner.run(start)
      self.history.extend(inner.history)
      self.iterations_per_restart.append(inner.iterations)

      score = result.get_objective()
      if score > best_score:
        best_score = score
        best = result

    self.iterations = sum(self.iterations_per_restart)
    self.final_state = best
    self.duration = time.perf_counter() - start_time
    return best
```

Fungsi	`def __init__(self, state_factory: StateFactory, max_iterations: int = 1000):`
Deskripsi	Menginisialisasi objek pencarian Random Restart Hill Climbing dengan parameter state factory, batas maksimum iterasi per restart, serta jumlah maksimum restart (max_restarts). Memanggil konstruktor induk melalui super().__init__, memvalidasi agar nilai batas restart tidak negatif (melempar ValueError jika kurang dari 0), serta menyiapkan atribut pelacak jumlah restart dan riwayat iterasi per restart.
Output	-
Source Code	

```
def __init__(self, state_factory: StateFactory,
max_iterations: int = 1000):
    if max_iterations < 0:
```

```python
      raise ValueError("Batas iterasi tidak boleh negatif")
    self.state_factory = state_factory
    self.max_iterations = max_iterations
    self.history: list[int] = []
    self.initial_state: State | None = None
    self.final_state: State | None = None
    self.duration: float = 0.0
    self.iterations: int = 0
```

Fungsi	`def visualize(self, title: str = "Hill Climbing", save_dir: str | None = None) -> None:`
Deskripsi	Mengambil alih (override) metode visualisasi dari kelas induk untuk menetapkan judul default "Random Restart Hill Climbing", serta menyusun kamus extra_info yang berisi parameter "Max restarts" dan "Iterasi per restart" untuk disertakan ke dalam laporan grafik visualisasi hasil.
Output	-

Source Code

```python
def visualize(self, title: str = "Hill Climbing", save_dir: str | None = None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih dahulu sebelum visualize()")
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
    )
```

Fungsi	`def _clone_vehicle(self, vehicle: Vehicle) -> Vehicle:`
Deskripsi	Menjalankan algoritma Random Restart Hill Climbing secara berulang sebanyak jumlah restart yang telah ditentukan. Pada setiap sesi restart, algoritma membangkitkan state awal acak baru, menjalankan Steepest Ascent Hill Climbing, memantau skor terbaik secara global (best score), serta mencatat riwayat iterasi dan durasi total eksekusi hingga seluruh siklus restart selesai.
Output	Objek instance State terakhir yang memiliki nilai objektif atau skor optimal terbaik secara global (best state) setelah seluruh proses pencarian restart berakhir.
Source Code	

```python
def _clone_vehicle(self, vehicle: Vehicle) ->
Vehicle:
    new_vehicle = copy(vehicle)
    new_vehicle.rect = copy(vehicle.rect)
    return new_vehicle
```

Fungsi	`def _fast_clone_state(self, state: State) -> State:`
Deskripsi	
Output	
Source Code	

```python
 def _fast_clone_state(self, state: State) -> State:
    vehicle_map = {
      v.id: self._clone_vehicle(v)
      for v in state.ship.vehicles +
state.vehicles_outside
    }
    new_ship = copy(state.ship)
    new_ship.rect = copy(state.ship.rect)
    new_ship.vehicles = [vehicle_map[v.id] for v in state.ship.vehicles]
    new_ship.slots = [row[:] for row in state.ship.slots]

    new_state = copy(state)
    new_state.ship = new_ship
    new_state.vehicles_outside = [vehicle_map[v.id] for v in state.vehicles_outside]
    return new_state
```

Fungsi	`def _iter_neighbors(self, state: State):`
Deskripsi	
Output	

Source Code

```
  def _iter_neighbors(self, state: State):
    vehicles = state.ship.vehicles + state.vehicles_outside

    for vehicle in vehicles:
      was_loaded = vehicle in state.ship.vehicles
      old_x, old_y = vehicle.rect.x, vehicle.rect.y
      old_orientation = vehicle.orientation

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

        yield self._fast_clone_state(state)

        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)
      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

      loaded_weight = sum(item.weight for item in state.ship.vehicles)
      if loaded_weight + vehicle.weight > state.ship.max_capacity:
        if was_loaded:
          state.vehicles_outside.remove(vehicle)
          state.ship.move(vehicle, old_x, old_y)
        continue

      orientations = ("horizontal", "vertical")
      if vehicle.rect.width == vehicle.rect.height:
        orientations = (vehicle.orientation,)

      for orientation in orientations:
        vehicle.set_orientation(orientation)
        for y in range(state.ship.rect.height):
          for x in range(state.ship.rect.width):
            if was_loaded and (x, y) == (old_x, old_y) and orientation == old_orientation:
              continue
            if not state.ship.vehicle_can_fit(vehicle, x, y):
              continue

            state.vehicles_outside.remove(vehicle)
            state.ship.move(vehicle, x, y)

            yield self._fast_clone_state(state)

            state.ship.remove(vehicle)
            state.vehicles_outside.append(vehicle)

      vehicle.set_orientation(old_orientation)
      if was_loaded:
        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)
```

Fungsi

```
def __init__(
    self,
    state_factory: StateFactory,
    max_iterations: int = 1000,
    max_restarts: int = 10,
  ):
```

Deskripsi

Output

Source Code

```
  def __init__(
    self,
    state_factory: StateFactory,
    max_iterations: int = 1000,
    max_restarts: int = 10,
  ):
    super().__init__(state_factory, max_iterations)
    if max_restarts < 0:
      raise ValueError("Batas restart tidak boleh
negatif")
    self.max_restarts = max_restarts
    self.restarts: int = 0
    self.iterations_per_restart: list[int] = []
```

Fungsi

```
  def visualize(self, title: str = "Random Restart
Hill Climbing", save_dir: str | None = None) ->
None:
```

Deskripsi

Output

Source Code

```python
  def visualize(self, title: str = "Random Restart Hill Climbing", save_dir: str | None = None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih dahulu sebelum visualize()")
    print(f"Banyak restart               : {self.restarts}")
    print(f"Banyak iterasi per restart   : {self.iterations_per_restart}")
    extra = {
      "Banyak restart": self.restarts,
      "Iterasi per restart": self.iterations_per_restart,
      "Max restarts": self.max_restarts,
    }
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
      extra_info=extra,
    )
```

Fungsi

```python
  def run(self, initial_state: State | None = None) -> State:
```

Deskripsi

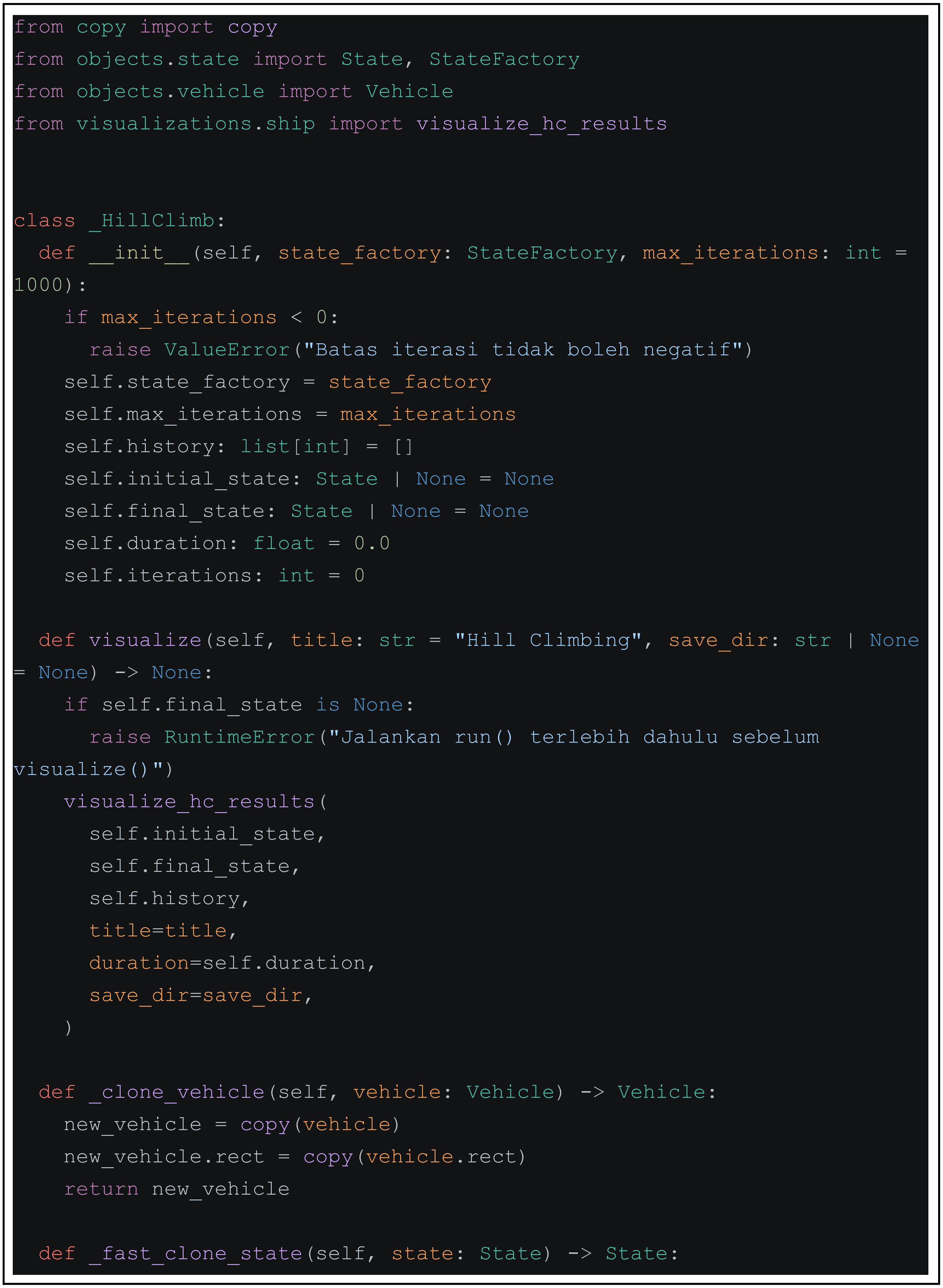
Output

Source Code

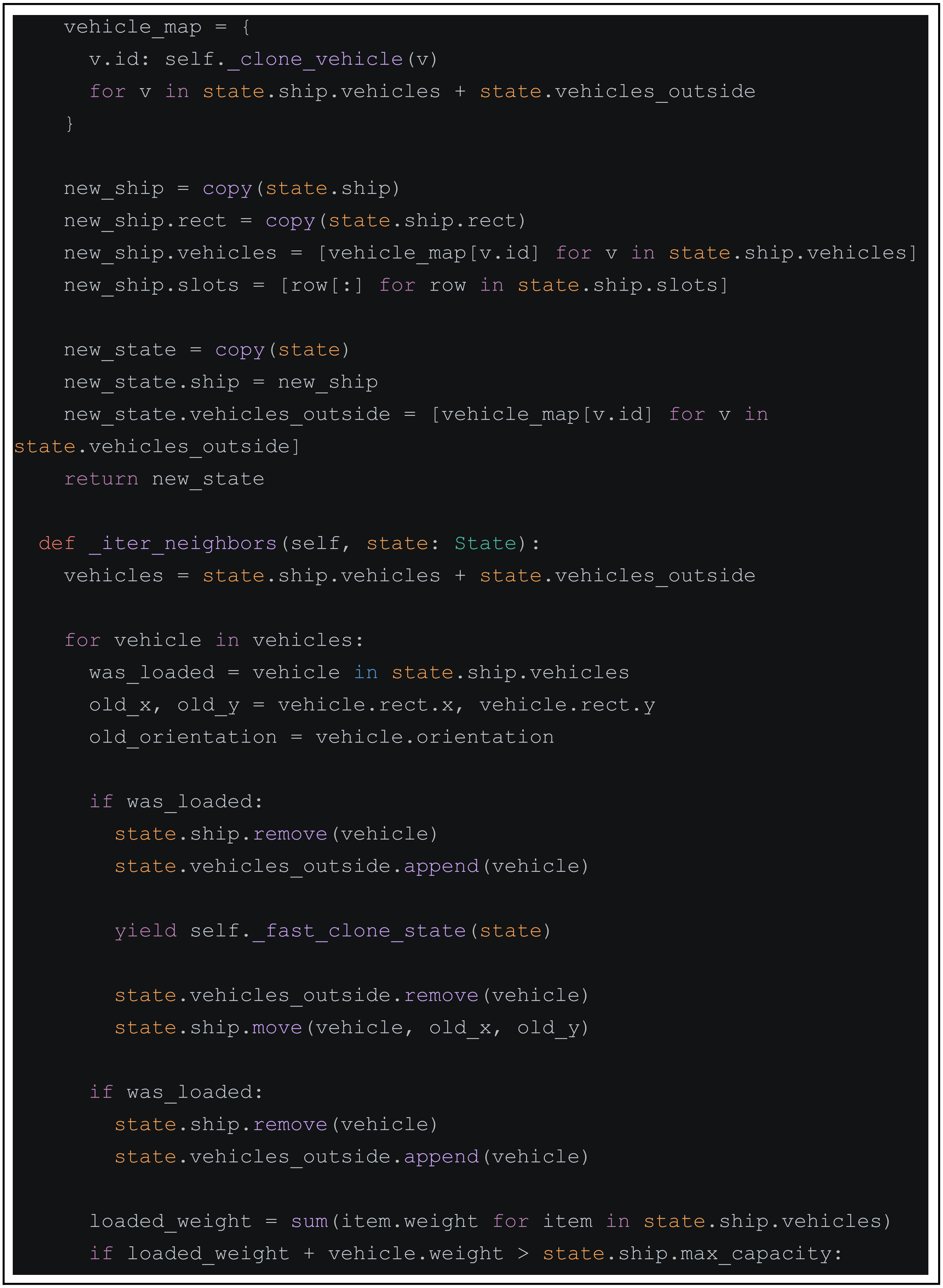
```python
def run(self, initial_state: State | None = None) -> State:
    start_time = time.perf_counter()
    inner =
SteepestAscentHillClimb(self.state_factory,
self.max_iterations)

    current = initial_state if initial_state is not
None else self.state_factory.create_random_state()
    self.initial_state = current
    best = inner.run(current)
    best_score = best.get_objective()
    self.history = list(inner.history)
    self.iterations_per_restart = [inner.iterations]
    self.restarts = 0

    for _ in range(self.max_restarts):
      self.restarts += 1
      start =
self.state_factory.create_random_state()
      result = inner.run(start)
      self.history.extend(inner.history)

self.iterations_per_restart.append(inner.iterations)

      score = result.get_objective()
      if score > best_score:
        best_score = score
        best = result

    self.iterations =
sum(self.iterations_per_restart)
    self.final_state = best
    self.duration = time.perf_counter() - start_time
    return best
```

2.2.4. Algoritma *Stochastic Hill-Climbing*

```
import random
import time
```



```
from copy import copy
from objects.state import State, StateFactory
from objects.vehicle import Vehicle
from visualizations.ship import visualize_hc_results


class _HillClimb:
  def __init__(self, state_factory: StateFactory, max_iterations: int =
1000):
    if max_iterations < 0:
      raise ValueError("Batas iterasi tidak boleh negatif")
    self.state_factory = state_factory
    self.max_iterations = max_iterations
    self.history: list[int] = []
    self.initial_state: State | None = None
    self.final_state: State | None = None
    self.duration: float = 0.0
    self.iterations: int = 0

  def visualize(self, title: str = "Hill Climbing", save_dir: str | None
= None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih dahulu sebelum
visualize()")
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
    )

  def _clone_vehicle(self, vehicle: Vehicle) -> Vehicle:
    new_vehicle = copy(vehicle)
    new_vehicle.rect = copy(vehicle.rect)
    return new_vehicle

  def _fast_clone_state(self, state: State) -> State:
```



```
    vehicle_map = {
      v.id: self._clone_vehicle(v)
      for v in state.ship.vehicles + state.vehicles_outside
    }

    new_ship = copy(state.ship)
    new_ship.rect = copy(state.ship.rect)
    new_ship.vehicles = [vehicle_map[v.id] for v in state.ship.vehicles]
    new_ship.slots = [row[:] for row in state.ship.slots]

    new_state = copy(state)
    new_state.ship = new_ship
    new_state.vehicles_outside = [vehicle_map[v.id] for v in
state.vehicles_outside]
    return new_state

  def _iter_neighbors(self, state: State):
    vehicles = state.ship.vehicles + state.vehicles_outside

    for vehicle in vehicles:
      was_loaded = vehicle in state.ship.vehicles
      old_x, old_y = vehicle.rect.x, vehicle.rect.y
      old_orientation = vehicle.orientation

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

        yield self._fast_clone_state(state)

        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

      loaded_weight = sum(item.weight for item in state.ship.vehicles)
      if loaded_weight + vehicle.weight > state.ship.max_capacity:
        if was_loaded:
          state.vehicles_outside.remove(vehicle)
          state.ship.move(vehicle, old_x, old_y)
        continue

      orientations = ("horizontal", "vertical")
      if vehicle.rect.width == vehicle.rect.height:
        orientations = (vehicle.orientation,)

      for orientation in orientations:
        vehicle.set_orientation(orientation)
        for y in range(state.ship.rect.height):
          for x in range(state.ship.rect.width):
            if was_loaded and (x, y) == (old_x, old_y) and orientation
== old_orientation:
              continue
            if not state.ship.vehicle_can_fit(vehicle, x, y):
              continue

            state.vehicles_outside.remove(vehicle)
            state.ship.move(vehicle, x, y)

            yield self._fast_clone_state(state)

            state.ship.remove(vehicle)
            state.vehicles_outside.append(vehicle)

      vehicle.set_orientation(old_orientation)
      if was_loaded:
        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)


class StochasticHillClimb(_HillClimb):
  def visualize(self, title: str = "Stochastic Hill Climbing", save_dir:
str | None = None) -> None:
    super().visualize(title=title, save_dir=save_dir)

  def run(self, initial_state: State | None = None) -> State:
    start_time = time.perf_counter()
    current = initial_state if initial_state is not None else
self.state_factory.create_random_state()
    self.initial_state = current
    self.history = [current.get_objective()]
    self.iterations = 0

    for _ in range(self.max_iterations):
      current_score = current.get_objective()
      better_neighbors = [
        neighbor for neighbor in self._iter_neighbors(current)
        if neighbor.get_objective() > current_score
      ]

      if not better_neighbors:
        break

      current = random.choice(better_neighbors)
      self.history.append(current.get_objective())
      self.iterations += 1

    self.final_state = current
    self.duration = time.perf_counter() - start_time
    return current
```

Fungsi

```
  def __init__(self, state_factory: StateFactory,
max_iterations: int = 1000):
```

Deskripsi

Output

Source Code

```
  def __init__(self, state_factory: StateFactory,
max_iterations: int = 1000):
    if max_iterations < 0:
      raise ValueError("Batas iterasi tidak boleh
negatif")
```

```
    self.state_factory = state_factory
    self.max_iterations = max_iterations
    self.history: list[int] = []
    self.initial_state: State | None = None
    self.final_state: State | None = None
    self.duration: float = 0.0
    self.iterations: int = 0
```

Fungsi

```
  def visualize(self, title: str = "Hill Climbing", save_dir: str | None = None) -> None:
```

Deskripsi

Output

Source Code

```
def visualize(self, title: str = "Hill Climbing", save_dir: str | None = None) -> None:
    if self.final_state is None:
      raise RuntimeError("Jalankan run() terlebih dahulu sebelum visualize()")
    visualize_hc_results(
      self.initial_state,
      self.final_state,
      self.history,
      title=title,
      duration=self.duration,
      save_dir=save_dir,
    )
```

Fungsi

```
  def _clone_vehicle(self, vehicle: Vehicle) -> Vehicle:
```

Deskripsi

Output

Source Code

```python
  def _clone_vehicle(self, vehicle: Vehicle) -> Vehicle:
    new_vehicle = copy(vehicle)
    new_vehicle.rect = copy(vehicle.rect)
    return new_vehicle
```

Fungsi

```python
  def _fast_clone_state(self, state: State) -> State:
```

Deskripsi

Output

Source Code

```python
   def _fast_clone_state(self, state: State) -> State:
    vehicle_map = {
      v.id: self._clone_vehicle(v)
      for v in state.ship.vehicles + state.vehicles_outside
    }

    new_ship = copy(state.ship)
    new_ship.rect = copy(state.ship.rect)
    new_ship.vehicles = [vehicle_map[v.id] for v in state.ship.vehicles]
    new_ship.slots = [row[:] for row in state.ship.slots]

    new_state = copy(state)
    new_state.ship = new_ship
    new_state.vehicles_outside = [vehicle_map[v.id] for v in state.vehicles_outside]
    return new_stat
```

Fungsi	`def _iter_neighbors(self, state: State):`
Deskripsi	
Output	
Source Code	

```python
def _iter_neighbors(self, state: State):
    vehicles = state.ship.vehicles + state.vehicles_outside

    for vehicle in vehicles:
      was_loaded = vehicle in state.ship.vehicles
      old_x, old_y = vehicle.rect.x, vehicle.rect.y
      old_orientation = vehicle.orientation

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

        yield self._fast_clone_state(state)

        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)

      if was_loaded:
        state.ship.remove(vehicle)
        state.vehicles_outside.append(vehicle)

      loaded_weight = sum(item.weight for item in state.ship.vehicles)
      if loaded_weight + vehicle.weight > state.ship.max_capacity:
        if was_loaded:
          state.vehicles_outside.remove(vehicle)
          state.ship.move(vehicle, old_x, old_y)
        continue
      orientations = ("horizontal", "vertical")
      if vehicle.rect.width == vehicle.rect.height:
        orientations = (vehicle.orientation,)

      for orientation in orientations:
        vehicle.set_orientation(orientation)
        for y in range(state.ship.rect.height):
          for x in range(state.ship.rect.width):
            if was_loaded and (x, y) == (old_x, old_y) and orientation == old_orientation:
              continue
            if not state.ship.vehicle_can_fit(vehicle, x, y):
              continue

            state.vehicles_outside.remove(vehicle)
            state.ship.move(vehicle, x, y)

            yield self._fast_clone_state(state)

            state.ship.remove(vehicle)
            state.vehicles_outside.append(vehicle)

      vehicle.set_orientation(old_orientation)
      if was_loaded:
        state.vehicles_outside.remove(vehicle)
        state.ship.move(vehicle, old_x, old_y)
```

Fungsi

```python
def visualize(self, title: str = "Stochastic Hill Climbing", save_dir: str | None = None) -> None:
```

Deskripsi	
Output	

Source Code

```
  def visualize(self, title: str = "Stochastic Hill
Climbing", save_dir: str | None = None) -> None:
    super().visualize(title=title,
save_dir=save_dir)
```

Fungsi	`def run(self, initial_state: State | None = None) -> State:`
Deskripsi	
Output	

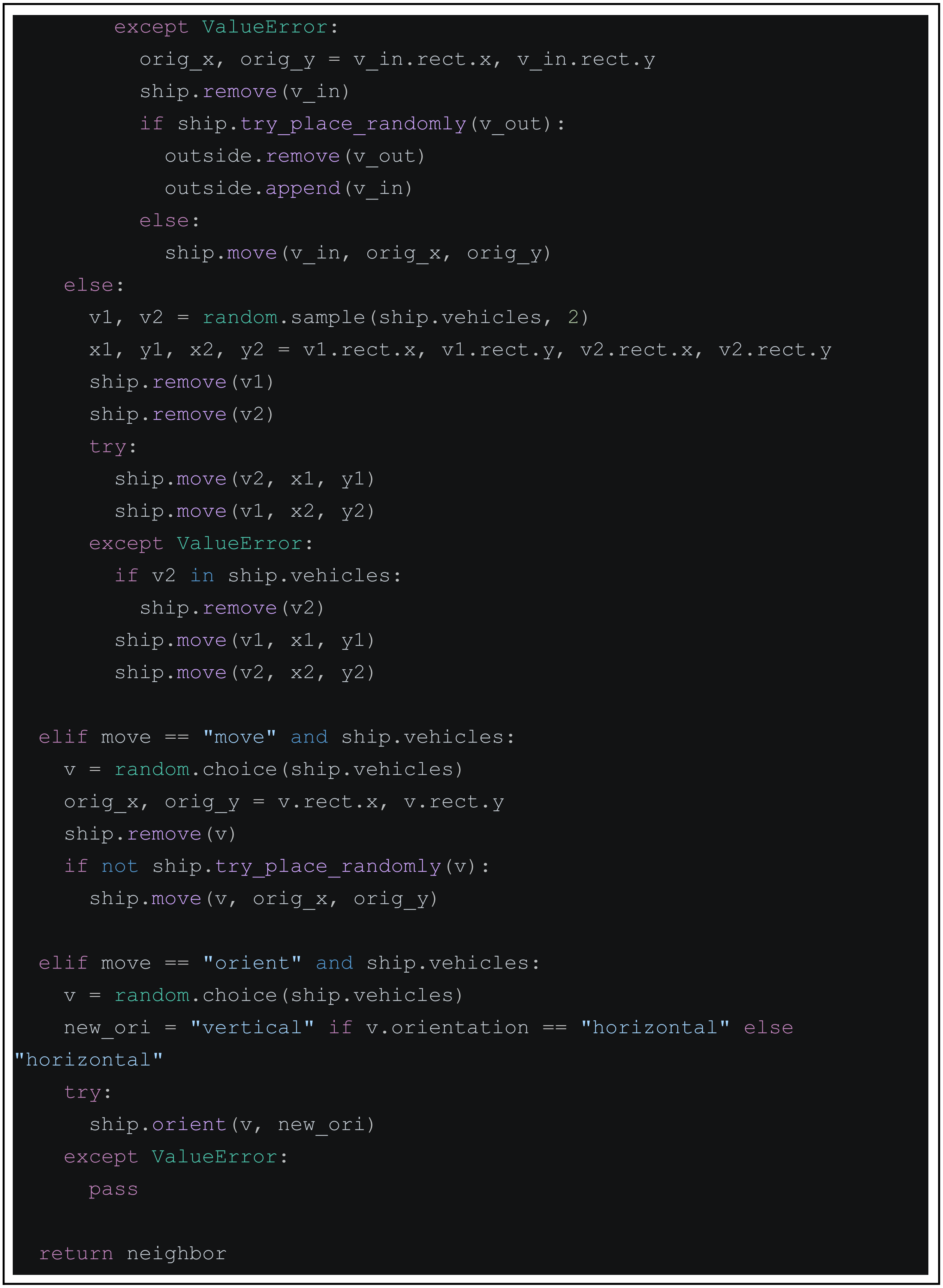
Source Code

```
def run(self, initial_state: State | None = None) ->
State:
    start_time = time.perf_counter()
    current = initial_state if initial_state is not
None else self.state_factory.create_random_state()
    self.initial_state = current
    self.history = [current.get_objective()]
    self.iterations = 0

    for _ in range(self.max_iterations):
      current_score = current.get_objective()
      better_neighbors = [
        neighbor for neighbor in
self._iter_neighbors(current)
        if neighbor.get_objective() > current_score
      ]

      if not better_neighbors:
        break

      current = random.choice(better_neighbors)
      self.history.append(current.get_objective())
      self.iterations += 1

    self.final_state = current
    self.duration = time.perf_counter() - start_time
    return current
```

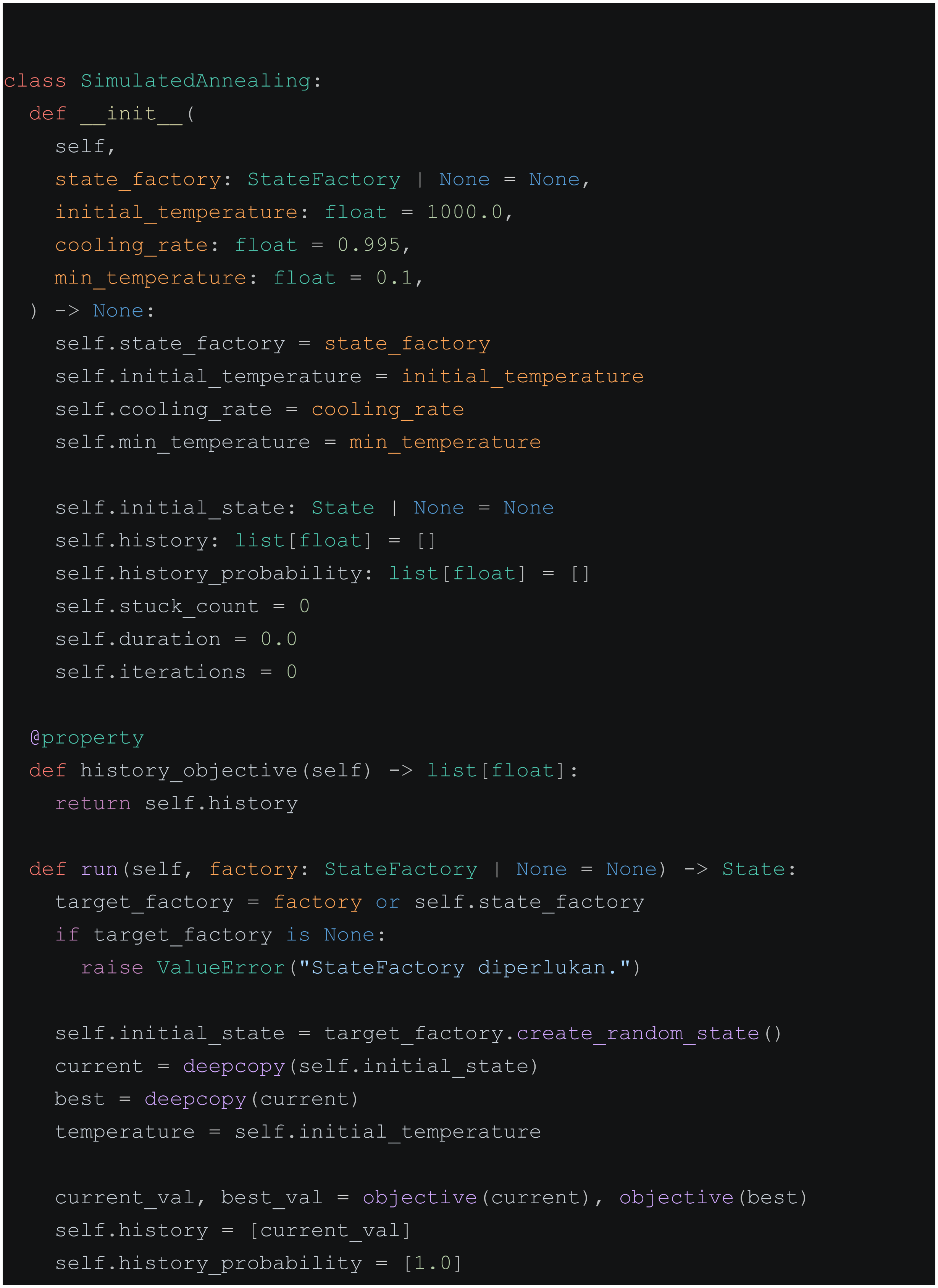
2.2.5. Algoritma *Simulated Annealing*

simulated_annealing.py

```
import math
import random
import time
from copy import deepcopy

from algo.objective import objective
from objects.state import State, StateFactory


def create_random_state(factory: StateFactory) -> State:
  return factory.create_random_state()


def _generate_neighbor(state: State) -> State:
  neighbor = deepcopy(state)
  ship, outside = neighbor.ship, neighbor.vehicles_outside
  if not ship.vehicles and not outside:
    return neighbor

  move = random.choice(["swap", "move", "orient"])

  if move == "swap" and (len(ship.vehicles) >= 2 or (ship.vehicles and
outside)):
    if outside and (len(ship.vehicles) < 2 or random.random() < 0.5):
      v_in, v_out = random.choice(ship.vehicles), random.choice(outside)
      if ship.total_weight - v_in.weight + v_out.weight <=
ship.max_capacity:
        try:
          ship.replace(v_in, v_out)
          outside.remove(v_out)
          outside.append(v_in)
```



```
        except ValueError:
          orig_x, orig_y = v_in.rect.x, v_in.rect.y
          ship.remove(v_in)
          if ship.try_place_randomly(v_out):
            outside.remove(v_out)
            outside.append(v_in)
          else:
            ship.move(v_in, orig_x, orig_y)
    else:
      v1, v2 = random.sample(ship.vehicles, 2)
      x1, y1, x2, y2 = v1.rect.x, v1.rect.y, v2.rect.x, v2.rect.y
      ship.remove(v1)
      ship.remove(v2)
      try:
        ship.move(v2, x1, y1)
        ship.move(v1, x2, y2)
      except ValueError:
        if v2 in ship.vehicles:
          ship.remove(v2)
        ship.move(v1, x1, y1)
        ship.move(v2, x2, y2)

  elif move == "move" and ship.vehicles:
    v = random.choice(ship.vehicles)
    orig_x, orig_y = v.rect.x, v.rect.y
    ship.remove(v)
    if not ship.try_place_randomly(v):
      ship.move(v, orig_x, orig_y)

  elif move == "orient" and ship.vehicles:
    v = random.choice(ship.vehicles)
    new_ori = "vertical" if v.orientation == "horizontal" else
"horizontal"
    try:
      ship.orient(v, new_ori)
    except ValueError:
      pass

  return neighbor
```



```
class SimulatedAnnealing:
  def __init__(
    self,
    state_factory: StateFactory | None = None,
    initial_temperature: float = 1000.0,
    cooling_rate: float = 0.995,
    min_temperature: float = 0.1,
  ) -> None:
    self.state_factory = state_factory
    self.initial_temperature = initial_temperature
    self.cooling_rate = cooling_rate
    self.min_temperature = min_temperature

    self.initial_state: State | None = None
    self.history: list[float] = []
    self.history_probability: list[float] = []
    self.stuck_count = 0
    self.duration = 0.0
    self.iterations = 0

  @property
  def history_objective(self) -> list[float]:
    return self.history

  def run(self, factory: StateFactory | None = None) -> State:
    target_factory = factory or self.state_factory
    if target_factory is None:
      raise ValueError("StateFactory diperlukan.")

    self.initial_state = target_factory.create_random_state()
    current = deepcopy(self.initial_state)
    best = deepcopy(current)
    temperature = self.initial_temperature

    current_val, best_val = objective(current), objective(best)
    self.history = [current_val]
    self.history_probability = [1.0]
    self.stuck_count = 0

    iterations = 0
    start_time = time.perf_counter()
    while temperature > self.min_temperature:
      iterations += 1
      neighbor = _generate_neighbor(current)
      neighbor_val = objective(neighbor)
      delta = neighbor_val - current_val

      prob = 1.0 if delta > 0 else math.exp(delta / temperature)
      if delta > 0 or random.random() < prob:
        current, current_val = neighbor, neighbor_val
        if current_val > best_val:
          best, best_val = deepcopy(current), current_val
      else:
        self.stuck_count += 1

      self.history.append(current_val)
      self.history_probability.append(prob)
      temperature *= self.cooling_rate

    self.iterations = iterations
    self.duration = time.perf_counter() - start_time
    return best
```

Fungsi	`def create_random_state(factory: StateFactory) -> State:`
Deskripsi	Menginisialisasi parameter utama untuk algoritma Simulated Annealing, termasuk objek pabrik state (state_factory), temperatur awal (initial_temperature), laju pendinginan (cooling_rate), serta temperatur minimum (min_temperature). Sekaligus menyiapkan atribut pelacak status, riwayat skor, probabilitas, jumlah

	iterasi terhenti (stuck count), durasi, dan total iterasi.
Output	-

Source Code

```
def create_random_state(factory: StateFactory) ->
State:
  return factory.create_random_state()
```

Fungsi	`def _generate_neighbor(state: State) -> State:`
Deskripsi	Membuat salinan mendalam (deepcopy) dari suatu state, lalu menghasilkan status tetangga (neighbor) secara acak melalui salah satu dari tiga jenis gerakan modifikasi: menukar kendaraan di dalam dan di luar kapal ("swap"), memindahkan posisi kendaraan di dalam kapal ("move"), atau mengubah orientasi rotasi kendaraan ("orient").
Output	Objek instance State baru yang bertindak sebagai tetangga hasil modifikasi acak (State).

Source Code

```
def _generate_neighbor(state: State) -> State:
  neighbor = deepcopy(state)
  ship, outside = neighbor.ship,
neighbor.vehicles_outside
  if not ship.vehicles and not outside:
    return neighbor

  move = random.choice(["swap", "move", "orient"])

  if move == "swap" and (len(ship.vehicles) >= 2 or
(ship.vehicles and outside)):
    if outside and (len(ship.vehicles) < 2 or
random.random() < 0.5):
      v_in, v_out = random.choice(ship.vehicles),
random.choice(outside)
      if ship.total_weight - v_in.weight +
v_out.weight <= ship.max_capacity:
        try:
          ship.replace(v_in, v_out)
          outside.remove(v_out)
          outside.append(v_in)
        except ValueError:
          orig_x, orig_y = v_in.rect.x, v_in.rect.y
          ship.remove(v_in)
          if ship.try_place_randomly(v_out):
            outside.remove(v_out)
            outside.append(v_in)
          else:
            ship.move(v_in, orig_x, orig_y)
    else:
      v1, v2 = random.sample(ship.vehicles, 2)
      x1, y1, x2, y2 = v1.rect.x, v1.rect.y,
v2.rect.x, v2.rect.y
      ship.remove(v1)
      ship.remove(v2)
      try:
        ship.move(v2, x1, y1)
        ship.move(v1, x2, y2)
      except ValueError:
        if v2 in ship.vehicles:
          ship.remove(v2)
        ship.move(v1, x1, y1)
        ship.move(v2, x2, y2)

  elif move == "move" and ship.vehicles:
    v = random.choice(ship.vehicles)
    orig_x, orig_y = v.rect.x, v.rect.y
    ship.remove(v)
    if not ship.try_place_randomly(v):
      ship.move(v, orig_x, orig_y)

  elif move == "orient" and ship.vehicles:
    v = random.choice(ship.vehicles)
    new_ori = "vertical" if v.orientation ==
"horizontal" else "horizontal"
```

```
    try:
      ship.orient(v, new_ori)
    except ValueError:
      pass

  return neighbor
```

Fungsi

```
def __init__(
    self,
    state_factory: StateFactory | None = None,
    initial_temperature: float = 1000.0,
    cooling_rate: float = 0.995,
    min_temperature: float = 0.1,
  ) -> None:
```

Deskripsi	Menginisialisasi parameter utama untuk algoritma Simulated Annealing, termasuk objek pabrik state (state_factory), temperatur awal (initial_temperature), laju pendinginan (cooling_rate), serta temperatur minimum (min_temperature). Sekaligus menyiapkan atribut pelacak status, riwayat skor, probabilitas, jumlah iterasi terhenti (stuck count), durasi, dan total iterasi.
Output	**-**

Source Code

```
def __init__(
    self,
    state_factory: StateFactory | None = None,
    initial_temperature: float = 1000.0,
    cooling_rate: float = 0.995,
    min_temperature: float = 0.1,
  ) -> None:
    self.state_factory = state_factory
    self.initial_temperature = initial_temperature
    self.cooling_rate = cooling_rate
    self.min_temperature = min_temperature

    self.initial_state: State | None = None
    self.history: list[float] = []
    self.history_probability: list[float] = []
    self.stuck_count = 0
    self.duration = 0.0
    self.iterations = 0
```

Fungsi	`def history_objective(self) -> list[float]:`
Deskripsi	Mengambil dan mengembalikan daftar riwayat nilai objektif (objective history) dari proses pencarian atau iterasi yang telah dilakukan.
Output	Berupa sebuah list berisi bilangan pecahan desimal (list[float]) yang merepresentasikan rekam jejak skor objektif.
Source Code	

```
def history_objective(self) -> list[float]:
    return self.history
```

Fungsi

```
 def run(self, factory: StateFactory | None = None)
-> State:
```

Deskripsi	Menjalankan algoritma Random Restart Hill Climbing dengan menginisialisasi status awal (atau membangkitkan random state jika tidak diberikan), lalu melakukan pencarian utama menggunakan Steepest Ascent Hill Climbing secara berulang sebanyak batas maksimum restart (max_restarts). Pada setiap siklusnya, algoritma memantau dan memperbarui skor optimal global terbaik (best

	state), mencatat riwayat iterasi per restart, serta menghitung total durasi waktu eksekusi pencarian.
Output	Mengembalikan objek instance kelas State (State) yang merepresentasikan status akhir dengan nilai objektif atau skor optimal terbaik secara global setelah seluruh proses pencarian restart selesai.
Source Code	

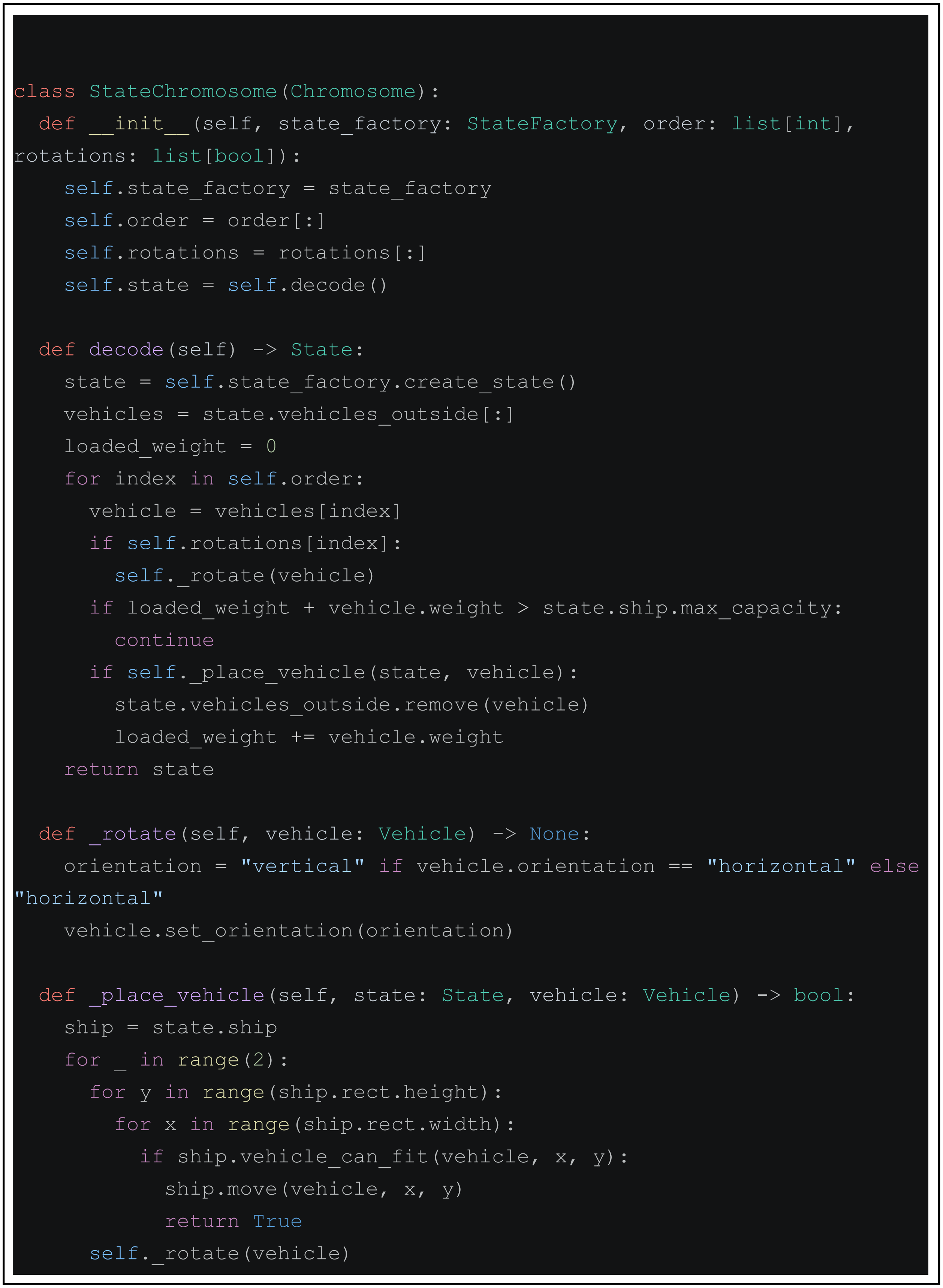
```
  def run(self, factory: StateFactory | None = None)
-> State:
    target_factory = factory or self.state_factory
    if target_factory is None:
      raise ValueError("StateFactory diperlukan.")

    self.initial_state =
target_factory.create_random_state()
    current = deepcopy(self.initial_state)
    best = deepcopy(current)
    temperature = self.initial_temperature

    current_val, best_val = objective(current),
objective(best)
    self.history = [current_val]
    self.history_probability = [1.0]
    self.stuck_count = 0

    iterations = 0
    start_time = time.perf_counter()
    while temperature > self.min_temperature:
      iterations += 1
      neighbor = _generate_neighbor(current)
      neighbor_val = objective(neighbor)
      delta = neighbor_val - current_val

      prob = 1.0 if delta > 0 else math.exp(delta /
temperature)
      if delta > 0 or random.random() < prob:
        current, current_val = neighbor,
neighbor_val
```

```
        if current_val > best_val:
          best, best_val = deepcopy(current),
current_val
      else:
        self.stuck_count += 1

      self.history.append(current_val)
      self.history_probability.append(prob)
      temperature *= self.cooling_rate

    self.iterations = iterations
    self.duration = time.perf_counter() - start_time
    return best
```

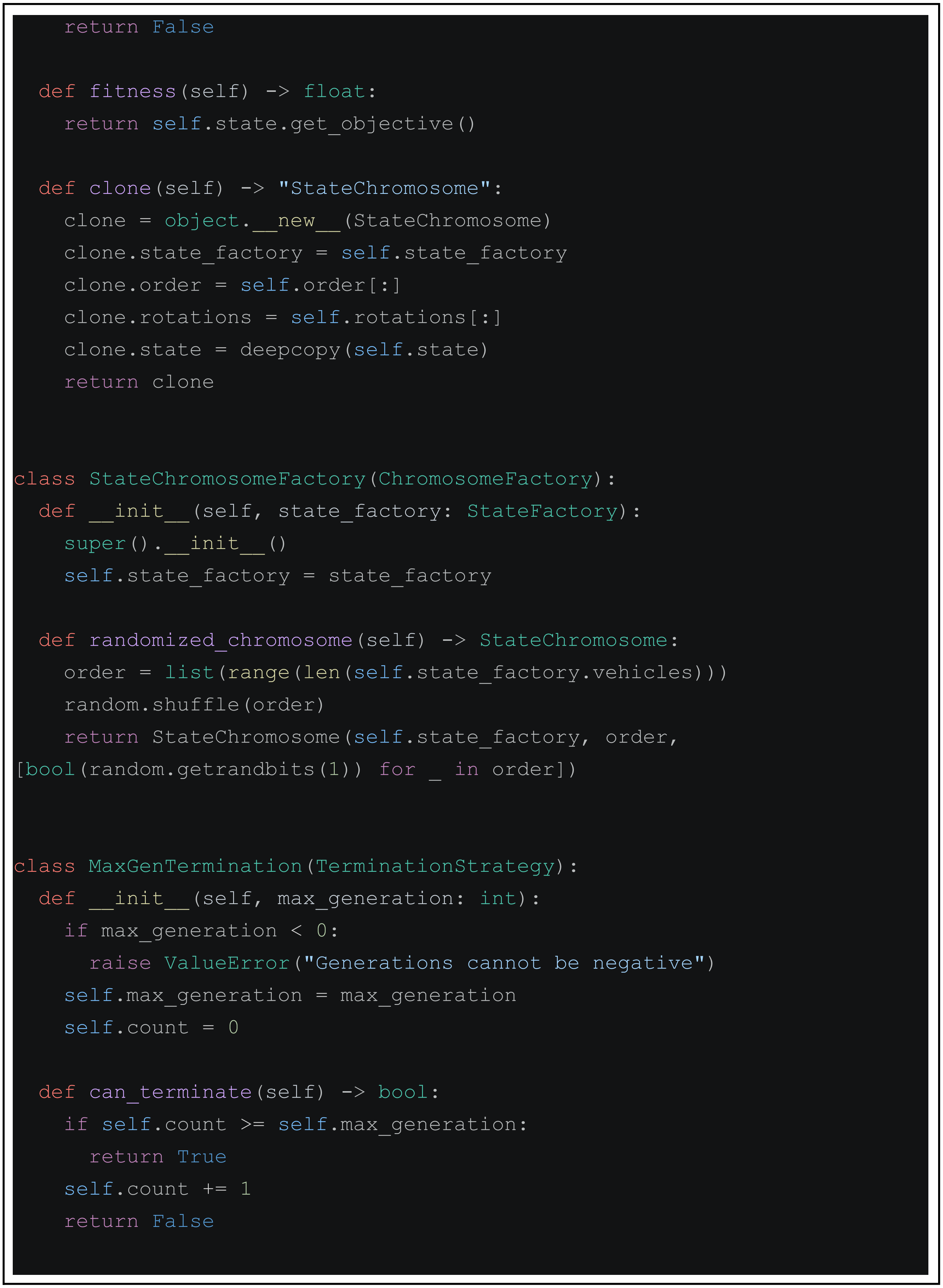
2.2.6. Algoritma *Genetic*

implementation.py

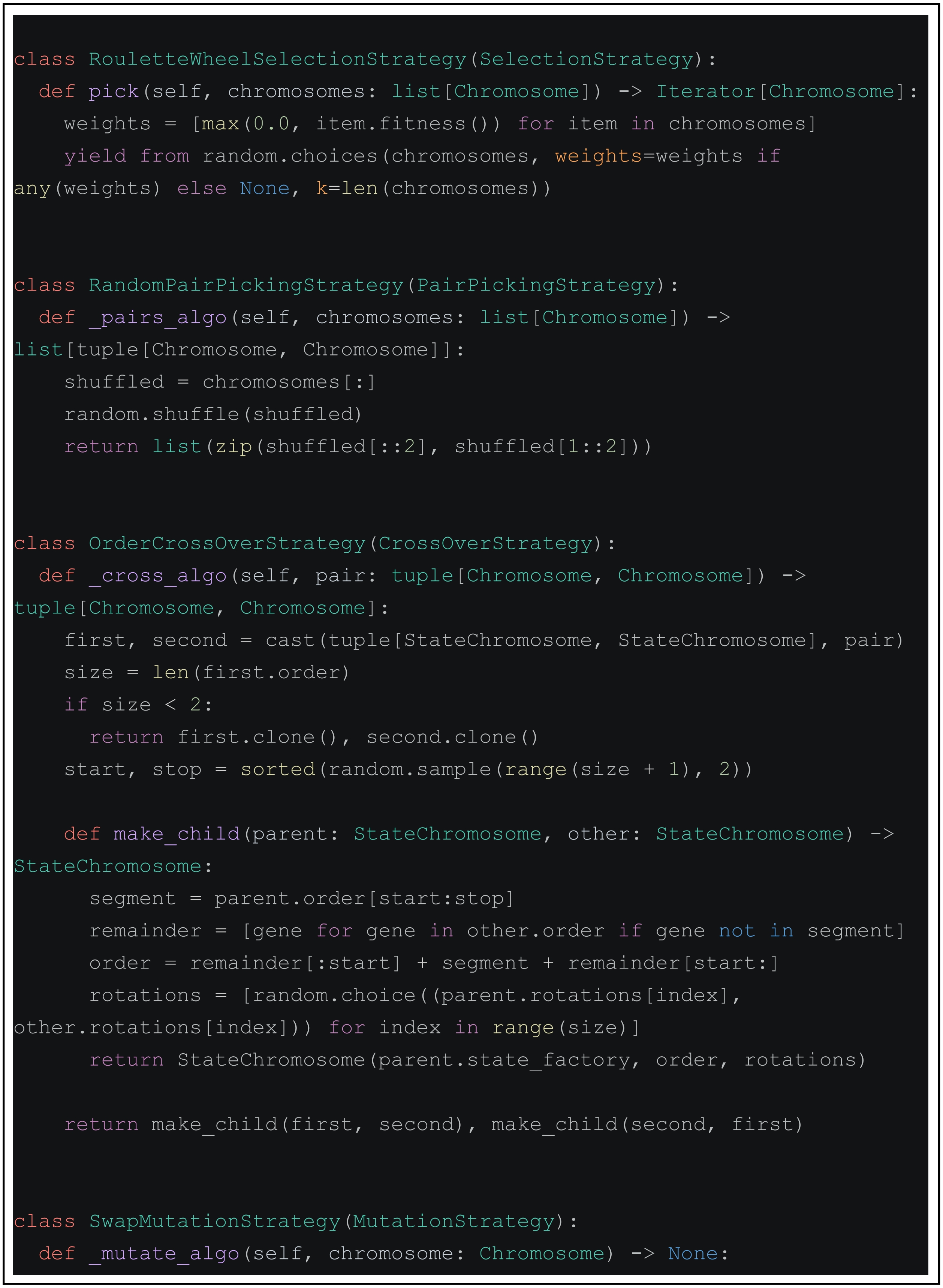
```
from copy import deepcopy
import random
from typing import Iterator, cast

from algo.genetic_algorithm.primitive import (
  Chromosome,
  ChromosomeFactory,
  CrossOverStrategy,
  GeneticAlgorithm,
  MutationStrategy,
  PairPickingStrategy,
  Population,
  SelectionStrategy,
  TerminationStrategy,
)
from objects.state import State, StateFactory
from objects.vehicle import Vehicle
```



```
class StateChromosome(Chromosome):
  def __init__(self, state_factory: StateFactory, order: list[int],
rotations: list[bool]):
    self.state_factory = state_factory
    self.order = order[:]
    self.rotations = rotations[:]
    self.state = self.decode()

  def decode(self) -> State:
    state = self.state_factory.create_state()
    vehicles = state.vehicles_outside[:]
    loaded_weight = 0
    for index in self.order:
      vehicle = vehicles[index]
      if self.rotations[index]:
        self._rotate(vehicle)
      if loaded_weight + vehicle.weight > state.ship.max_capacity:
        continue
      if self._place_vehicle(state, vehicle):
        state.vehicles_outside.remove(vehicle)
        loaded_weight += vehicle.weight
    return state

  def _rotate(self, vehicle: Vehicle) -> None:
    orientation = "vertical" if vehicle.orientation == "horizontal" else
"horizontal"
    vehicle.set_orientation(orientation)

  def _place_vehicle(self, state: State, vehicle: Vehicle) -> bool:
    ship = state.ship
    for _ in range(2):
      for y in range(ship.rect.height):
        for x in range(ship.rect.width):
          if ship.vehicle_can_fit(vehicle, x, y):
            ship.move(vehicle, x, y)
            return True
      self._rotate(vehicle)
```



```
    return False

  def fitness(self) -> float:
    return self.state.get_objective()

  def clone(self) -> "StateChromosome":
    clone = object.__new__(StateChromosome)
    clone.state_factory = self.state_factory
    clone.order = self.order[:]
    clone.rotations = self.rotations[:]
    clone.state = deepcopy(self.state)
    return clone


class StateChromosomeFactory(ChromosomeFactory):
  def __init__(self, state_factory: StateFactory):
    super().__init__()
    self.state_factory = state_factory

  def randomized_chromosome(self) -> StateChromosome:
    order = list(range(len(self.state_factory.vehicles)))
    random.shuffle(order)
    return StateChromosome(self.state_factory, order,
[bool(random.getrandbits(1)) for _ in order])


class MaxGenTermination(TerminationStrategy):
  def __init__(self, max_generation: int):
    if max_generation < 0:
      raise ValueError("Generations cannot be negative")
    self.max_generation = max_generation
    self.count = 0

  def can_terminate(self) -> bool:
    if self.count >= self.max_generation:
      return True
    self.count += 1
    return False
```



```
class RouletteWheelSelectionStrategy(SelectionStrategy):
  def pick(self, chromosomes: list[Chromosome]) -> Iterator[Chromosome]:
    weights = [max(0.0, item.fitness()) for item in chromosomes]
    yield from random.choices(chromosomes, weights=weights if
any(weights) else None, k=len(chromosomes))


class RandomPairPickingStrategy(PairPickingStrategy):
  def _pairs_algo(self, chromosomes: list[Chromosome]) ->
list[tuple[Chromosome, Chromosome]]:
    shuffled = chromosomes[:]
    random.shuffle(shuffled)
    return list(zip(shuffled[::2], shuffled[1::2]))


class OrderCrossOverStrategy(CrossOverStrategy):
  def _cross_algo(self, pair: tuple[Chromosome, Chromosome]) ->
tuple[Chromosome, Chromosome]:
    first, second = cast(tuple[StateChromosome, StateChromosome], pair)
    size = len(first.order)
    if size < 2:
      return first.clone(), second.clone()
    start, stop = sorted(random.sample(range(size + 1), 2))

    def make_child(parent: StateChromosome, other: StateChromosome) ->
StateChromosome:
      segment = parent.order[start:stop]
      remainder = [gene for gene in other.order if gene not in segment]
      order = remainder[:start] + segment + remainder[start:]
      rotations = [random.choice((parent.rotations[index],
other.rotations[index])) for index in range(size)]
      return StateChromosome(parent.state_factory, order, rotations)

    return make_child(first, second), make_child(second, first)


class SwapMutationStrategy(MutationStrategy):
  def _mutate_algo(self, chromosome: Chromosome) -> None:
    state_chromosome = cast(StateChromosome, chromosome)
    if not state_chromosome.order:
      return
    if len(state_chromosome.order) >= 2:
      first, second = random.sample(range(len(state_chromosome.order)), 2)
      order = state_chromosome.order
      order[first], order[second] = order[second], order[first]
    index = random.randrange(len(state_chromosome.order))
    state_chromosome.rotations[index] = not state_chromosome.rotations[index]
    state_chromosome.state = state_chromosome.decode()


class GARunner:
  def __init__(self, state_factory: StateFactory, population_size: int = 16, generations: int = 100, crossover_rate: float = 0.85, mutation_rate: float = 0.25):
    factory = StateChromosomeFactory(state_factory)
    population = Population(population_size, factory)
    termination = MaxGenTermination(generations)
    crossover = OrderCrossOverStrategy(crossover_rate)
    selection = RouletteWheelSelectionStrategy()
    pairing = RandomPairPickingStrategy()
    mutation = SwapMutationStrategy(mutation_rate)
    self.genetic_algorithm = GeneticAlgorithm(population, termination, crossover, selection, pairing, mutation)

  @property
  def history(self) -> list[float]:
    return self.genetic_algorithm.history

  def run(self) -> State:
    return cast(StateChromosome, self.genetic_algorithm.run()).state
```

Fungsi	`def __init__(self, state_factory: StateFactory, order: list[int], rotations: list[bool])`
Deskripsi	Menginisialisasi kromosom dengan menyimpan state factory, salinan daftar urutan pemuatan kendaraan (order), dan salinan daftar penanda rotasi tiap kendaraan (rotations), lalu langsung memanggil decode() untuk membangun state penataan kapal dari kromosom tersebut.
Output	**-**
Source Code	

```
def __init__(self, state_factory: StateFactory,
order: list[int], rotations: list[bool]):
    self.state_factory = state_factory
    self.order = order[:]
    self.rotations = rotations[:]
    self.state = self.decode()
```

Fungsi	`def decode(self) -> State:`
Deskripsi	Menerjemahkan kromosom menjadi state penataan kapal. Membuat state kosong, lalu memproses kendaraan sesuai urutan pada order. Kendaraan dirotasi jika penanda rotasinya True, dilewati jika total berat akan melebihi kapasitas kapal, dan jika masih memenuhi, ditempatkan dengan _place_vehicle. Kendaraan yang berhasil dimuat dihapus dari daftar kendaraan di luar kapal dan beratnya ditambahkan ke total muatan.
Output	Objek State hasil penataan kendaraan berdasarkan kromosom.
Source Code	

```
def decode(self) -> State:
    state = self.state_factory.create_state()
    vehicles = state.vehicles_outside[:]
    loaded_weight = 0
    for index in self.order:
      vehicle = vehicles[index]
      if self.rotations[index]:
        self._rotate(vehicle)
      if loaded_weight + vehicle.weight >
state.ship.max_capacity:
        continue
      if self._place_vehicle(state, vehicle):
        state.vehicles_outside.remove(vehicle)
        loaded_weight += vehicle.weight
    return state
```

Fungsi

```
def _rotate(self, vehicle: Vehicle) -> None:
```

Deskripsi	Fungsi pembantu untuk membalik orientasi kendaraan (horizontal menjadi vertical atau sebaliknya) melalui set_orientation, sehingga lebar dan panjang kendaraan tertukar.
Output	-

Source Code

```
  def _rotate(self, vehicle: Vehicle) -> None:
    orientation = "vertical" if vehicle.orientation
== "horizontal" else "horizontal"
    vehicle.set_orientation(orientation)
```

Fungsi

```
def _place_vehicle(self, state: State, vehicle:
Vehicle) -> bool:
```

Deskripsi	Mencari posisi valid pertama untuk kendaraan dengan memindai grid kapal mulai dari baris paling

	atas dan kolom paling kiri. Jika ditemukan, kendaraan dipindahkan ke posisi itu. Jika tidak ada posisi valid, kendaraan dirotasi lalu pemindaian diulang, sehingga kedua orientasi dicoba.
Output	True jika kendaraan berhasil ditempatkan, False jika tidak muat pada kedua orientasi.

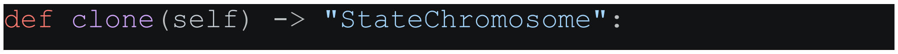
Source Code

```python
def _place_vehicle(self, state: State, vehicle: Vehicle) -> bool:
    ship = state.ship
    for _ in range(2):
      for y in range(ship.rect.height):
        for x in range(ship.rect.width):
          if ship.vehicle_can_fit(vehicle, x, y):
            ship.move(vehicle, x, y)
            return True
      self._rotate(vehicle)
    return False
```

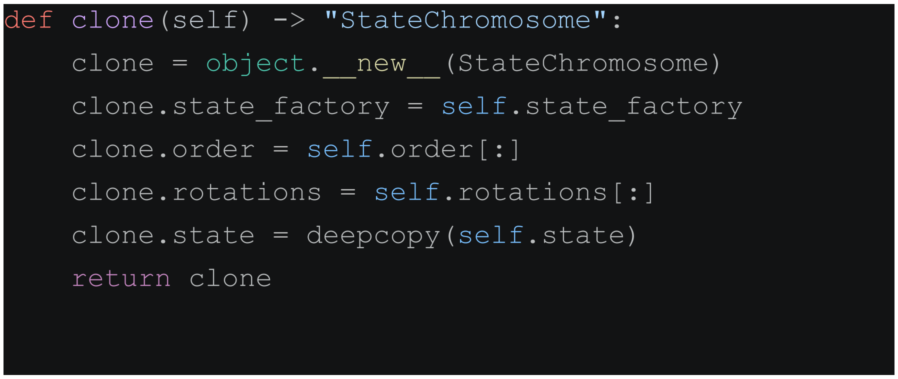
Fungsi	`def fitness(self) -> float:`
Deskripsi	Menghitung dan mengembalikan nilai fitness dari kromosom saat ini berdasarkan hasil pemanggilan fungsi objektif (objective value) dari state yang didekode.
Output	Bilangan pecahan desimal (float) yang merepresentasikan nilai fitness kromosom.

Source Code

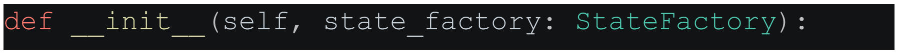
```python
def fitness(self) -> float:
    return self.state.get_objective()
```

Fungsi	 `def clone(self) -> "StateChromosome":`
Deskripsi	Membuat dan mengembalikan salinan mendalam (deep copy) dari objek StateChromosome saat ini, termasuk menyalin state factory, urutan gen (order), rotasi, dan objek state-nya.
Output	Objek instance baru dari kelas StateChromosome ("StateChromosome").

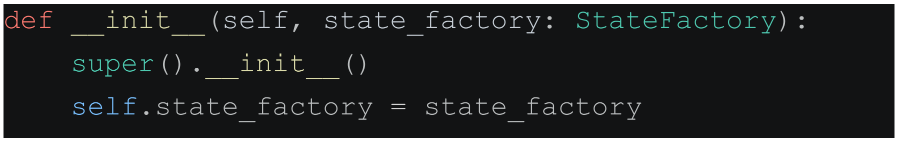
Source Code



```
def clone(self) -> "StateChromosome":
    clone = object.__new__(StateChromosome)
    clone.state_factory = self.state_factory
    clone.order = self.order[:]
    clone.rotations = self.rotations[:]
    clone.state = deepcopy(self.state)
    return clone
```

Fungsi	 `def __init__(self, state_factory: StateFactory):`
Deskripsi	Menginisialisasi chromosome factory dengan menerima objek state factory sebagai acuan untuk membangun dan mengelola kromosom.
Output	**-**

Source Code



```
def __init__(self, state_factory: StateFactory):
    super().__init__()
    self.state_factory = state_factory
```

Fungsi	`def randomized_chromosome(self) -> StateChromosome:`
Deskripsi	Membangkitkan dan mengembalikan sebuah kromosom acak baru (StateChromosome) dengan mengacak urutan indeks kendaraan serta menentukan status rotasi secara acak menggunakan nilai boolean.
Output	Objek instance kromosom acak
Source Code	

```
def randomized_chromosome(self) -> StateChromosome:
    order =
list(range(len(self.state_factory.vehicles)))
    random.shuffle(order)
    return StateChromosome(self.state_factory,
order, [bool(random.getrandbits(1)) for _ in order])
```

Fungsi	`def __init__(self, max_generation: int):`
Deskripsi	Menginisialisasi strategi terminasi berdasarkan batas maksimum generasi (max_generation). Memvalidasi agar nilai generasi tidak negatif (melempar ValueError jika kurang dari 0) serta menyetel penghitung iterasi ke 0.
Output	**-**
Source Code	

```
def __init__(self, max_generation: int):
    if max_generation < 0:
      raise ValueError("Generations cannot be
negative")
    self.max_generation = max_generation
    self.count = 0
```

Fungsi	`def can_terminate(self) -> bool:`
Deskripsi	Memeriksa apakah jumlah iterasi saat ini telah mencapai atau melampaui batas maksimum generasi yang ditentukan. Jika sudah, mengembalikan True; jika belum, menaikkan counter generasi sebesar 1 dan mengembalikan False.
Output	**True jika iterasi harus berhenti, False jika belum.**
Source Code	

```
def can_terminate(self) -> bool:
    if self.count >= self.max_generation:
      return True
    self.count += 1
    return False
```

Fungsi	`def pick(self, chromosomes: list[Chromosome]) -> Iterator[Chromosome]:`
Deskripsi	Memilih sekumpulan kromosom dari populasi menggunakan metode seleksi roda rolet (roulette wheel selection) berdasarkan bobot nilai fitness masing-masing kromosom.
Output	Iterator yang menghasilkan pilihan kromosom
Source Code	

```
def pick(self, chromosomes: list[Chromosome]) ->
Iterator[Chromosome]:
    weights = [max(0.0, item.fitness()) for item in
chromosomes]
    yield from random.choices(chromosomes,
weights=weights if any(weights) else None,
k=len(chromosomes))
```

Fungsi	`def _pairs_algo(self, chromosomes: list[Chromosome]) -> list[tuple[Chromosome, Chromosome]]:`
Deskripsi	Mengacak urutan daftar kromosom dalam populasi, lalu memasangkan kromosom-kromosom tersebut secara berpasangan (pairs) untuk tahap crossover.
Output	Daftar pasangan kromosom dalam bentuk list of tuples (list[tuple[Chromosome, Chromosome]]).

Source Code

```
def _pairs_algo(self, chromosomes: list[Chromosome])
-> list[tuple[Chromosome, Chromosome]]:
    shuffled = chromosomes[:]
    random.shuffle(shuffled)
    return list(zip(shuffled[::2], shuffled[1::2]))
```

Fungsi	`def _cross_algo(self, pair: tuple[Chromosome, Chromosome]) -> tuple[Chromosome, Chromosome]:`
Deskripsi	Melakukan algoritma pindah silang (crossover) menggunakan strategi berbasis urutan (order crossover) pada sepasang kromosom orang tua untuk menghasilkan dua kromosom anak yang baru.
Output	Sepasang kromosom anak (tuple of two chromosomes) (tuple[Chromosome, Chromosome]).

Source Code

```
def _cross_algo(self, pair: tuple[Chromosome,
Chromosome]) -> tuple[Chromosome, Chromosome]:
    first, second = cast(tuple[StateChromosome,
StateChromosome], pair)
    size = len(first.order)
    if size < 2:
      return first.clone(), second.clone()
    start, stop = sorted(random.sample(range(size +
1), 2))
```

Fungsi	`def make_child(parent: StateChromosome, other: StateChromosome) -> StateChromosome:`
Deskripsi	Fungsi pembantu (helper) untuk membentuk satu kromosom anak dengan mengambil segmen acak dari kromosom orang tua pertama dan melengkapinya dengan sisa gen dari orang tua kedua, serta menggabungkan status rotasi keduanya secara acak.
Output	Objek instance kromosom anak (child chromosome)

Source Code

```
def make_child(parent: StateChromosome, other:
StateChromosome) -> StateChromosome:
      segment = parent.order[start:stop]
      remainder = [gene for gene in other.order if
gene not in segment]
      order = remainder[:start] + segment +
remainder[start:]
      rotations =
[random.choice((parent.rotations[index],
other.rotations[index])) for index in range(size)]
      return StateChromosome(parent.state_factory,
order, rotations)

    return make_child(first, second),
make_child(second, first)
```


Fungsi	`def _mutate_algo(self, chromosome: Chromosome) -> None:`
Deskripsi	Menerapkan algoritma mutasi pada kromosom dengan menukar posisi dua gen secara acak (swap mutation) dalam urutan kromosom, serta membalikkan status rotasi pada salah satu indeks secara acak, lalu mendekode ulang state kromosom tersebut.
Output	-

Source Code

```python
def _mutate_algo(self, chromosome: Chromosome) -> None:
    state_chromosome = cast(StateChromosome, chromosome)
    if not state_chromosome.order:
      return
    if len(state_chromosome.order) >= 2:
      first, second = random.sample(range(len(state_chromosome.order)), 2)
      order = state_chromosome.order
      order[first], order[second] = order[second], order[first]
    index = random.randrange(len(state_chromosome.order))
    state_chromosome.rotations[index] = not state_chromosome.rotations[index]
    state_chromosome.state = state_chromosome.decode()
```

Fungsi	`def __init__(self, state_factory: StateFactory,`

	`population_size: int = 16, generations: int = 100, crossover_rate: float = 0.85, mutation_rate: float = 0.25):`
Deskripsi	Menginisialisasi pengelola utama algoritma genetika (GARunner) dengan parameter pabrik state, ukuran populasi, jumlah generasi, laju crossover, dan laju mutasi. Menyiapkan seluruh komponen pendukung seperti pabrik kromosom, populasi, strategi terminasi, crossover, seleksi, pairing, dan mutasi.
Output	-
Source Code	

```python
def __init__(self, state_factory: StateFactory, population_size: int = 16, generations: int = 100, crossover_rate: float = 0.85, mutation_rate: float = 0.25):
    factory = StateChromosomeFactory(state_factory)
    population = Population(population_size, factory)
    termination = MaxGenTermination(generations)
    crossover = OrderCrossOverStrategy(crossover_rate)
    selection = RouletteWheelSelectionStrategy()
    pairing = RandomPairPickingStrategy()
    mutation = SwapMutationStrategy(mutation_rate)
    self.genetic_algorithm = GeneticAlgorithm(population, termination, crossover, selection, pairing, mutation)
```

Fungsi	`def history(self) -> list[float]:`
Deskripsi	Mengambil data rekam jejak nilai fitness terbaik pada setiap generasi dari proses algoritma genetika.

Output	Daftar nilai pecahan desimal (list[float]) yang berisi riwayat skor fitness terbaik per generasi.
Source Code	

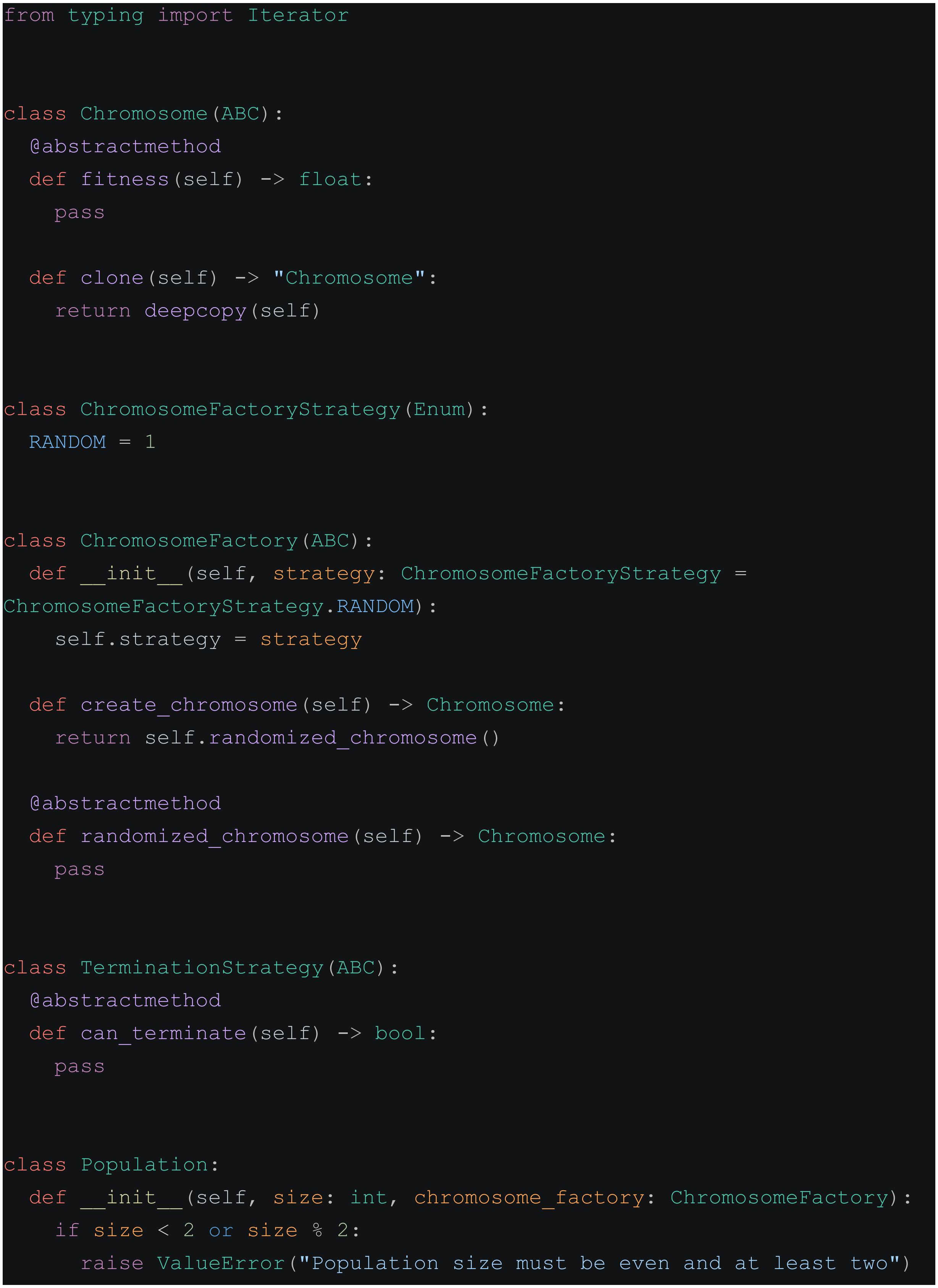
```python
def history(self) -> list[float]:
    return self.genetic_algorithm.history
```

Fungsi	`def run(self) -> State:`
Deskripsi	Menjalankan proses utama algoritma genetika dari awal hingga kondisi terminasi terpenuhi, mengukur durasi waktu eksekusi, lalu mengembalikan objek state hasil optimasi terbaik dari kromosom akhir.
Output	Objek instance kelas State (State) yang merepresentasikan konfigurasi penempatan muatan terbaik hasil optimasi algoritma genetika.
Source Code	

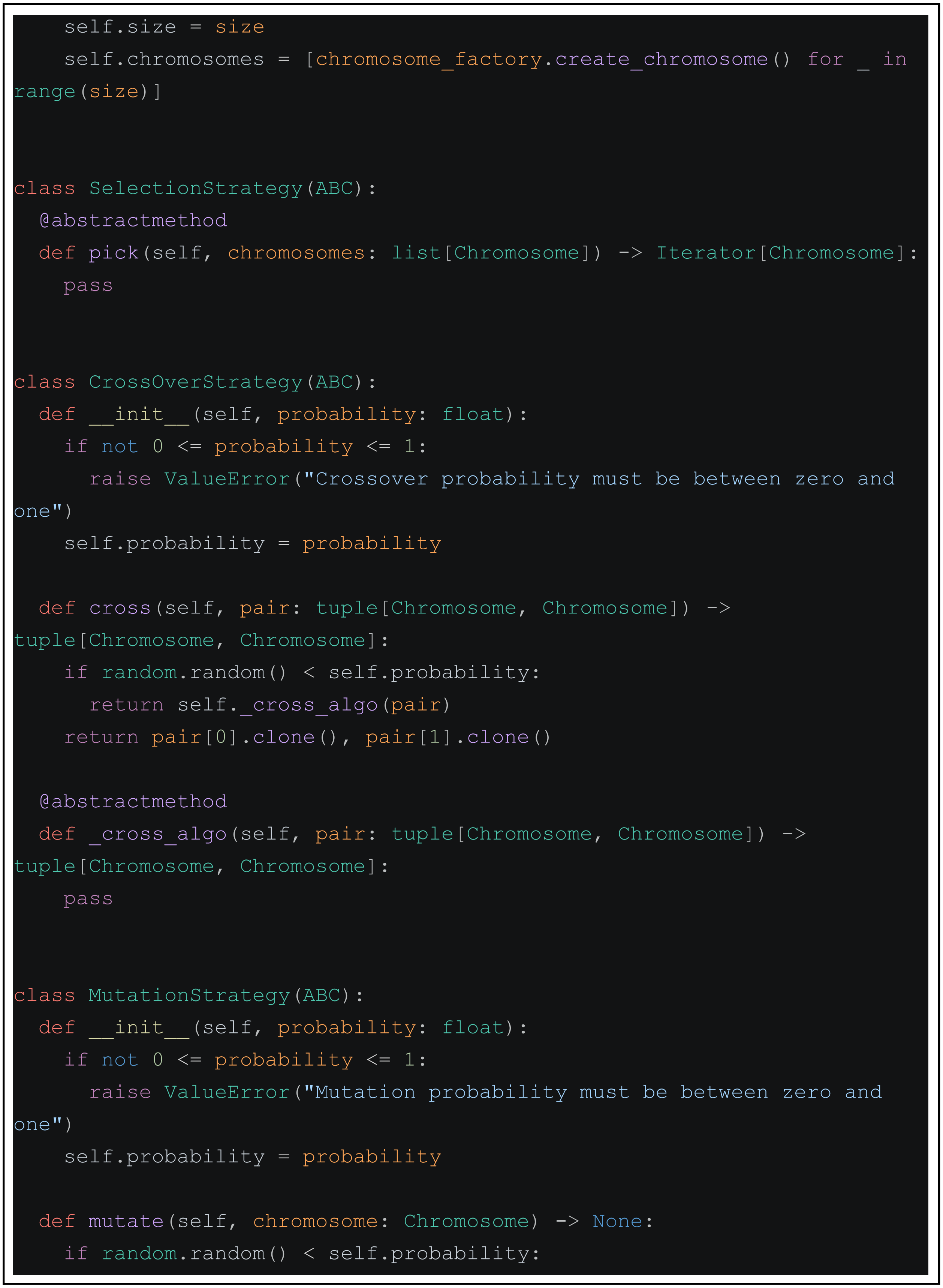
```python
def run(self) -> State:
    return cast(StateChromosome,
self.genetic_algorithm.run()).state
```

primitive.py

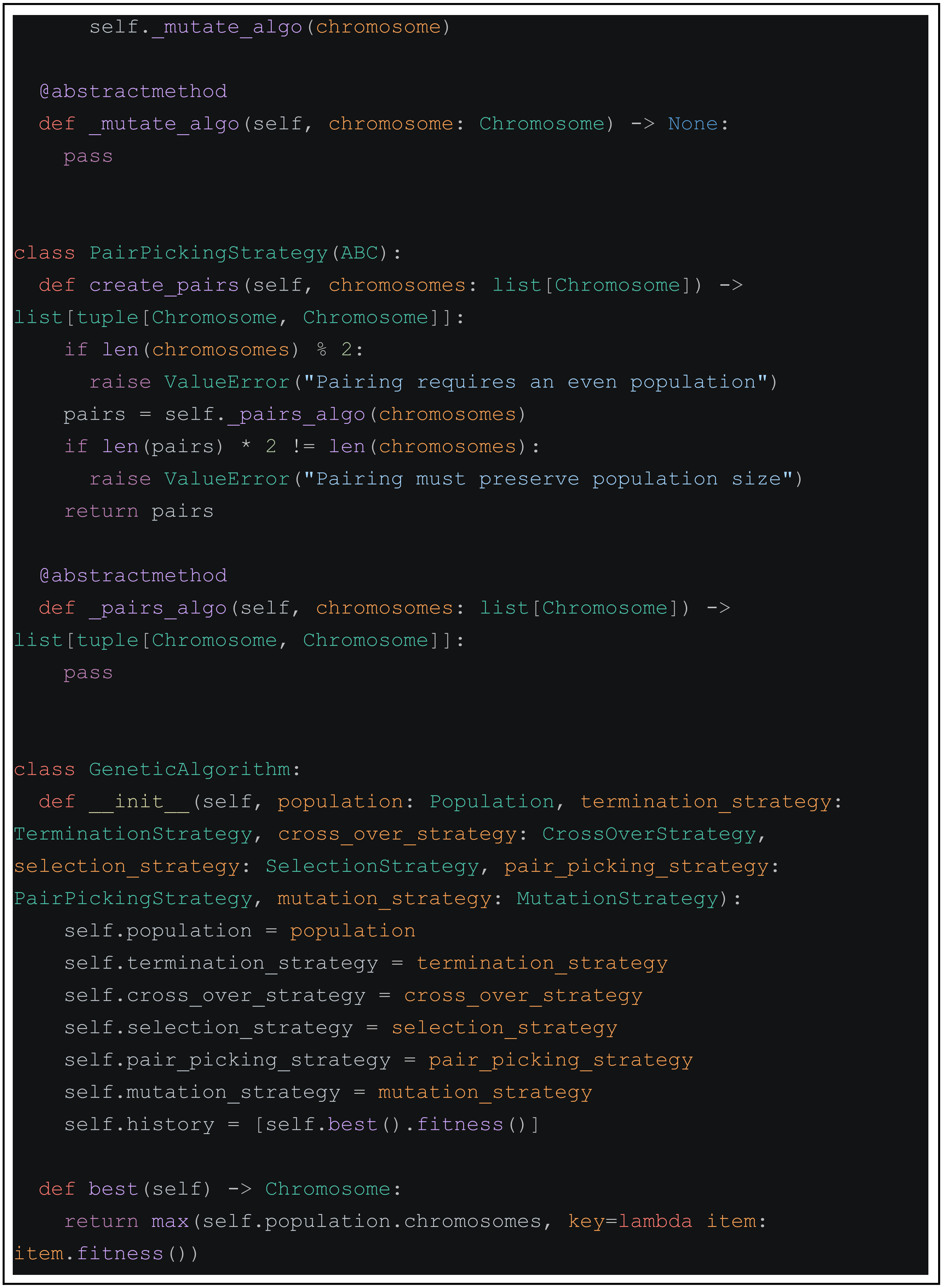
```python
from abc import ABC, abstractmethod
from copy import deepcopy
from enum import Enum
import random
```



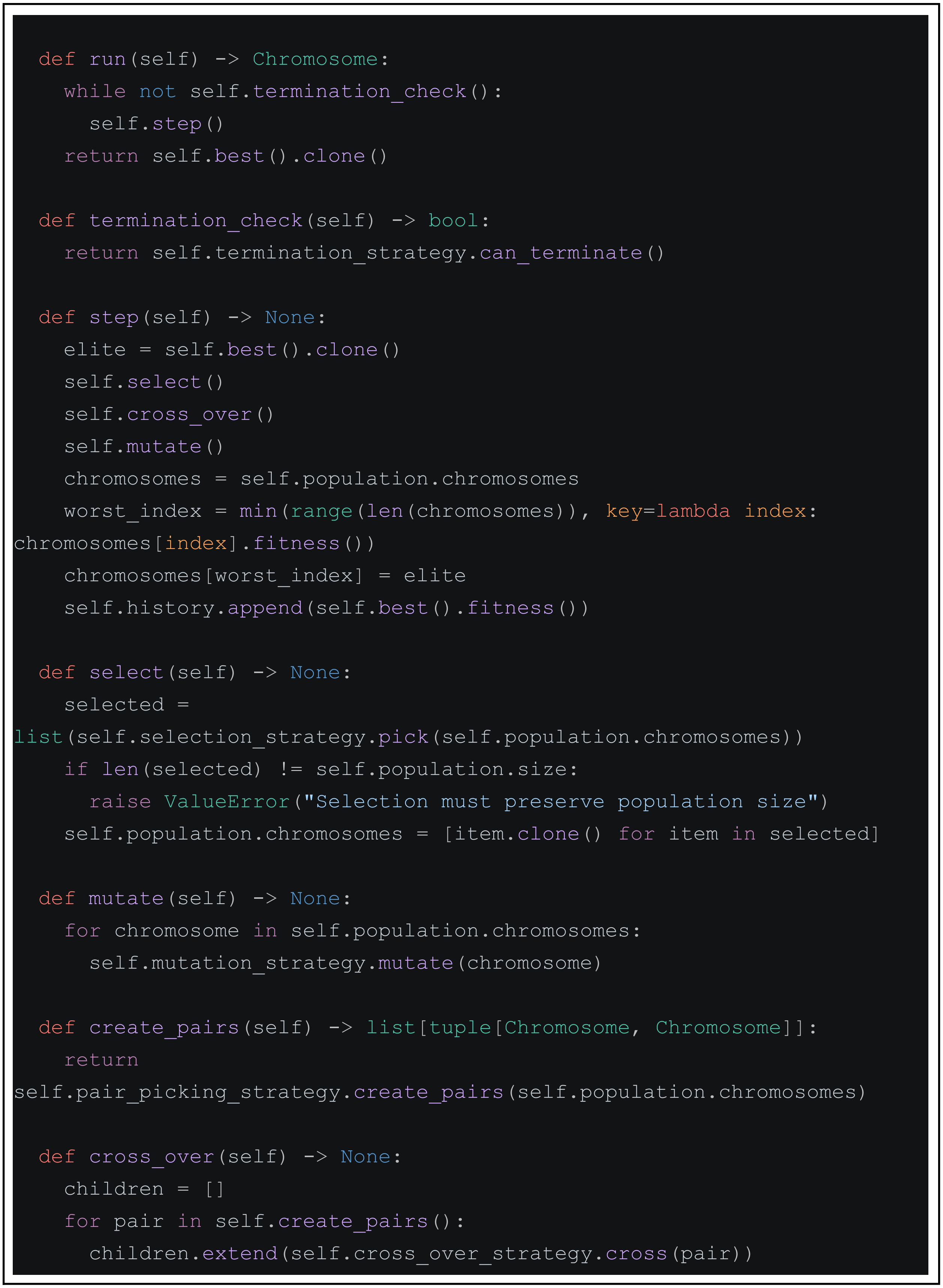
```python
from typing import Iterator


class Chromosome(ABC):
  @abstractmethod
  def fitness(self) -> float:
    pass

  def clone(self) -> "Chromosome":
    return deepcopy(self)


class ChromosomeFactoryStrategy(Enum):
  RANDOM = 1


class ChromosomeFactory(ABC):
  def __init__(self, strategy: ChromosomeFactoryStrategy =
ChromosomeFactoryStrategy.RANDOM):
    self.strategy = strategy

  def create_chromosome(self) -> Chromosome:
    return self.randomized_chromosome()

  @abstractmethod
  def randomized_chromosome(self) -> Chromosome:
    pass


class TerminationStrategy(ABC):
  @abstractmethod
  def can_terminate(self) -> bool:
    pass


class Population:
  def __init__(self, size: int, chromosome_factory: ChromosomeFactory):
    if size < 2 or size % 2:
      raise ValueError("Population size must be even and at least two")
```



```python
    self.size = size
    self.chromosomes = [chromosome_factory.create_chromosome() for _ in range(size)]


class SelectionStrategy(ABC):
  @abstractmethod
  def pick(self, chromosomes: list[Chromosome]) -> Iterator[Chromosome]:
    pass


class CrossOverStrategy(ABC):
  def __init__(self, probability: float):
    if not 0 <= probability <= 1:
      raise ValueError("Crossover probability must be between zero and one")
    self.probability = probability

  def cross(self, pair: tuple[Chromosome, Chromosome]) -> tuple[Chromosome, Chromosome]:
    if random.random() < self.probability:
      return self._cross_algo(pair)
    return pair[0].clone(), pair[1].clone()

  @abstractmethod
  def _cross_algo(self, pair: tuple[Chromosome, Chromosome]) -> tuple[Chromosome, Chromosome]:
    pass


class MutationStrategy(ABC):
  def __init__(self, probability: float):
    if not 0 <= probability <= 1:
      raise ValueError("Mutation probability must be between zero and one")
    self.probability = probability

  def mutate(self, chromosome: Chromosome) -> None:
    if random.random() < self.probability:
```



```
      self._mutate_algo(chromosome)

  @abstractmethod
  def _mutate_algo(self, chromosome: Chromosome) -> None:
    pass


class PairPickingStrategy(ABC):
  def create_pairs(self, chromosomes: list[Chromosome]) ->
list[tuple[Chromosome, Chromosome]]:
    if len(chromosomes) % 2:
      raise ValueError("Pairing requires an even population")
    pairs = self._pairs_algo(chromosomes)
    if len(pairs) * 2 != len(chromosomes):
      raise ValueError("Pairing must preserve population size")
    return pairs

  @abstractmethod
  def _pairs_algo(self, chromosomes: list[Chromosome]) ->
list[tuple[Chromosome, Chromosome]]:
    pass


class GeneticAlgorithm:
  def __init__(self, population: Population, termination_strategy:
TerminationStrategy, cross_over_strategy: CrossOverStrategy,
selection_strategy: SelectionStrategy, pair_picking_strategy:
PairPickingStrategy, mutation_strategy: MutationStrategy):
    self.population = population
    self.termination_strategy = termination_strategy
    self.cross_over_strategy = cross_over_strategy
    self.selection_strategy = selection_strategy
    self.pair_picking_strategy = pair_picking_strategy
    self.mutation_strategy = mutation_strategy
    self.history = [self.best().fitness()]

  def best(self) -> Chromosome:
    return max(self.population.chromosomes, key=lambda item:
item.fitness())
```



```
  def run(self) -> Chromosome:
    while not self.termination_check():
      self.step()
    return self.best().clone()

  def termination_check(self) -> bool:
    return self.termination_strategy.can_terminate()

  def step(self) -> None:
    elite = self.best().clone()
    self.select()
    self.cross_over()
    self.mutate()
    chromosomes = self.population.chromosomes
    worst_index = min(range(len(chromosomes)), key=lambda index:
chromosomes[index].fitness())
    chromosomes[worst_index] = elite
    self.history.append(self.best().fitness())

  def select(self) -> None:
    selected =
list(self.selection_strategy.pick(self.population.chromosomes))
    if len(selected) != self.population.size:
      raise ValueError("Selection must preserve population size")
    self.population.chromosomes = [item.clone() for item in selected]

  def mutate(self) -> None:
    for chromosome in self.population.chromosomes:
      self.mutation_strategy.mutate(chromosome)

  def create_pairs(self) -> list[tuple[Chromosome, Chromosome]]:
    return
self.pair_picking_strategy.create_pairs(self.population.chromosomes)

  def cross_over(self) -> None:
    children = []
    for pair in self.create_pairs():
      children.extend(self.cross_over_strategy.cross(pair))
    self.population.chromosomes = children
```

Fungsi	`def fitness(self) -> float:`
Deskripsi	Metode abstrak/dasar untuk menghitung nilai fitness dari suatu kromosom (didefinisikan ulang pada kelas turunan).
Output	Bilangan pecahan desimal (float) yang merepresentasikan nilai fitness.

Source Code

```
def fitness(self) -> float:
    pass
```

Fungsi	`def clone(self) -> "Chromosome":`
Deskripsi	Membuat salinan mendalam (deep copy) dari objek kromosom saat ini agar tidak mengubah referensi aslinya.
Output	Objek instance kromosom baru yang merupakan salinan ("Chromosome").

Source Code

```
def clone(self) -> "Chromosome":
    return deepcopy(self)
```

Fungsi	`def __init__(self, strategy: ChromosomeFactoryStrategy = ChromosomeFactoryStrategy.RANDOM):`
Deskripsi	Menginisialisasi chromosome factory dengan

	menetapkan strategi pembuatan kromosom (secara default menggunakan ChromosomeFactoryStrategy.RANDOM).
Output	-

Source Code

```python
def __init__(self, strategy: ChromosomeFactoryStrategy = ChromosomeFactoryStrategy.RANDOM):
    self.strategy = strategy
```

Fungsi	`def create_chromosome(self) -> Chromosome:`
Deskripsi	Memanggil dan mengembalikan proses pembuatan kromosom baru secara acak menggunakan metode pabrik (*randomized chromosome*).
Output	Objek instance kromosom yang baru dibuat (Chromosome)

Source Code

```python
def create_chromosome(self) -> Chromosome:
    return self.randomized_chromosome()
```

Fungsi	`def randomized_chromosome(self) -> Chromosome:`
Deskripsi	Metode dasar untuk membangkitkan kromosom acak (diimplementasikan pada kelas konkret).
Output	Objek instance kromosom.

Source Code

```python
def randomized_chromosome(self) -> Chromosome:
    pass
```

Fungsi

```python
def can_terminate(self) -> bool:
```

Deskripsi	Metode dasar untuk mengevaluasi apakah kondisi penghentian algoritma genetika sudah terpenuhi.
Output	boolean

Source Code

```python
def can_terminate(self) -> bool:
    pass
```

Fungsi

```python
def __init__(self, size: int, chromosome_factory: ChromosomeFactory):
    if size < 2 or size % 2:
```

Deskripsi	Menginisialisasi populasi dengan ukuran tertentu (size). Melakukan validasi apabila ukuran populasi kurang dari 2 atau bernilai ganjil, maka akan melempar ValueError. Sekaligus membangkitkan daftar kromosom awal melalui factory.
Output	-

Source Code

```python
def __init__(self, size: int, chromosome_factory: ChromosomeFactory):
    if size < 2 or size % 2:
      raise ValueError("Population size must be even and at least two")
    self.size = size
    self.chromosomes = [chromosome_factory.create_chromosome() for _ in range(size)]
```

Fungsi

```python
def pick(self, chromosomes: list[Chromosome]) -> Iterator[Chromosome]:
```

Deskripsi	Metode dasar untuk memilih kromosom dari populasi berdasarkan strategi seleksi tertentu.
Output	Iterator yang menghasilkan kromosom pilihan

Source Code

```
def pick(self, chromosomes: list[Chromosome]) ->
Iterator[Chromosome]:
    pass
```

Fungsi	`def __init__(self, probability: float):`
Deskripsi	Menginisialisasi strategi crossover dengan parameter probabilitas. Melakukan validasi agar nilai probabilitas berada di antara 0 dan 1 (melempar ValueError jika di luar rentang tersebut).
Output	-

Source Code

```
def __init__(self, probability: float):
    if not 0 <= probability <= 1:
      raise ValueError("Crossover probability must
be between zero and one")
    self.probability = probability
```

Fungsi	`def cross(self, pair: tuple[Chromosome, Chromosome]) -> tuple[Chromosome, Chromosome]:`
Deskripsi	Melakukan proses pindah silang (crossover) pada sepasang kromosom berdasarkan probabilitas yang ditentukan. Jika acak kurang dari probabilitas, algoritma crossover dijalankan; jika tidak, mengembalikan salinan kromosom aslinya
Output	Sepasang kromosom hasil crossover atau salinannya

	(tuple[Chromosome, Chromosome]).
Source Code	

```
def cross(self, pair: tuple[Chromosome, Chromosome]) -> tuple[Chromosome, Chromosome]:
    if random.random() < self.probability:
      return self._cross_algo(pair)
    return pair[0].clone(), pair[1].clone()
```

Fungsi	`def _cross_algo(self, pair: tuple[Chromosome, Chromosome]) -> tuple[Chromosome, Chromosome]:`
Deskripsi	Metode dasar internal untuk mengeksekusi algoritma utama pindah silang (crossover) pada sepasang kromosom.
Output	Sepasang kromosom hasil modifikasi (tuple[Chromosome, Chromosome]).
Source Code	

```
def _cross_algo(self, pair: tuple[Chromosome, Chromosome]) -> tuple[Chromosome, Chromosome]:
    pass
```

Fungsi	`def __init__(self, probability: float):`
Deskripsi	Menginisialisasi strategi mutasi dengan parameter probabilitas mutasi. Memvalidasi agar nilai probabilitas berada di antara 0 dan 1 (melempar ValueError jika tidak valid).
Output	-
Source Code	

```
def __init__(self, probability: float):
    if not 0 <= probability <= 1:
      raise ValueError("Mutation probability must be between zero and one")
    self.probability = probability
```

Fungsi	`def mutate(self, chromosome: Chromosome) -> None:`
Deskripsi	Menerapkan mutasi pada kromosom jika nilai acak memenuhi probabilitas mutasi yang ditetapkan.
Output	-
Source Code	

```
def mutate(self, chromosome: Chromosome) -> None:
    if random.random() < self.probability:
      self._mutate_algo(chromosome)
```

Fungsi	`def _mutate_algo(self, chromosome: Chromosome) -> None:`
Deskripsi	Metode dasar internal untuk menjalankan algoritma mutasi spesifik pada suatu kromosom.
Output	-
Source Code	

```
def _mutate_algo(self, chromosome: Chromosome) -> None:
    pass
```

Fungsi	`def create_pairs(self, chromosomes: list[Chromosome]) -> list[tuple[Chromosome, Chromosome]]:`
Deskripsi	Membentuk pasangan kromosom untuk proses crossover. Melakukan validasi agar jumlah populasi genap serta memastikan ukuran pasangan yang dihasilkan tetap mempertahankan ukuran populasi.

Output	Daftar pasangan kromosom (list[tuple[Chromosome, Chromosome]]).

Source Code

```python
def create_pairs(self, chromosomes:
list[Chromosome]) -> list[tuple[Chromosome,
Chromosome]]:
    if len(chromosomes) % 2:
      raise ValueError("Pairing requires an even
population")
    pairs = self._pairs_algo(chromosomes)
    if len(pairs) * 2 != len(chromosomes):
      raise ValueError("Pairing must preserve
population size")
    return pairs
```

Fungsi

```python
def _pairs_algo(self, chromosomes: list[Chromosome])
-> list[tuple[Chromosome, Chromosome]]:
```

Deskripsi	Metode dasar internal untuk mengatur algoritma pemilihan pasangan kromosom.
Output	Daftar pasangan kromosom (list[tuple[Chromosome, Chromosome]]).

Source Code

```python
def _pairs_algo(self, chromosomes: list[Chromosome])
-> list[tuple[Chromosome, Chromosome]]:
    pass
```

Fungsi

```python
def __init__(self, population: Population,
termination_strategy: TerminationStrategy,
cross_over_strategy: CrossOverStrategy,
selection_strategy: SelectionStrategy,
pair_picking_strategy: PairPickingStrategy,
mutation_strategy: MutationStrategy):
```

Deskripsi	Menginisialisasi kerangka utama algoritma genetika dengan menerima parameter populasi, strategi terminasi, crossover, seleksi, pairing, dan mutasi, serta mencatat riwayat nilai fitness terbaik awal.
Output	-

Source Code

```
def __init__(self, population: Population,
termination_strategy: TerminationStrategy,
cross_over_strategy: CrossOverStrategy,
selection_strategy: SelectionStrategy,
pair_picking_strategy: PairPickingStrategy,
mutation_strategy: MutationStrategy):
    self.population = population
    self.termination_strategy = termination_strategy
    self.cross_over_strategy = cross_over_strategy
    self.selection_strategy = selection_strategy
    self.pair_picking_strategy =
pair_picking_strategy
    self.mutation_strategy = mutation_strategy
    self.history = [self.best().fitness()]
```

Fungsi	`def best(self) -> Chromosome:`
Deskripsi	Mencari dan mengembalikan kromosom dengan nilai fitness tertinggi di dalam populasi saat ini.
Output	Objek kromosom terbaik

Source Code

```
def best(self) -> Chromosome:
    return max(self.population.chromosomes,
key=lambda item: item.fitness())
```

Fungsi	`def run(self) -> Chromosome:`

Deskripsi	Menjalankan loop utama algoritma genetika secara berulang selama kondisi terminasi belum terpenuhi (memanggil fungsi step), lalu mengembalikan salinan kromosom terbaik di akhir iterasi.
Output	Objek kromosom optimal terbaik hasil akhir pencarian (Chromosome).
Source Code	

```python
def run(self) -> Chromosome:
    while not self.termination_check():
      self.step()
    return self.best().clone()
```

Fungsi	`def termination_check(self) -> bool:`
Deskripsi	Memeriksa status keberlanjutan algoritma melalui strategi terminasi yang digunakan.
Output	Nilai boolean (bool) yang menyatakan apakah proses harus berhenti.
Source Code	

```python
def termination_check(self) -> bool:
    return self.termination_strategy.can_terminate()
```

Fungsi	`def step(self) -> None:`
Deskripsi	Menjalankan satu siklus penuh generasi algoritma genetika (mencakup seleksi, pairing, crossover, mutasi, dan pembentukan populasi generasi berikutnya).
Output	-
Source Code	`def step(self) -> None:`

```
    elite = self.best().clone()
    self.select()
    self.cross_over()
    self.mutate()
    chromosomes = self.population.chromosomes
    worst_index = min(range(len(chromosomes)), key=lambda index: chromosomes[index].fitness())
    chromosomes[worst_index] = elite
    self.history.append(self.best().fitness())
```

Fungsi

```
def select(self) -> None:
```

Deskripsi

Melakukan proses seleksi populasi menggunakan strategi seleksi yang telah ditentukan untuk memilih kromosom-kromosom generasi berikutnya. Memvalidasi agar jumlah kromosom yang terpilih tetap mempertahankan ukuran populasi awal (melempar ValueError jika tidak sesuai), lalu memperbarui populasi dengan salinan mendalam (deep copy) dari kromosom terpilih.

Output

-

Source Code

```
def select(self) -> None:
    selected = list(self.selection_strategy.pick(self.population.chromosomes))
    if len(selected) != self.population.size:
      raise ValueError("Selection must preserve population size")
    self.population.chromosomes = [item.clone() for item in selected]
```

Fungsi

```
def mutate(self) -> None:
```

Deskripsi	Menjalankan proses mutasi pada seluruh kromosom yang ada di dalam populasi saat ini dengan menerapkan strategi mutasi yang telah dikonfigurasi.
Output	-

Source Code

```
def mutate(self) -> None:
    for chromosome in self.population.chromosomes:
      self.mutation_strategy.mutate(chromosome)
```

Fungsi	`def create_pairs(self) -> list[tuple[Chromosome, Chromosome]]:`
Deskripsi	Membentuk pasangan-pasangan kromosom dari populasi saat ini dengan memanfaatkan strategi pemilihan pasangan (pair picking strategy) untuk tahapan pindah silang (crossover berikutnya).
Output	Daftar pasangan kromosom dalam bentuk list of tuples (list[tuple[Chromosome, Chromosome]]).

Source Code

```
def create_pairs(self) -> list[tuple[Chromosome, Chromosome]]:
    return self.pair_picking_strategy.create_pairs(self.population.chromosomes)
```

Fungsi	`def cross_over(self) -> None:`
Deskripsi	**Memasangkan kromosom, melakuka**
Output	

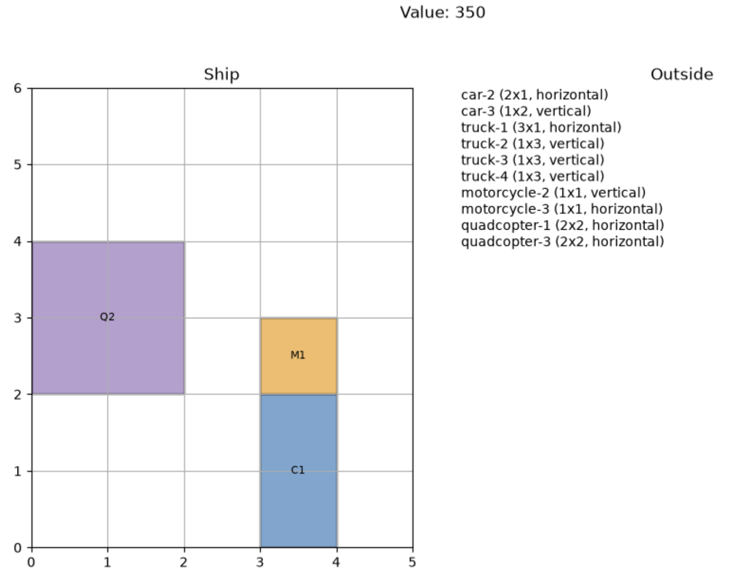
Source Code

```
def cross_over(self) -> None:
    children = []
    for pair in self.create_pairs():

children.extend(self.cross_over_strategy.cross(pair)
)
    self.population.chromosomes = children
```

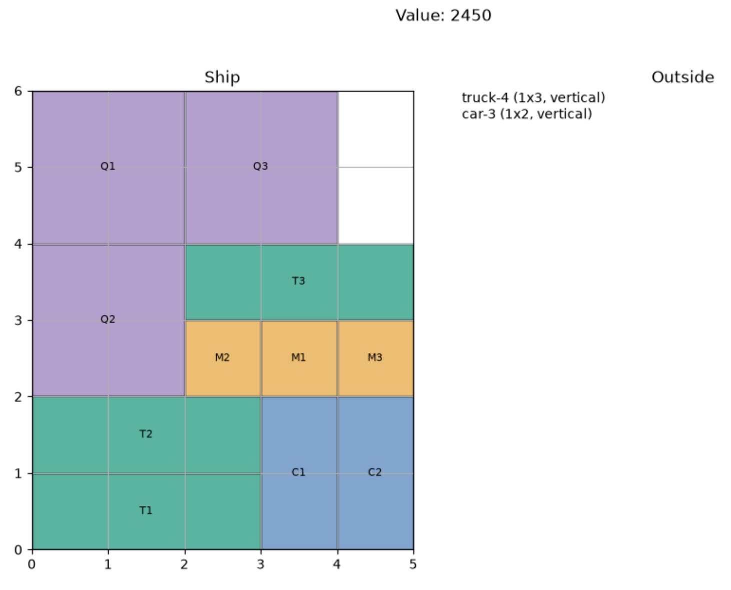
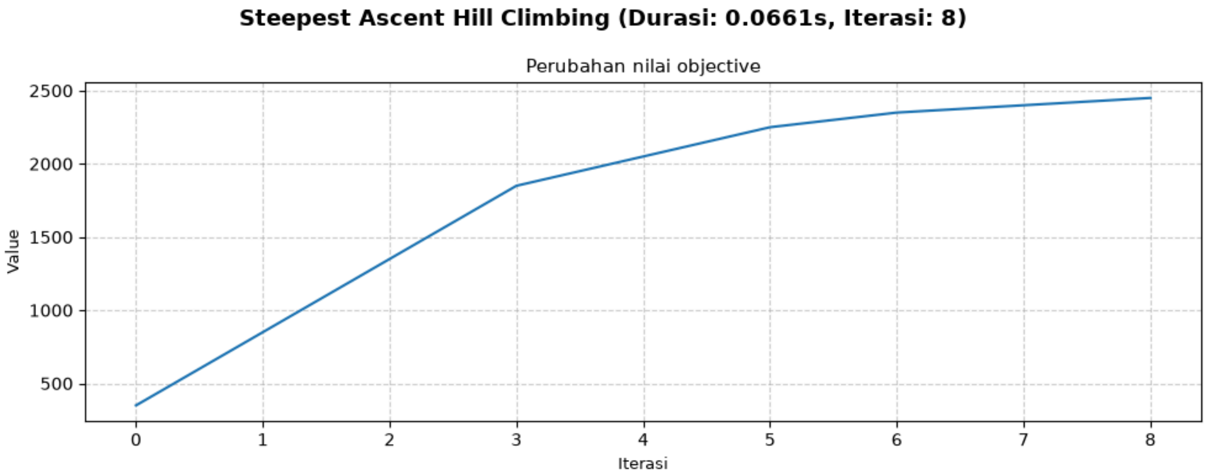
2.3. Hasil Eksperimen

2.3.1. Algoritma *Steepest Ascent Hill-Climbing*

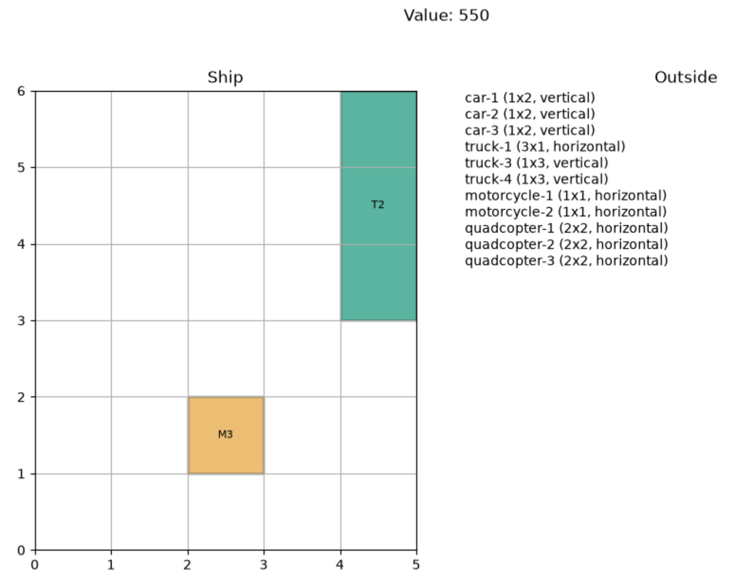
Berikut merupakan hasil eksperimen yang dilakukan sebanyak tiga kali menggunakan algoritma *steepest ascent hill-climbing.*

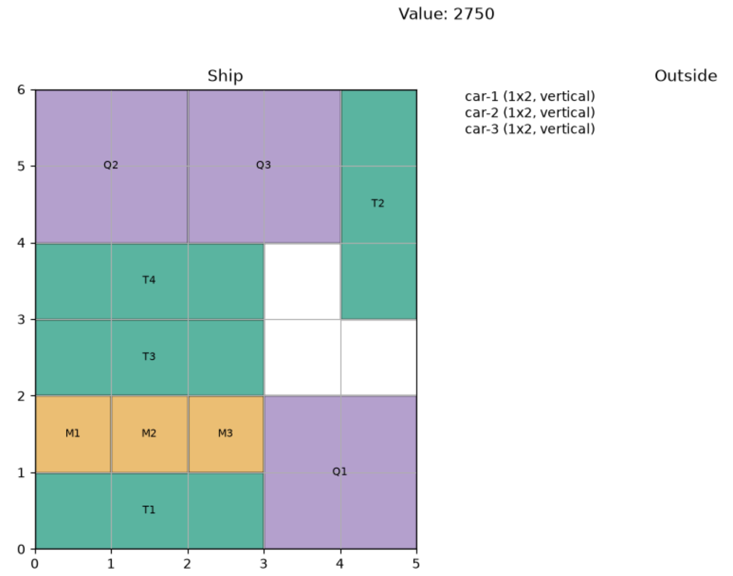
Eksperimen 1	
Visualisasi State Awal	

Value: 350
Ship
Outside
Q2
M1
C1
0
1
2
3
4
5
6
car-2 (2x1, horizontal)
car-3 (1x2, vertical)
truck-1 (3x1, horizontal)
truck-2 (1x3, vertical)
truck-3 (1x3, vertical)
truck-4 (1x3, vertical)
motorcycle-2 (1x1, vertical)
motorcycle-3 (1x1, horizontal)
quadcopter-1 (2x2, horizontal)
quadcopter-3 (2x2, horizontal)

Visualisasi State Akhir	 Value: 2450 Ship Outside truck-4 (1x3, vertical) car-3 (1x2, vertical) Q1 Q3 Q2 T3 M2 M1 M3 T2 C1 C2 T1 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2450**
Durasi Proses Pencarian	**0.0661s**
Banyak Iterasi	**8**
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

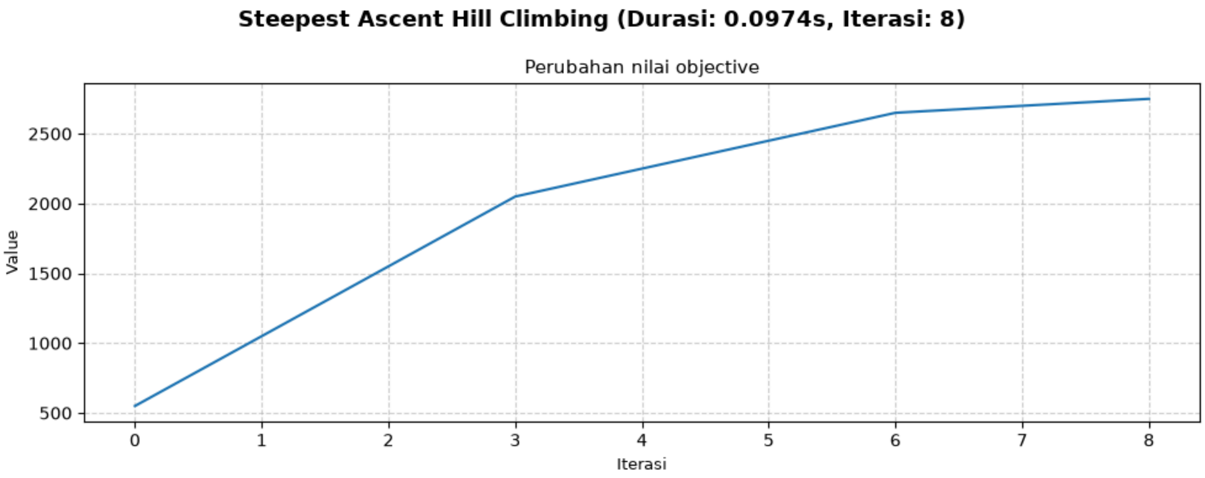
Eksperimen 2

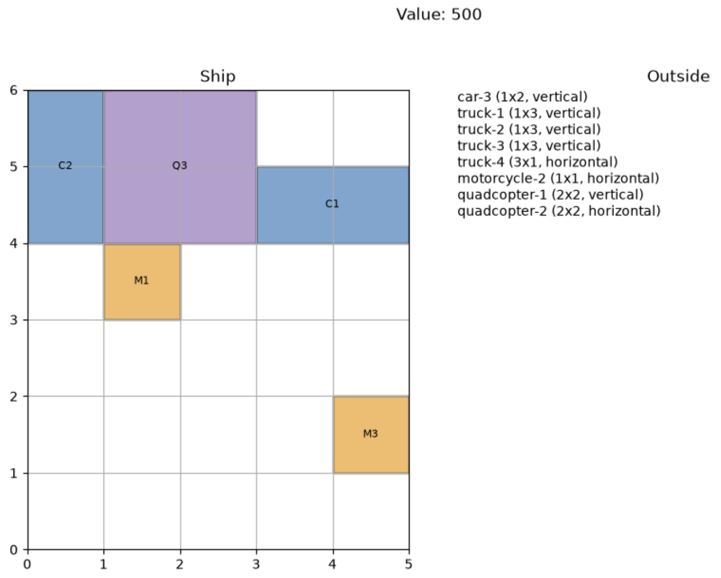
<table>
<tr><td>Visualisasi State Awal</td><td>
Value: 550
Ship
Outside
car-1 (1x2, vertical)
car-2 (1x2, vertical)
car-3 (1x2, vertical)
truck-1 (3x1, horizontal)
truck-3 (1x3, vertical)
truck-4 (1x3, vertical)
motorcycle-1 (1x1, horizontal)
motorcycle-2 (1x1, horizontal)
quadcopter-1 (2x2, horizontal)
quadcopter-2 (2x2, horizontal)
quadcopter-3 (2x2, horizontal)
T2
M3
0
1
2
3
4
5
6

</td></tr>
<tr><td>Visualisasi State Akhir</td><td>
Value: 2750
Ship
Outside
car-1 (1x2, vertical)
car-2 (1x2, vertical)
car-3 (1x2, vertical)
Q2
Q3
T2
T4
T3
M1
M2
M3
Q1
T1
0
1
2
3
4
5
6

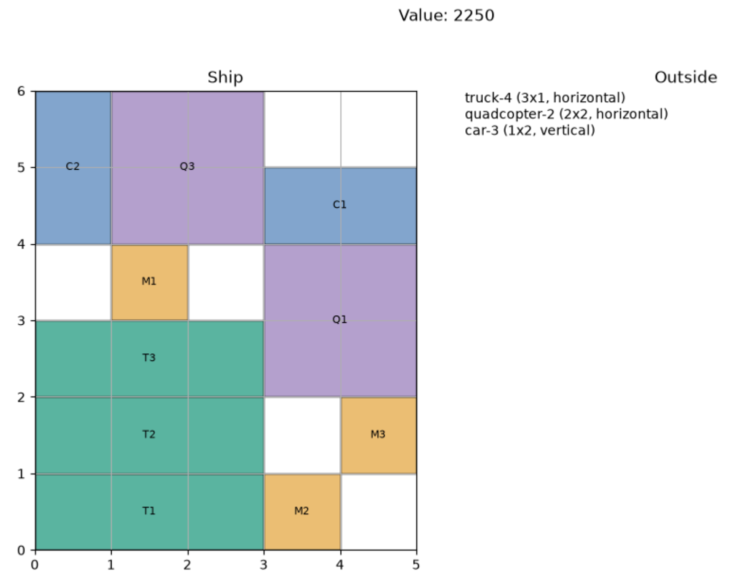
</td></tr>
</table>

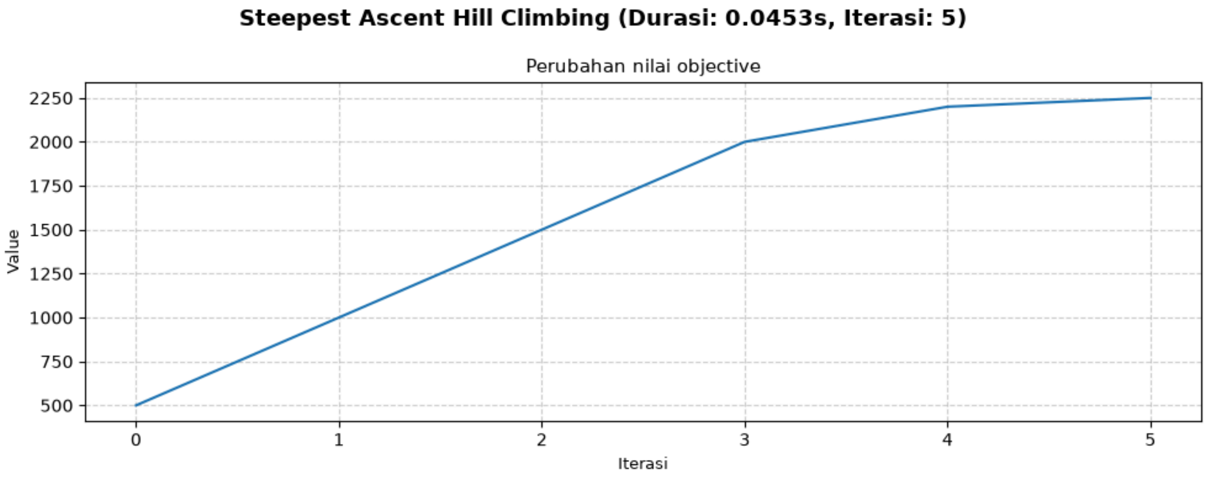
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	**0.0974s**
Banyak Iterasi	**8**
Plot nilai objective function terhadap banyak iterasi yang dilewati	



Eksperimen 3	
Visualisasi State Awal	

Value: 500
Ship
C2
Q3
C1
M1
M3
0
1
2
3
4
5
6
Outside
car-3 (1x2, vertical)
truck-1 (1x3, vertical)
truck-2 (1x3, vertical)
truck-3 (1x3, vertical)
truck-4 (3x1, horizontal)
motorcycle-2 (1x1, horizontal)
quadcopter-1 (2x2, vertical)
quadcopter-2 (2x2, horizontal)

<table>
<tr><td>Visualisasi State Akhir</td><td colspan="2">
Value: 2250
Ship
Outside
truck-4 (3x1, horizontal)
quadcopter-2 (2x2, horizontal)
car-3 (1x2, vertical)
C2
Q3
C1
M1
Q1
T3
T2
M3
T1
M2
0
1
2
3
4
5
6

</td></tr>
<tr><td colspan="2">Nilai <i>objective function</i> yang dicapai</td><td>2250</td></tr>
<tr><td colspan="2">Durasi Proses Pencarian</td><td>0.0453s</td></tr>
<tr><td colspan="2">Banyak Iterasi</td><td>5</td></tr>
<tr><td colspan="3">Plot nilai objective function terhadap banyak iterasi yang dilewati</td></tr>
<tr><td colspan="3">
</td></tr>
</table>

Berdasarkan hasil tersebut, didapatkan informasi sebagai berikut:

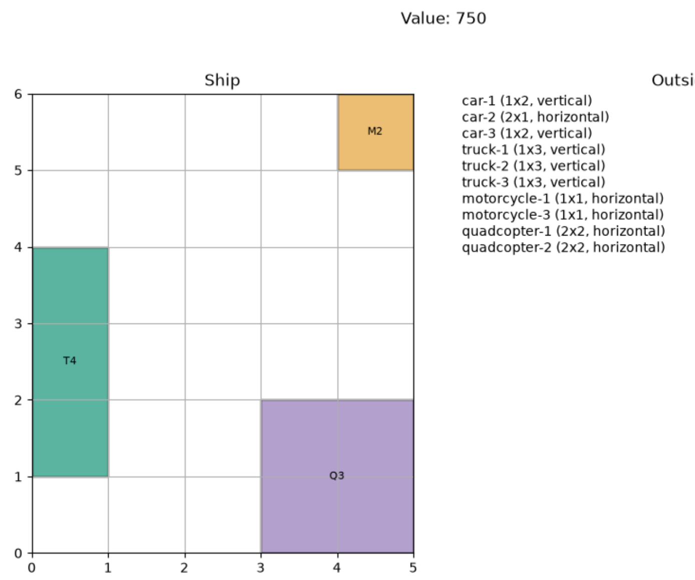
1. Eksperimen 1 menghasilkan final state dengan nilai objective function sebesar 2450, durasi selama 0.0661 detik, dan iterasi sebanyak 8.

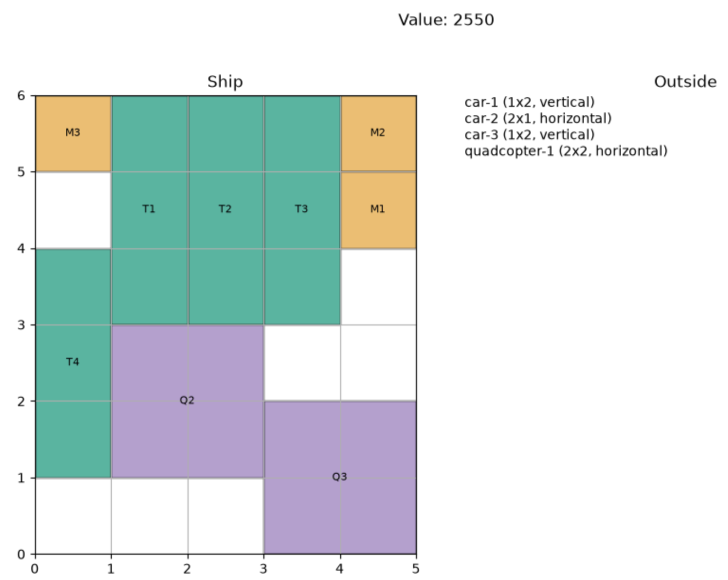
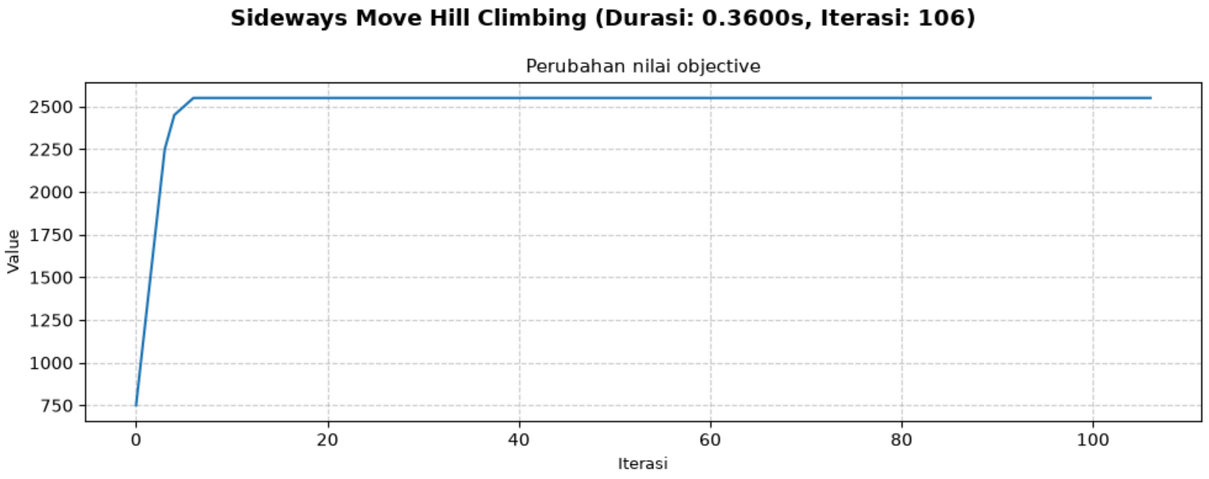
2. Eksperimen 2 menghasilkan final state dengan nilai objective function sebesar 2750, durasi selama 0.0974 detik, dan iterasi sebanyak 8.
3. Eksperimen 3 menghasilkan final state dengan nilai objective function sebesar 2250, durasi selama 0.0453 detik, dan iterasi sebanyak 5.

Berdasarkan 3 eksperimen yang telah dijalankan, algoritma Steepest Ascent Hill-Climbing menunjukkan variasi kinerja yang dipengaruhi oleh perbedaan state awal pada tiap percobaan. Eksperimen 2 mencatatkan hasil optimal tertinggi dengan nilai objective function sebesar 2750 dalam 8 iterasi dan durasi 0.0974 detik. Sementara itu, Eksperimen 3 menghasilkan nilai terendah sebesar 2250 dengan jumlah iterasi paling sedikit yaitu 5, yang mengindikasikan algoritma berhenti lebih awal akibat terjebak pada local optimum. Perbedaan durasi dan jumlah iterasi ini terjadi karena proses evaluasi kandidat tetangga pada setiap langkah pencarian memiliki lintasan yang berbeda, di mana algoritma secara konsisten akan berhenti ketika tidak lagi ditemukan tetangga yang memberikan peningkatan nilai objective function.

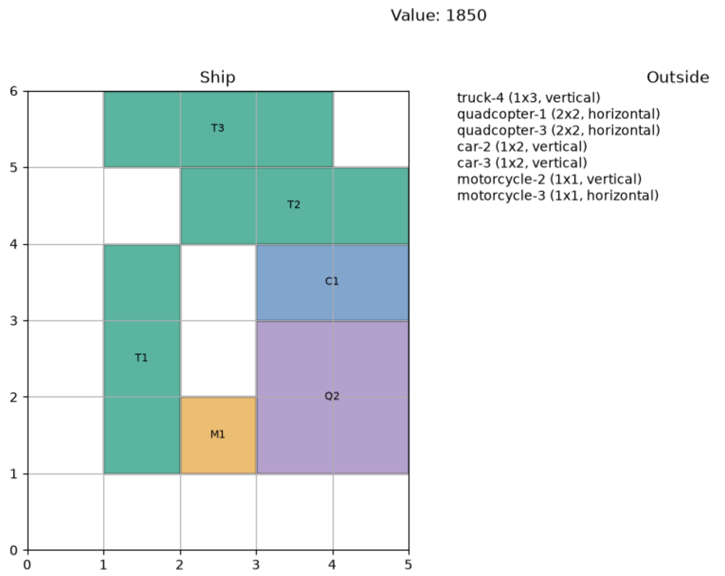
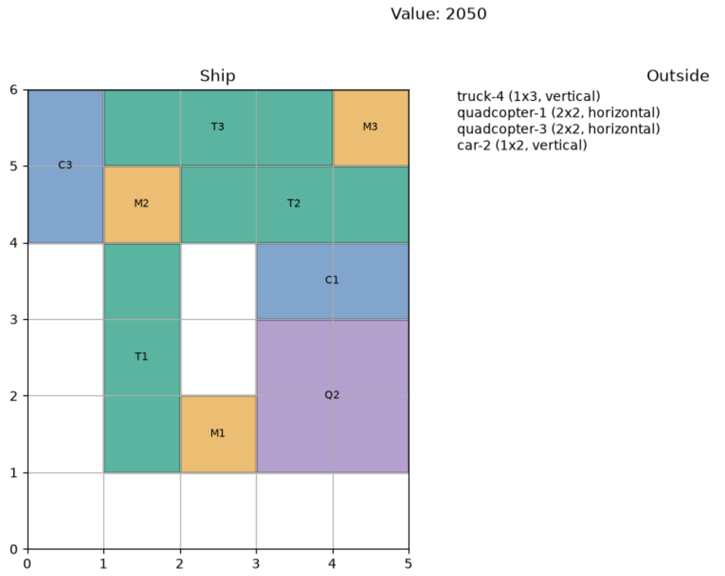
2.3.2. Algoritma *Hill-Climbing with Sideways Move*

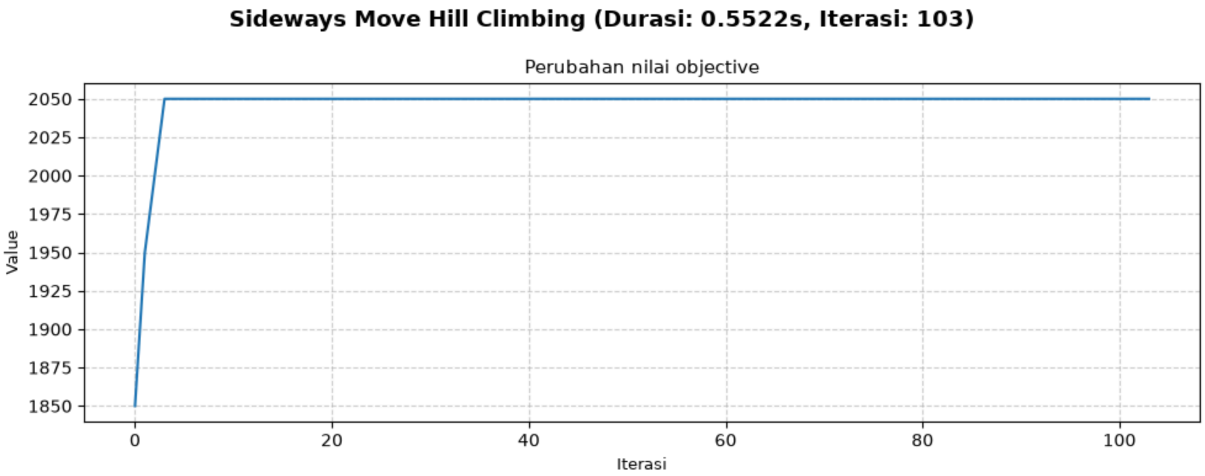
Berikut merupakan hasil eksperimen yang dilakukan sebanyak tiga kali menggunakan algoritma *steepest ascent hill-climbing.*

Eksperimen 1	
Visualisasi State Awal	 Value: 750 Ship Outside car-1 (1x2, vertical) car-2 (2x1, horizontal) car-3 (1x2, vertical) truck-1 (1x3, vertical) truck-2 (1x3, vertical) truck-3 (1x3, vertical) motorcycle-1 (1x1, horizontal) motorcycle-3 (1x1, horizontal) quadcopter-1 (2x2, horizontal) quadcopter-2 (2x2, horizontal) M2 T4 Q3 0 1 2 3 4 5 6

Visualisasi State Akhir	 Value: 2550 Ship Outside car-1 (1x2, vertical) car-2 (2x1, horizontal) car-3 (1x2, vertical) quadcopter-1 (2x2, horizontal) M3 M2 T1 T2 T3 M1 T4 Q2 Q3 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2550**
Durasi Proses Pencarian	**0.3600s**
Banyak Iterasi	**106**
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

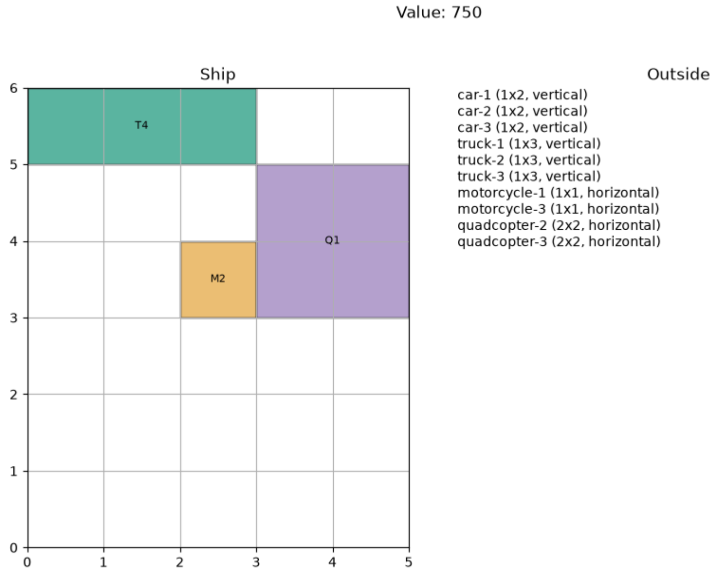
Eksperimen 2

<table>
<tr>
<td>Visualisasi State Awal</td>
<td>

Value: 1850
Ship
Outside
truck-4 (1x3, vertical)
quadcopter-1 (2x2, horizontal)
quadcopter-3 (2x2, horizontal)
car-2 (1x2, vertical)
car-3 (1x2, vertical)
motorcycle-2 (1x1, vertical)
motorcycle-3 (1x1, horizontal)
T3
T2
C1
T1
Q2
M1
0
1
2
3
4
5
6
</td>
</tr>
<tr>
<td>Visualisasi State Akhir</td>
<td>

Value: 2050
Ship
Outside
truck-4 (1x3, vertical)
quadcopter-1 (2x2, horizontal)
quadcopter-3 (2x2, horizontal)
car-2 (1x2, vertical)
C3
T3
M3
M2
T2
C1
T1
Q2
M1
0
1
2
3
4
5
6
</td>
</tr>
<tr>
<td>Nilai objective function yang dicapai</td>
<td>2050</td>
</tr>
<tr>
<td>Durasi Proses Pencarian</td>
<td>0.5522s</td>
</tr>
<tr>
<td>Banyak Iterasi</td>
<td>103</td>
</tr>
<tr>
<td colspan="2">Plot nilai objective function terhadap banyak iterasi yang dilewati</td>
</tr>
</table>

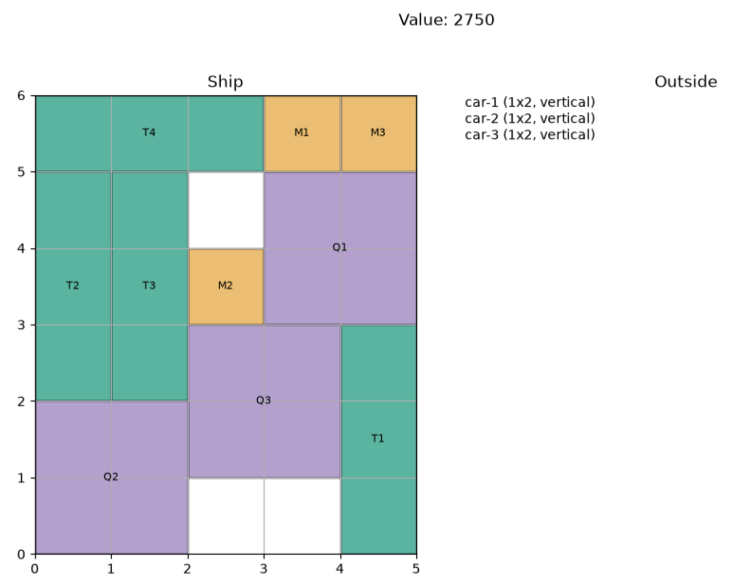
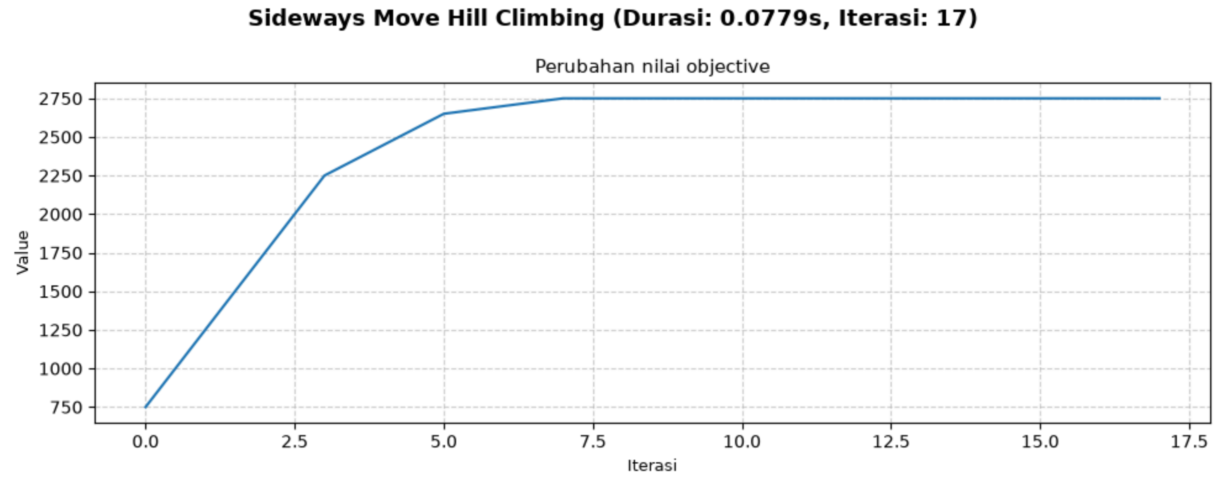


Eksperimen 3

Visualisasi State Awal


Value: 750
Ship
Outside
T4
M2
Q1
0
1
2
3
4
5
6

car-1 (1x2, vertical)
car-2 (1x2, vertical)
car-3 (1x2, vertical)
truck-1 (1x3, vertical)
truck-2 (1x3, vertical)
truck-3 (1x3, vertical)
motorcycle-1 (1x1, horizontal)
motorcycle-3 (1x1, horizontal)
quadcopter-2 (2x2, horizontal)
quadcopter-3 (2x2, horizontal)

Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (1x2, vertical) car-3 (1x2, vertical) T4 M1 M3 T2 T3 M2 Q1 Q3 T1 Q2 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	**0.0779s**
Banyak Iterasi	**17**
Plot nilai objective function terhadap banyak iterasi yang dilewati	
Sideways Move Hill Climbing (Durasi: 0.0779s, Iterasi: 17) 	

Berdasarkan hasil tersebut, didapatkan informasi sebagai berikut:

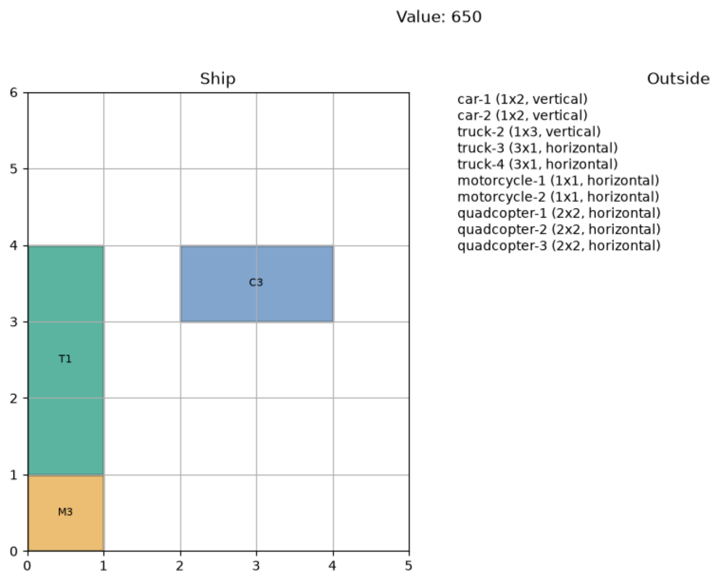
1. Eksperimen 1 menghasilkan *final state* dengan nilai *objective function* sebesar 2550, durasi selama 0.3600 detik, dan iterasi sebanyak 106.
2. Eksperimen 2 menghasilkan *final state* dengan nilai *objective function* sebesar 2050, durasi selama 0.5522 detik, dan iterasi sebanyak 103.

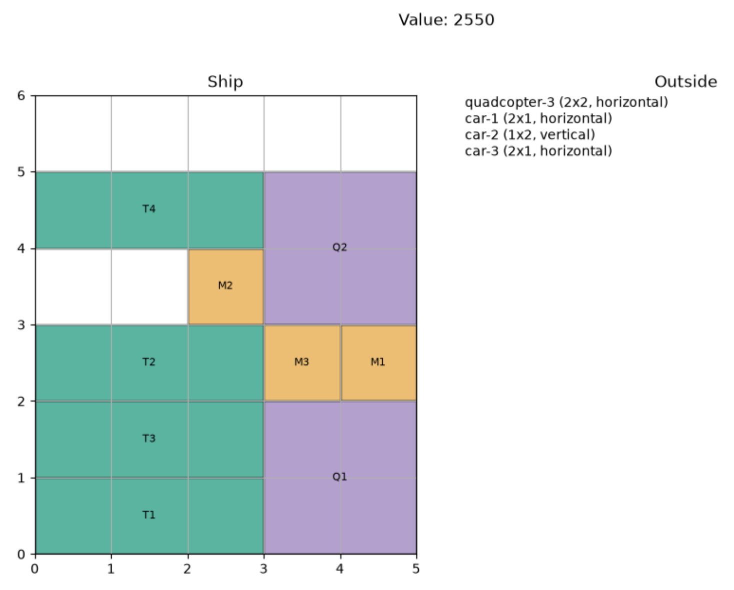
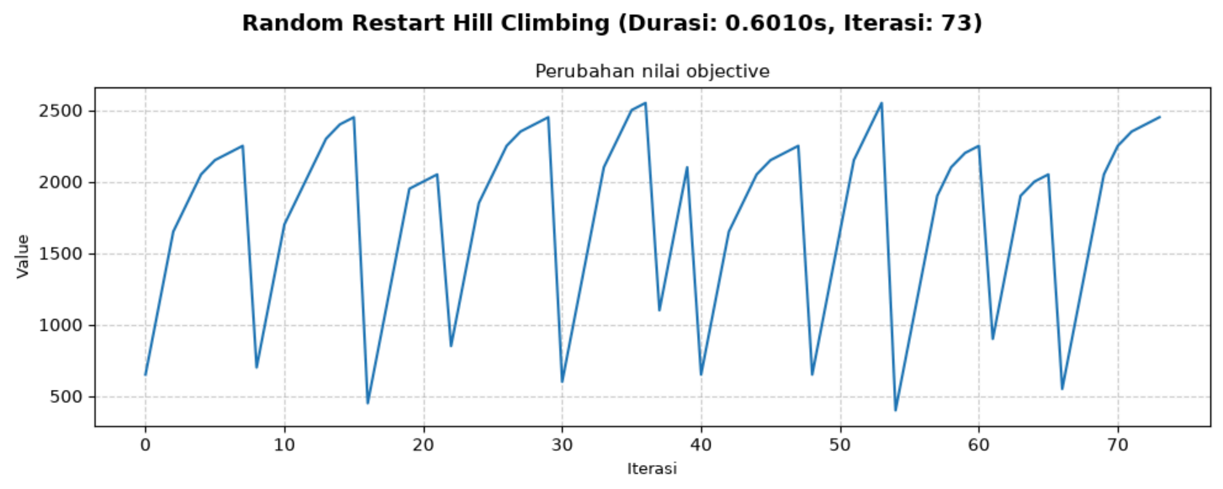
3. Eksperimen 3 menghasilkan *final state* dengan nilai *objective function* sebesar 2750, durasi selama 0.0779 detik, dan iterasi sebanyak 17.

Berdasarkan 3 eksperimen yang telah dijalankan, algoritma *Hill-Climbing with Sideways Move* menunjukkan karakteristik pencarian yang mampu melewati plateau (dataran) berkat toleransi gerakan menyamping saat nilai *objective function* tidak mengalami peningkatan. Eksperimen 3 mencatatkan hasil optimal terbaik dengan nilai *objective function* mencapai 2750 dalam jumlah iterasi paling sedikit yaitu 17 iterasi dan durasi 0.0779 detik. Sebaliknya, Eksperimen 1 dan 2 memerlukan jumlah iterasi yang jauh lebih banyak (di atas 100 iterasi) karena algoritma harus menelusuri area *plateau* yang lebih luas sebelum akhirnya konvergen atau mencapai batas maksimum gerakan menyamping yang diizinkan. Hal ini menjelaskan mengapa durasi eksekusi pada eksperimen dengan iterasi tinggi menjadi lebih lama secara signifikan.

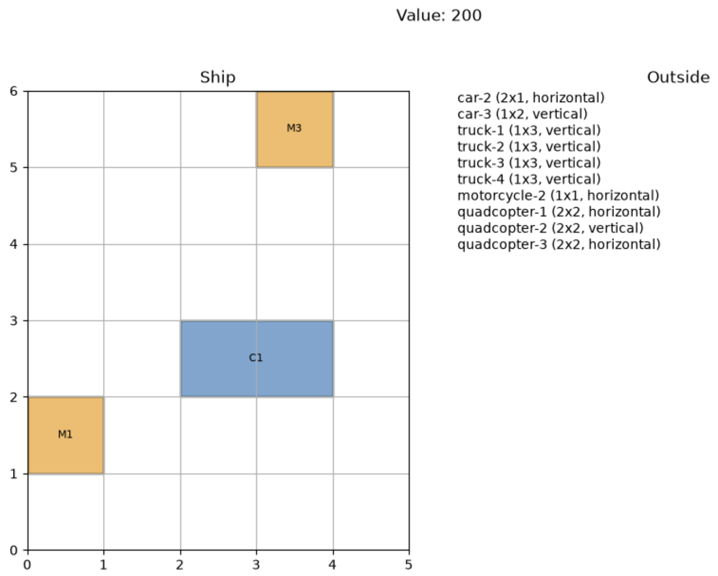
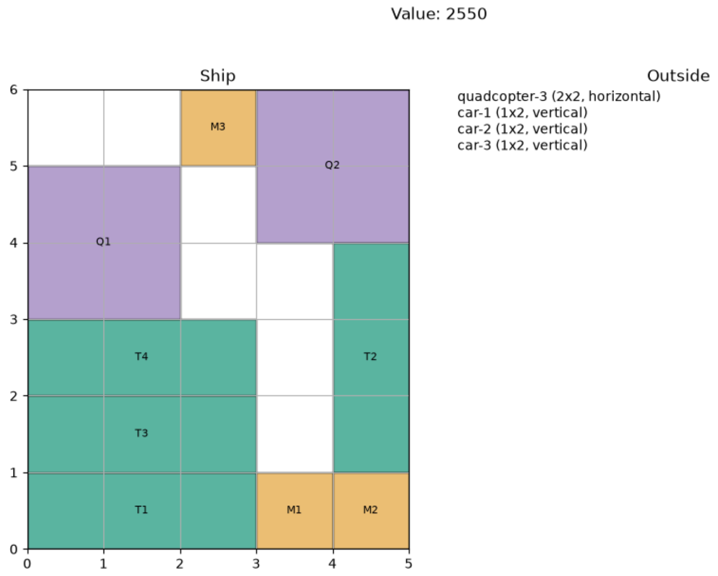
2.3.3. Algoritma *Random Restart Hill-Climbing*

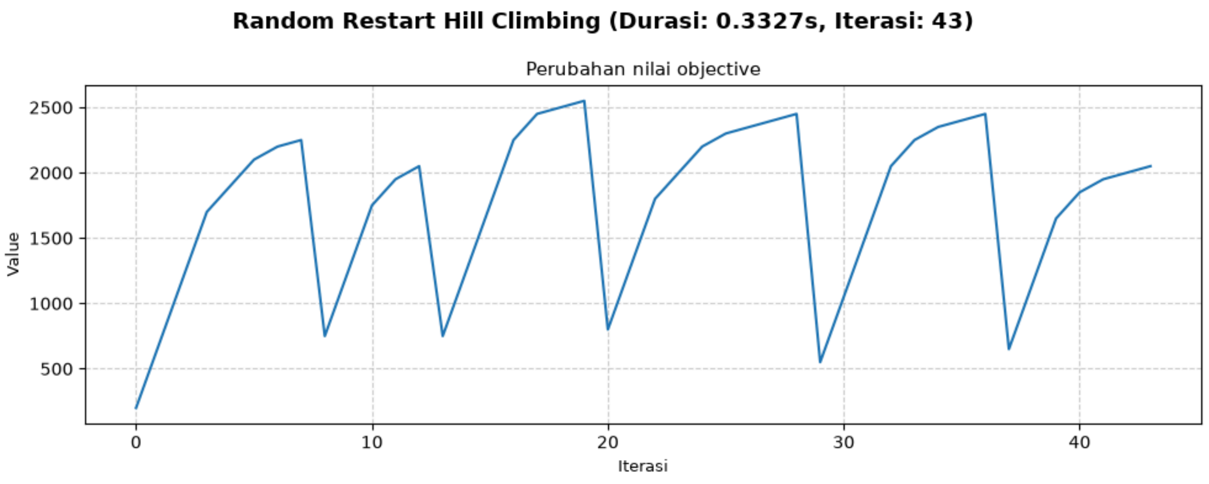
Berikut merupakan hasil eksperimen yang dilakukan sebanyak tiga kali menggunakan algoritma *steepest ascent hill-climbing.*

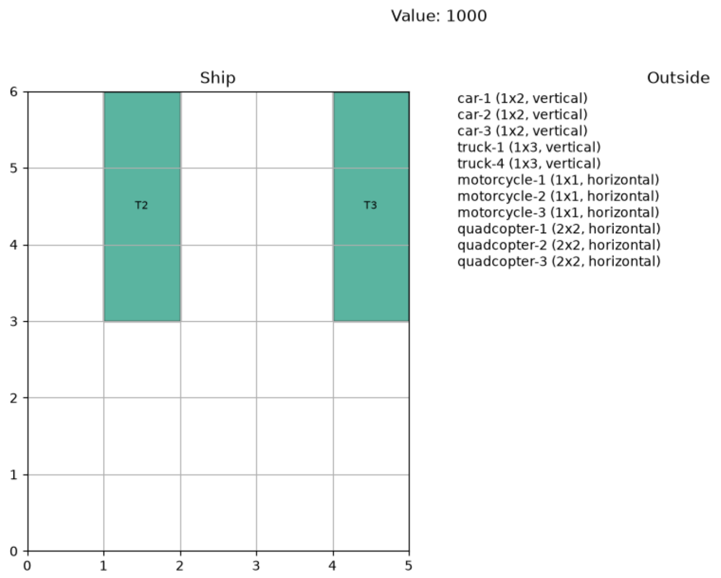
Eksperimen 1	
Visualisasi State Awal	 Value: 650 Ship Outside car-1 (1x2, vertical) car-2 (1x2, vertical) truck-2 (1x3, vertical) truck-3 (3x1, horizontal) truck-4 (3x1, horizontal) motorcycle-1 (1x1, horizontal) motorcycle-2 (1x1, horizontal) quadcopter-1 (2x2, horizontal) quadcopter-2 (2x2, horizontal) quadcopter-3 (2x2, horizontal) T1 C3 M3 0 1 2 3 4 5 6

Visualisasi State Akhir	 Value: 2550 Ship Outside quadcopter-3 (2x2, horizontal) car-1 (2x1, horizontal) car-2 (1x2, vertical) car-3 (2x1, horizontal) T4 M2 Q2 T2 M3 M1 T3 Q1 T1 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2550**
Durasi Proses Pencarian	**0.6010s**
Banyak Iterasi	**73**
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

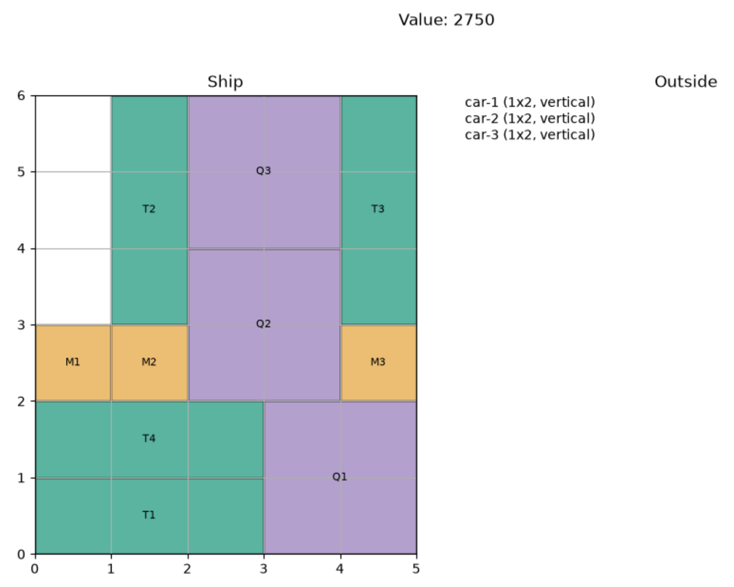
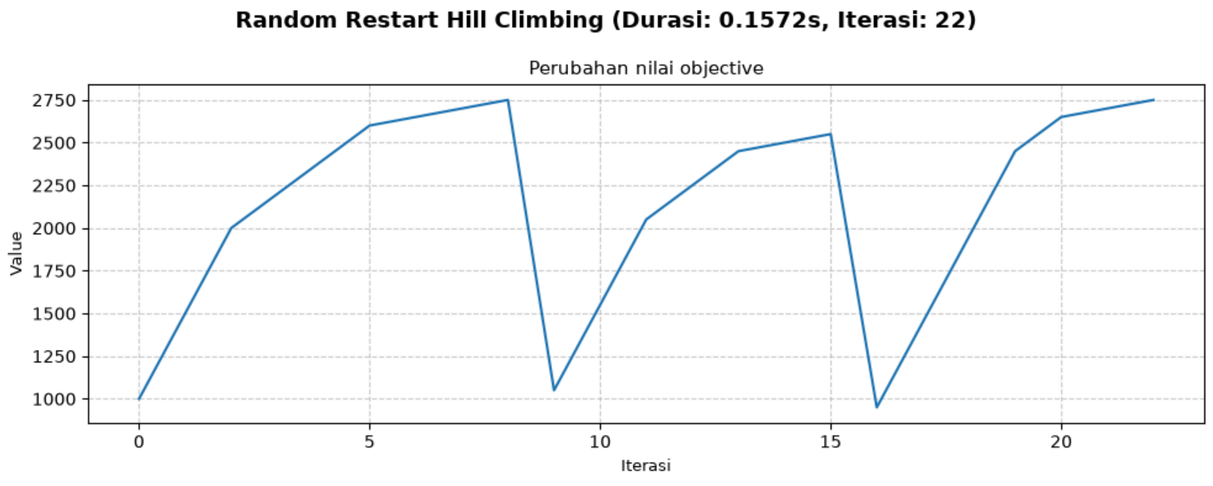
Eksperimen 2

<table>
<tr>
<td>Visualisasi State Awal</td>
<td colspan="2">

Value: 200
Ship
Outside
M3
C1
M1
0
1
2
3
4
5
6
0
1
2
3
4
5
car-2 (2x1, horizontal)
car-3 (1x2, vertical)
truck-1 (1x3, vertical)
truck-2 (1x3, vertical)
truck-3 (1x3, vertical)
truck-4 (1x3, vertical)
motorcycle-2 (1x1, horizontal)
quadcopter-1 (2x2, horizontal)
quadcopter-2 (2x2, vertical)
quadcopter-3 (2x2, horizontal)
</td>
</tr>
<tr>
<td>Visualisasi State Akhir</td>
<td colspan="2">

Value: 2550
Ship
Outside
M3
Q2
Q1
T4
T2
T3
T1
M1
M2
0
1
2
3
4
5
6
0
1
2
3
4
5
quadcopter-3 (2x2, horizontal)
car-1 (1x2, vertical)
car-2 (1x2, vertical)
car-3 (1x2, vertical)
</td>
</tr>
<tr>
<td colspan="2">Nilai <i>objective function</i> yang dicapai</td>
<td>2550</td>
</tr>
<tr>
<td colspan="2">Durasi Proses Pencarian</td>
<td>0.3327s</td>
</tr>
<tr>
<td colspan="2">Banyak Iterasi</td>
<td>43</td>
</tr>
<tr>
<td colspan="3">Plot nilai objective function terhadap banyak iterasi yang dilewati</td>
</tr>
</table>



Eksperimen 3	
Visualisasi State Awal	

Value: 1000
Ship
Outside
T2
T3
0
1
2
3
4
5
6
car-1 (1x2, vertical)
car-2 (1x2, vertical)
car-3 (1x2, vertical)
truck-1 (1x3, vertical)
truck-4 (1x3, vertical)
motorcycle-1 (1x1, horizontal)
motorcycle-2 (1x1, horizontal)
motorcycle-3 (1x1, horizontal)
quadcopter-1 (2x2, horizontal)
quadcopter-2 (2x2, horizontal)
quadcopter-3 (2x2, horizontal)

Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (1x2, vertical) car-3 (1x2, vertical) Q3 T2 T3 Q2 M1 M2 M3 T4 Q1 T1 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	**0.1572**
Banyak Iterasi	**22**
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

Dari ketiga eksperimen tersebut, didapatkan

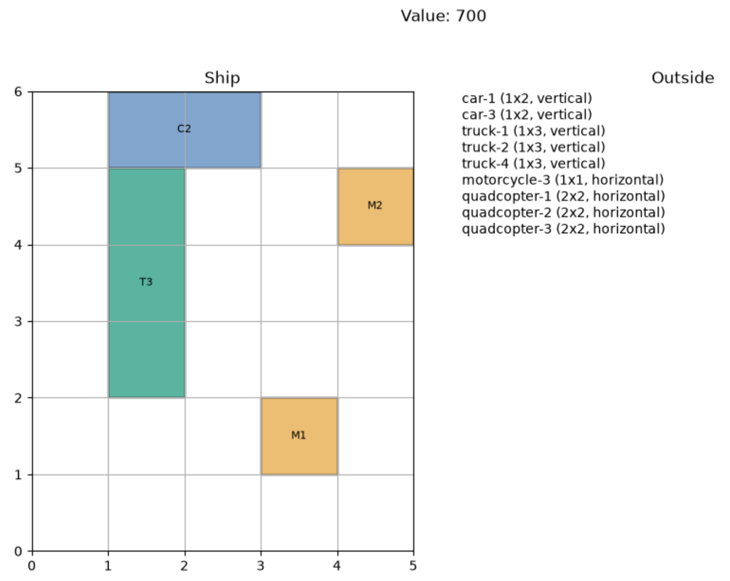
1. Run 1 menghasilkan final state dengan nilai objektif sebesar 2550, durasi 0,601 detik, dan iterasi sebanyak 73.

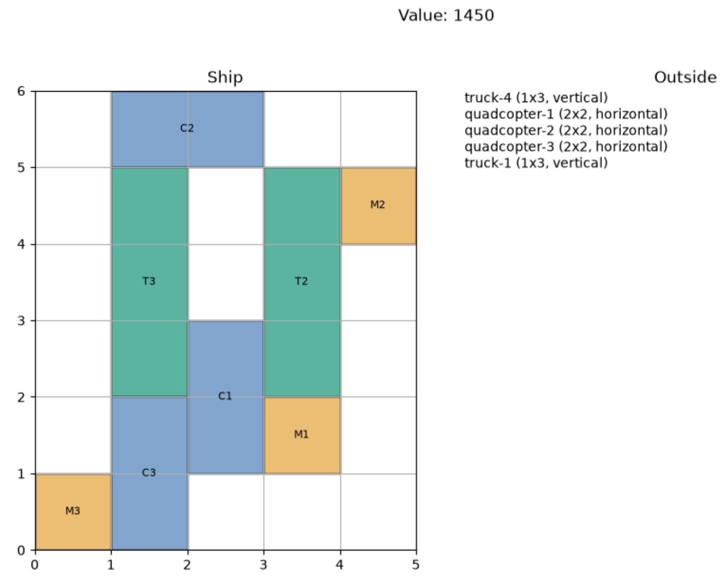
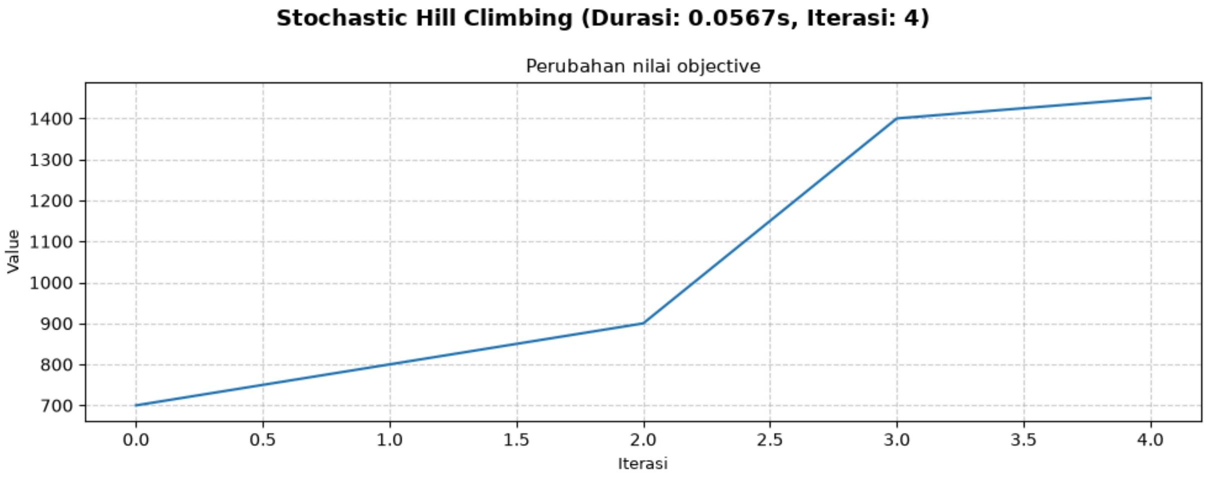
2. Run 2 menghasilkan final state dengan nilai objektif sebesar 2550, durasi 0,3327 detik, dan iterasi sebanyak 43.
3. Run 3 menghasilkan final state dengan nilai objektif sebesar 2750, durasi 0,1572 detik, dan iterasi sebanyak 22.

Berdasarkan tiga run tersebut, Random Restart Hill Climbing menghasilkan nilai objektif akhir rata-rata sebesar 2616,67. Run 1 dan Run 2 memperoleh nilai yang sama, sementara Run 3 menghasilkan nilai tertinggi dengan jumlah iterasi dan durasi paling rendah di antara ketiga run.

2.3.4. Algoritma *Stochastic Hill-Climbing*

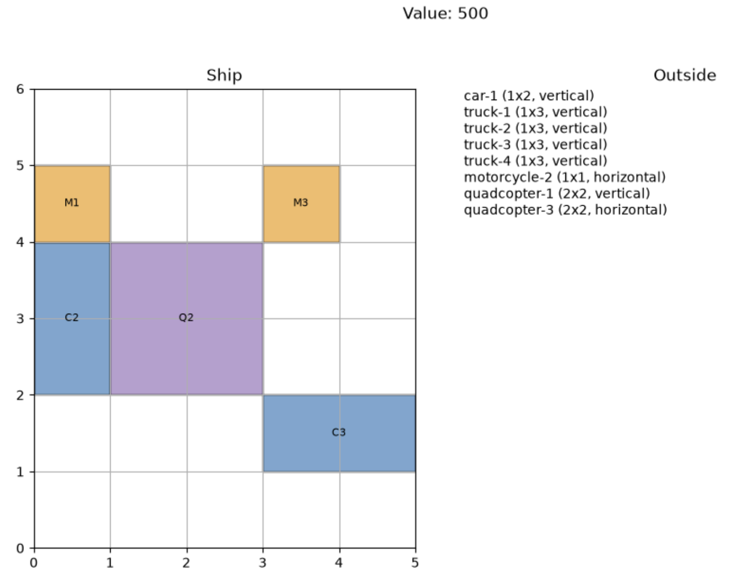
Berikut merupakan hasil eksperimen yang dilakukan sebanyak tiga kali menggunakan algoritma *steepest ascent hill-climbing.*

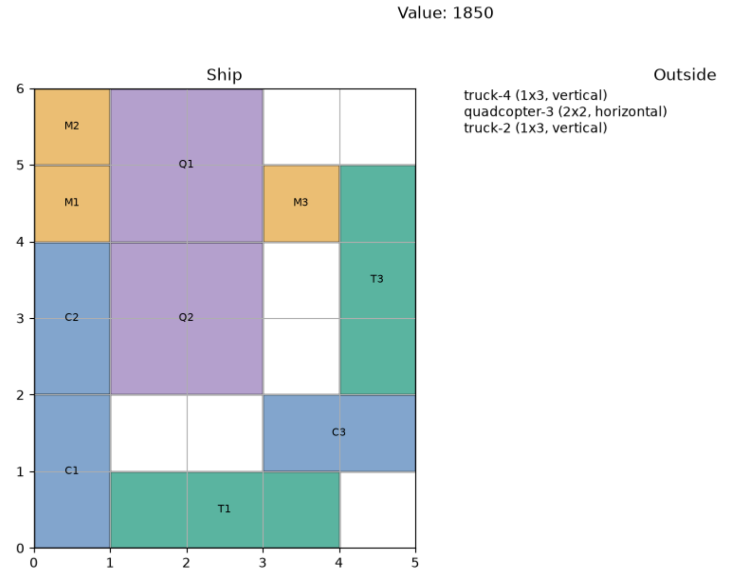
Eksperimen 1	
Visualisasi State Awal	 Value: 700 Ship Outside C2 T3 M2 M1 0 1 2 3 4 5 6 car-1 (1x2, vertical) car-3 (1x2, vertical) truck-1 (1x3, vertical) truck-2 (1x3, vertical) truck-4 (1x3, vertical) motorcycle-3 (1x1, horizontal) quadcopter-1 (2x2, horizontal) quadcopter-2 (2x2, horizontal) quadcopter-3 (2x2, horizontal)

<table>
<tr><td>Visualisasi State Akhir</td><td>

Value: 1450
Ship
Outside
truck-4 (1x3, vertical)
quadcopter-1 (2x2, horizontal)
quadcopter-2 (2x2, horizontal)
quadcopter-3 (2x2, horizontal)
truck-1 (1x3, vertical)
C2
T3
T2
M2
C1
M1
C3
M3
0
1
2
3
4
5
6
</td></tr>
<tr><td>Nilai <i>objective function</i> yang dicapai</td><td>1450</td></tr>
<tr><td>Durasi Proses Pencarian</td><td>0.0567s</td></tr>
<tr><td>Banyak Iterasi</td><td>4</td></tr>
<tr><td colspan="2">Plot nilai objective function terhadap banyak iterasi yang dilewati</td></tr>
<tr><td colspan="2">


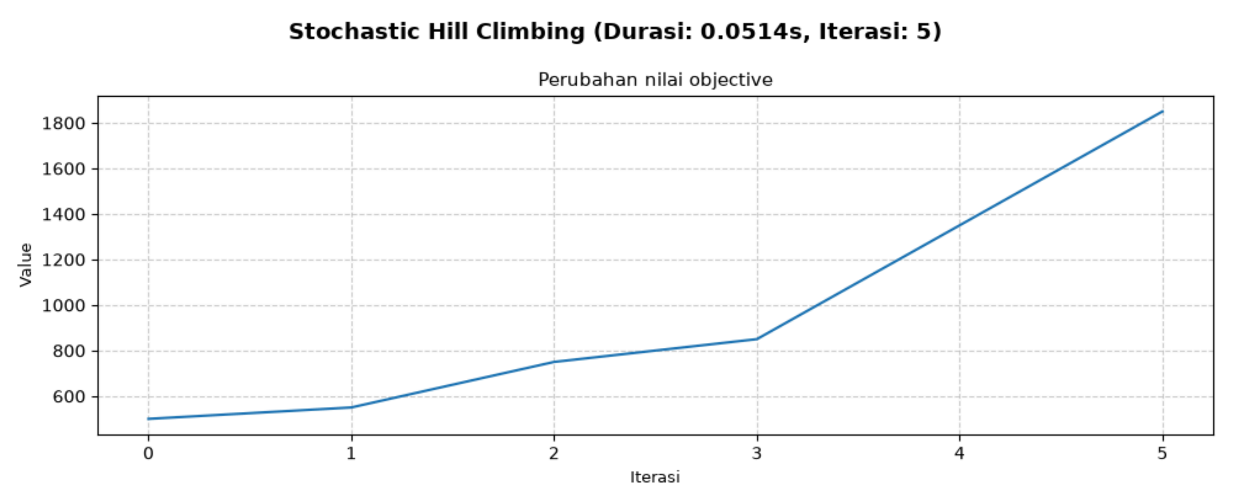
</td></tr>
</table>

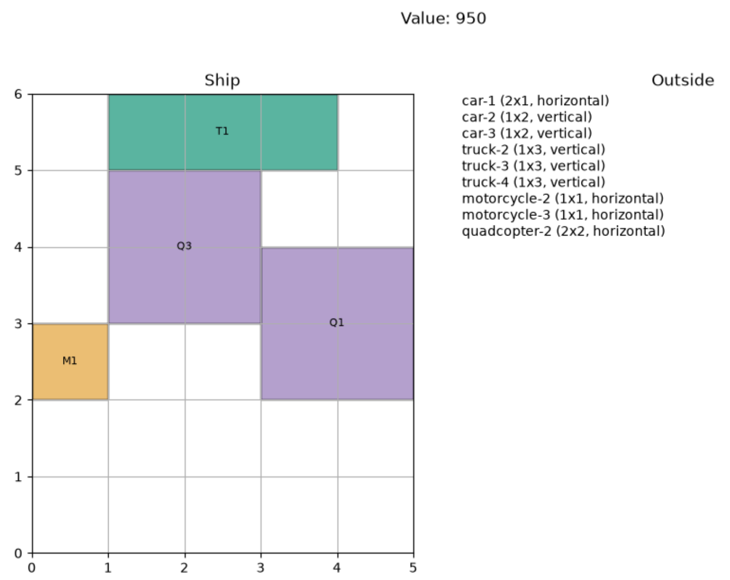
Eksperimen 2

<table>
<tr><td>Visualisasi State Awal</td><td>
Value: 500
Ship
Outside
car-1 (1x2, vertical)
truck-1 (1x3, vertical)
truck-2 (1x3, vertical)
truck-3 (1x3, vertical)
truck-4 (1x3, vertical)
motorcycle-2 (1x1, horizontal)
quadcopter-1 (2x2, vertical)
quadcopter-3 (2x2, horizontal)
M1
M3
C2
Q2
C3
0
1
2
3
4
5
6

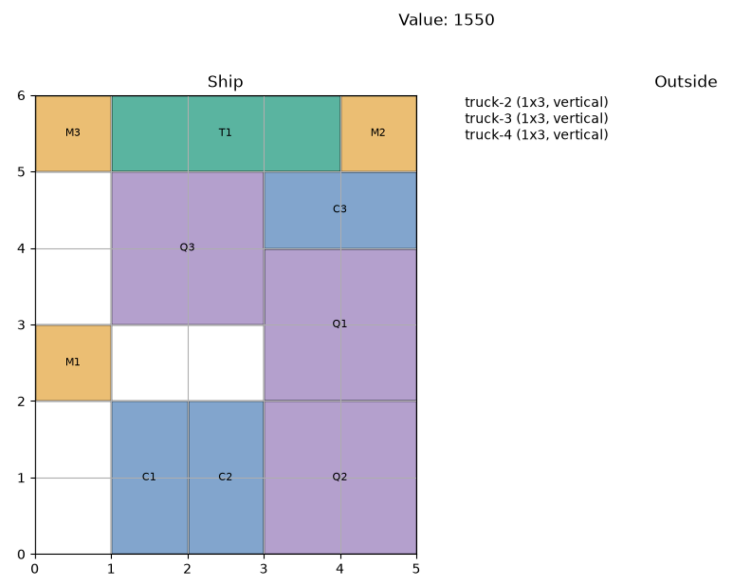
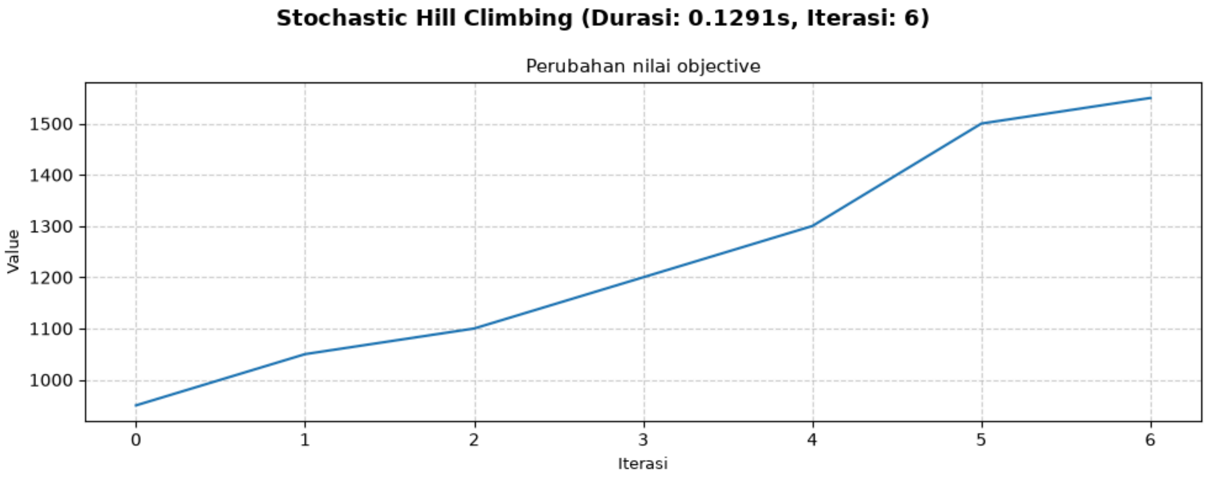
</td></tr>
<tr><td>Visualisasi State Akhir</td><td>
Value: 1850
Ship
Outside
truck-4 (1x3, vertical)
quadcopter-3 (2x2, horizontal)
truck-2 (1x3, vertical)
M2
Q1
M1
M3
T3
C2
Q2
C3
C1
T1
0
1
2
3
4
5
6

</td></tr>
<tr><td>Nilai <i>objective function</i> yang dicapai</td><td>1850</td></tr>
<tr><td>Durasi Proses Pencarian</td><td>0.0514s</td></tr>
<tr><td>Banyak Iterasi</td><td>5</td></tr>
<tr><td colspan="2">Plot nilai objective function terhadap banyak iterasi yang dilewati</td></tr>
</table>



Eksperimen 3	
Visualisasi State Awal	

Value: 950
Ship
Outside
car-1 (2x1, horizontal)
car-2 (1x2, vertical)
car-3 (1x2, vertical)
truck-2 (1x3, vertical)
truck-3 (1x3, vertical)
truck-4 (1x3, vertical)
motorcycle-2 (1x1, horizontal)
motorcycle-3 (1x1, horizontal)
quadcopter-2 (2x2, horizontal)
T1
Q3
Q1
M1
0
1
2
3
4
5
6

Visualisasi State Akhir	 Value: 1550 Ship Outside truck-2 (1x3, vertical) truck-3 (1x3, vertical) truck-4 (1x3, vertical) M3 T1 M2 Q3 C3 Q1 M1 C1 C2 Q2 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**1550**
Durasi Proses Pencarian	**0.1291s**
Banyak Iterasi	**6**
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

Dari ketiga eksperimen tersebut, didapatkan

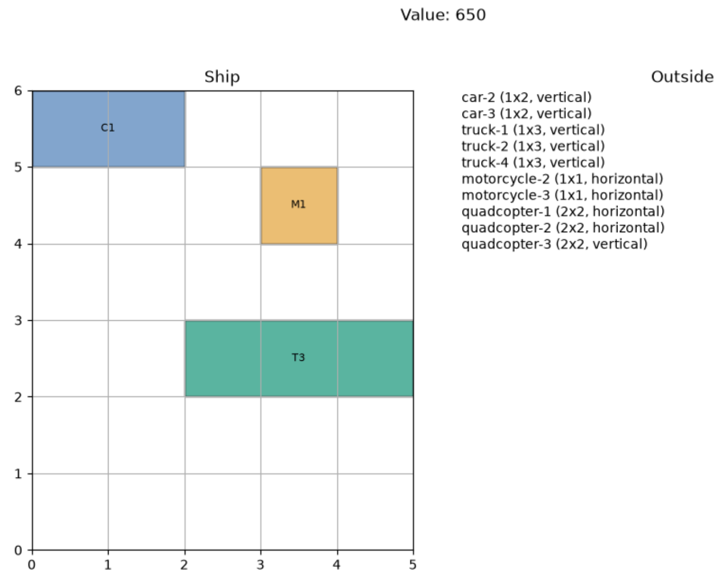
1. Run 1 menghasilkan final state dengan nilai objektif sebesar 1450, durasi 0,0567 detik, dan iterasi sebanyak 4.
2. Run 2 menghasilkan final state dengan nilai objektif sebesar 1850, durasi 0,0514 detik, dan iterasi sebanyak 5.

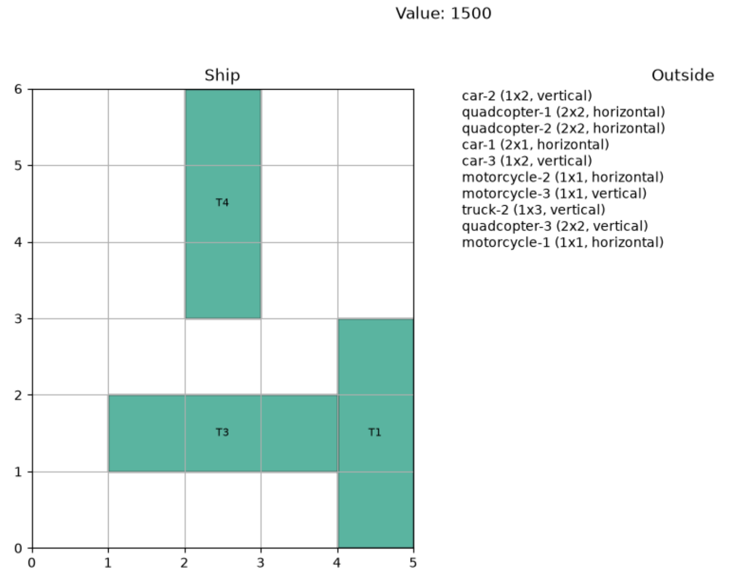
3. Run 3 menghasilkan final state dengan nilai objektif sebesar 1550, durasi 0,1291 detik, dan iterasi sebanyak 6.

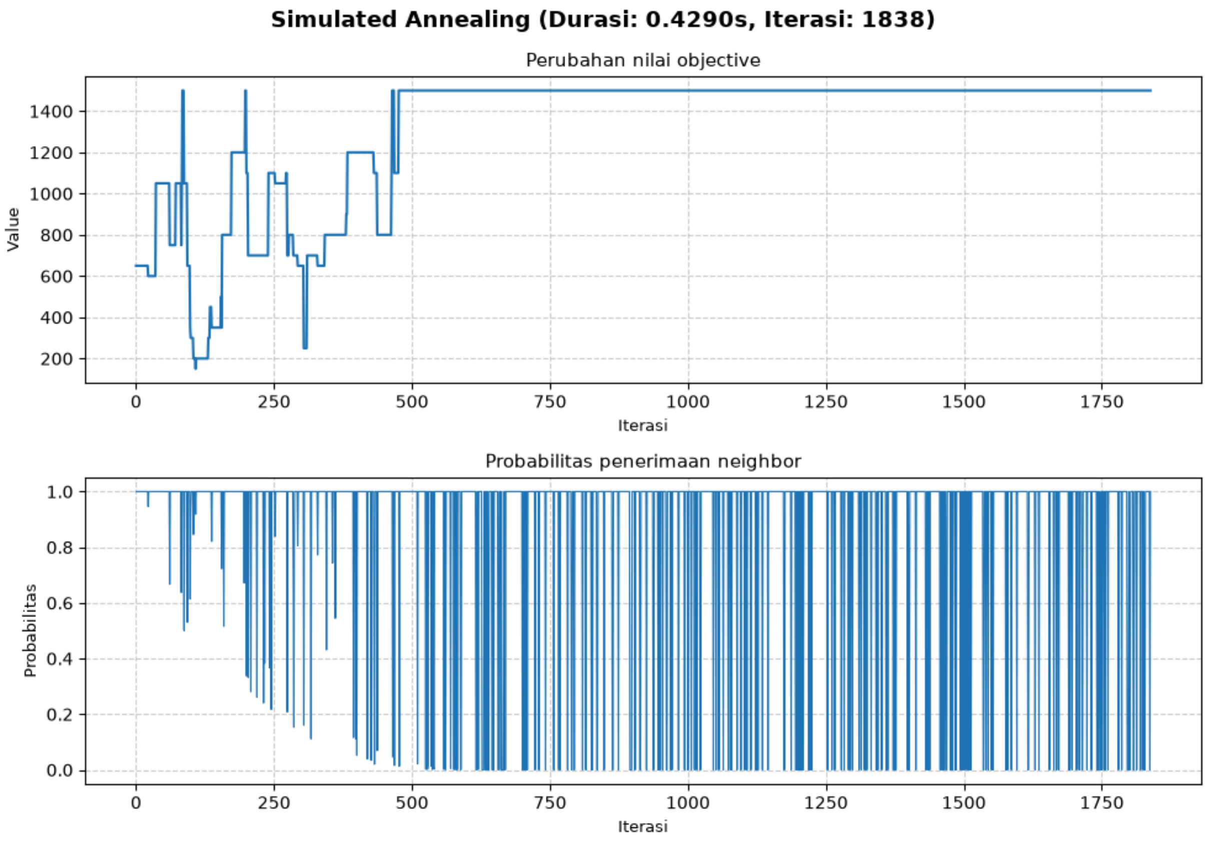
Berdasarkan tiga run tersebut, Stochastic Hill Climbing menghasilkan nilai objektif akhir rata-rata sebesar 1616,67. Algoritma ini berjalan dalam waktu singkat pada ketiga run, tetapi nilai objektif akhirnya lebih rendah dibandingkan Random Restart Hill Climbing dan Sideways Move Hill Climbing.

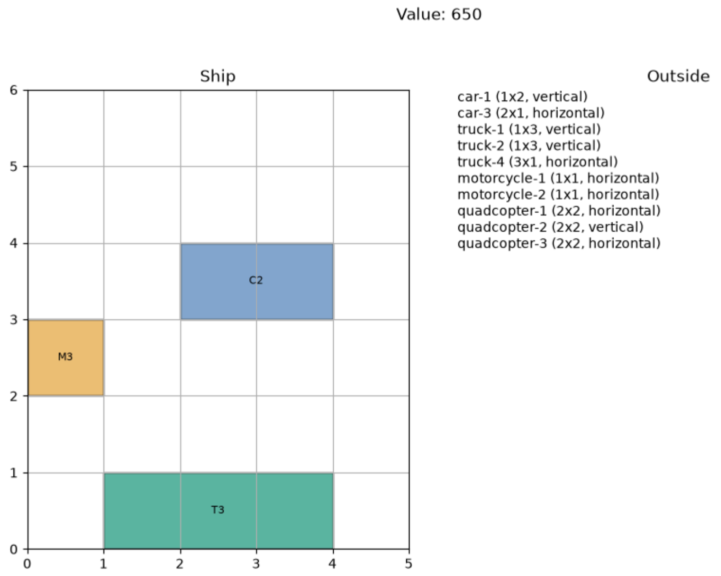
2.3.5. Algoritma *Simulated Annealing*

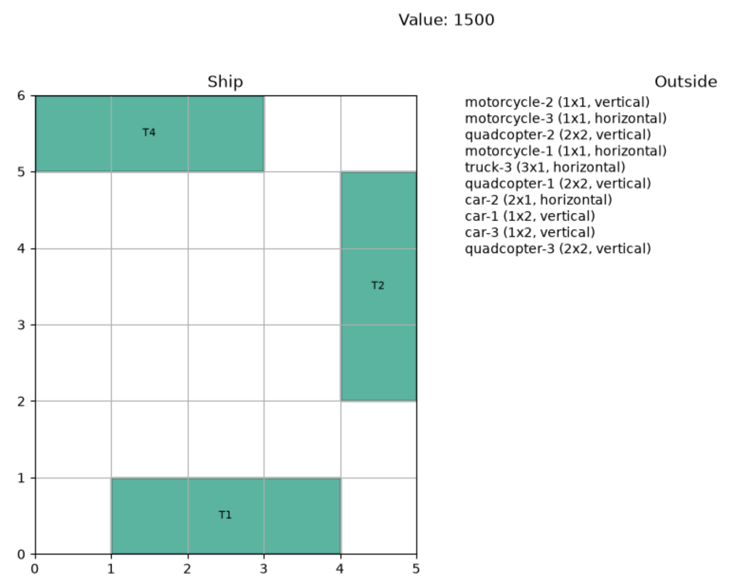
Berikut merupakan hasil eksperimen yang dilakukan sebanyak tiga kali menggunakan algoritma *steepest ascent hill-climbing*.

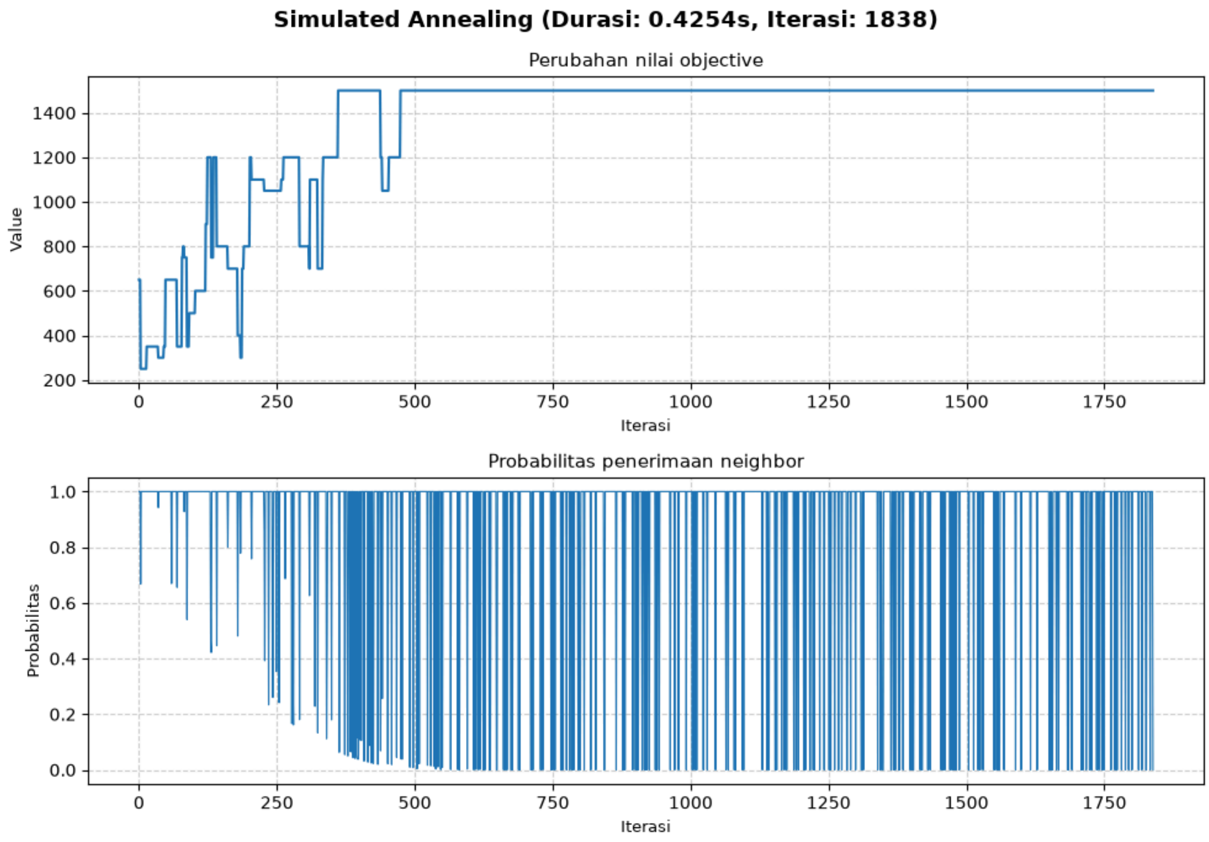
Eksperimen 1	
Visualisasi State Awal	 Value: 650 Ship C1 M1 T3 0 1 2 3 4 5 6 Outside car-2 (1x2, vertical) car-3 (1x2, vertical) truck-1 (1x3, vertical) truck-2 (1x3, vertical) truck-4 (1x3, vertical) motorcycle-2 (1x1, horizontal) motorcycle-3 (1x1, horizontal) quadcopter-1 (2x2, horizontal) quadcopter-2 (2x2, horizontal) quadcopter-3 (2x2, vertical)

Visualisasi State Akhir	 Value: 1500 Ship Outside car-2 (1x2, vertical) quadcopter-1 (2x2, horizontal) quadcopter-2 (2x2, horizontal) car-1 (2x1, horizontal) car-3 (1x2, vertical) motorcycle-2 (1x1, horizontal) motorcycle-3 (1x1, vertical) truck-2 (1x3, vertical) quadcopter-3 (2x2, vertical) motorcycle-1 (1x1, horizontal) T4 T3 T1 0 1 2 3 4 5 6 0 1 2 3 4 5
Nilai *objective function* yang dicapai	**1500**
Durasi Proses Pencarian	**0.4290s**
Banyak Iterasi	**1838**
Plot nilai objective function terhadap banyak iterasi yang dilewati	



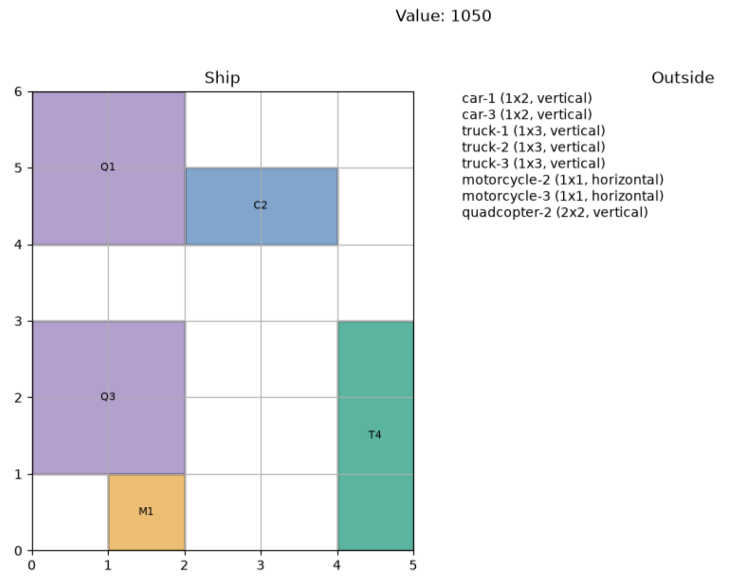
Eksperimen 2	
Visualisasi State Awal	 Value: 650 Ship C2 M3 T3 0 1 2 3 4 5 6 Outsidecar-1 (1x2, vertical) car-3 (2x1, horizontal) truck-1 (1x3, vertical) truck-2 (1x3, vertical) truck-4 (3x1, horizontal) motorcycle-1 (1x1, horizontal) motorcycle-2 (1x1, horizontal) quadcopter-1 (2x2, horizontal) quadcopter-2 (2x2, vertical) quadcopter-3 (2x2, horizontal)

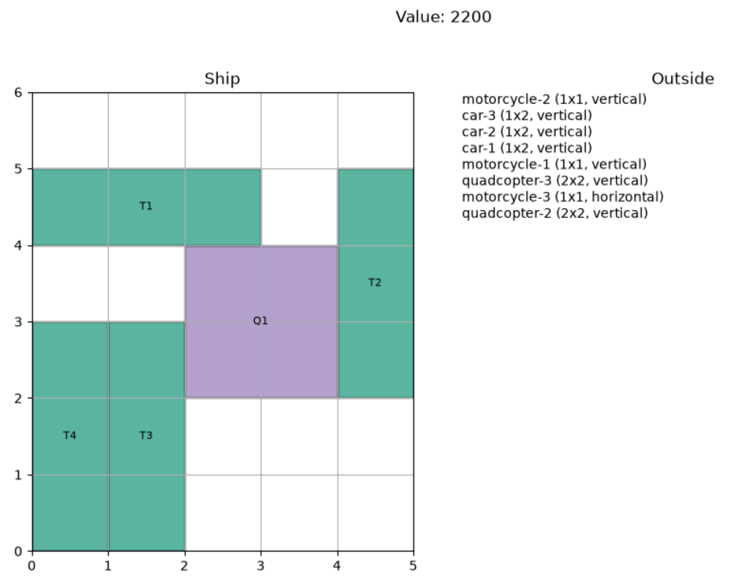
Visualisasi State Akhir	
Nilai *objective function* yang dicapai	**1500**
Durasi Proses Pencarian	**0.4254**
Banyak Iterasi	**1838**
Plot nilai objective function terhadap banyak iterasi yang dilewati	

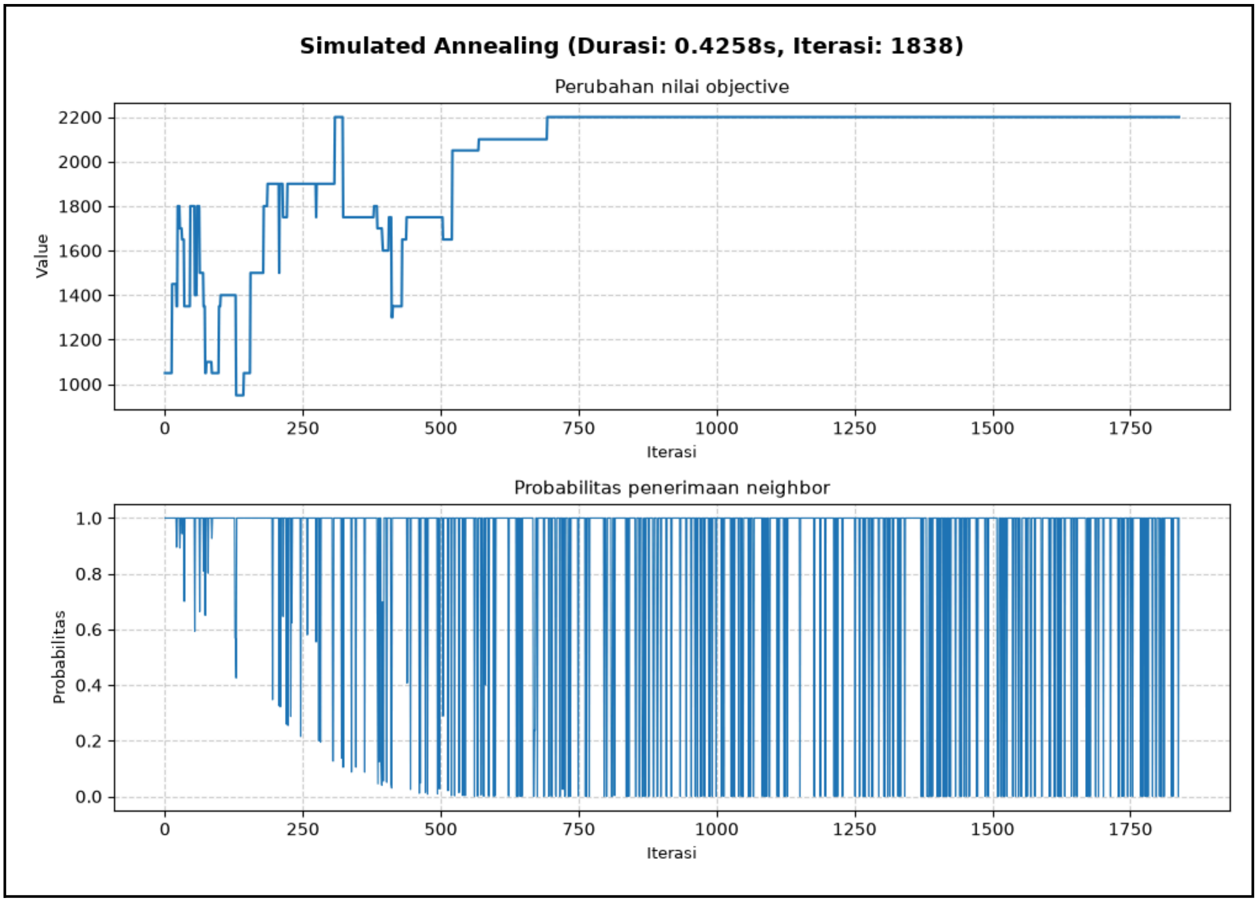


Eksperimen 3

Visualisasi State Awal


Value: 1050
Ship
Q1
C2
Q3
T4
M1
0
1
2
3
4
5
6
Outside
car-1 (1x2, vertical)
car-3 (1x2, vertical)
truck-1 (1x3, vertical)
truck-2 (1x3, vertical)
truck-3 (1x3, vertical)
motorcycle-2 (1x1, horizontal)
motorcycle-3 (1x1, horizontal)
quadcopter-2 (2x2, vertical)

Visualisasi State Akhir	
Nilai *objective function* yang dicapai	**2200**
Durasi Proses Pencarian	**0.4258s**
Banyak Iterasi	**1838**
Plot nilai objective function terhadap banyak iterasi yang dilewati	



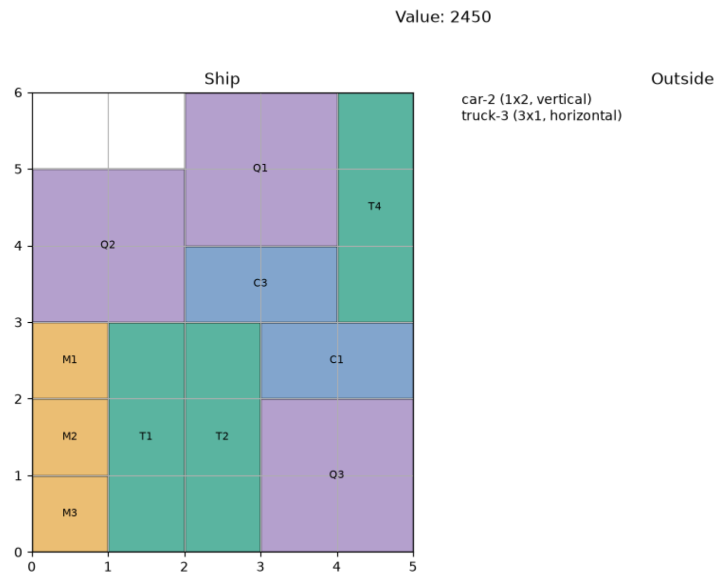
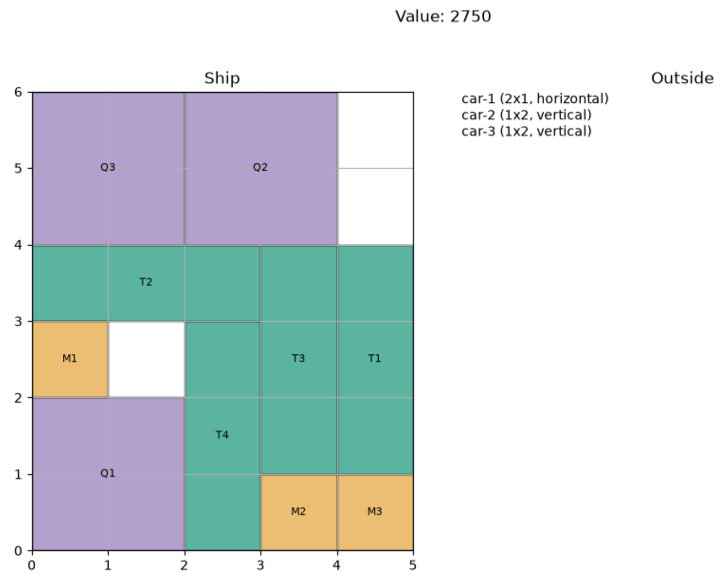
Dari ketiga eksperimen tersebut, didapatkan

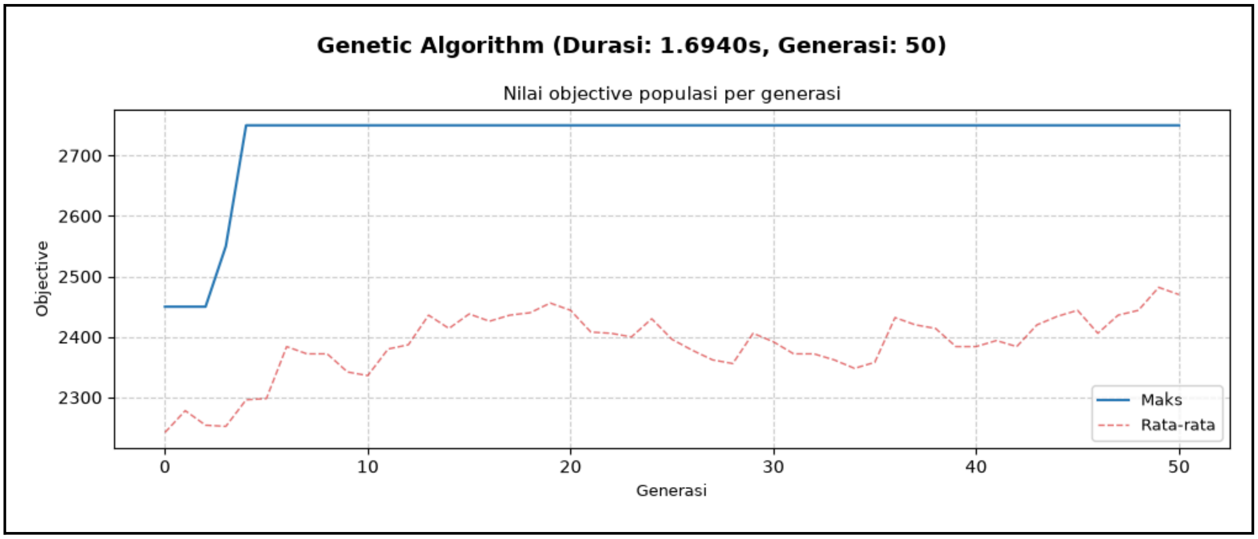
1. Run 1 menghasilkan final state dengan nilai objektif sebesar 1500, durasi 0,429 detik, dan iterasi sebanyak 1838.
2. Run 2 menghasilkan final state dengan nilai objektif sebesar 1500, durasi 0,4254 detik, dan iterasi sebanyak 1838.
3. Run 3 menghasilkan final state dengan nilai objektif sebesar 2200, durasi 0,4258 detik, dan iterasi sebanyak 1838.

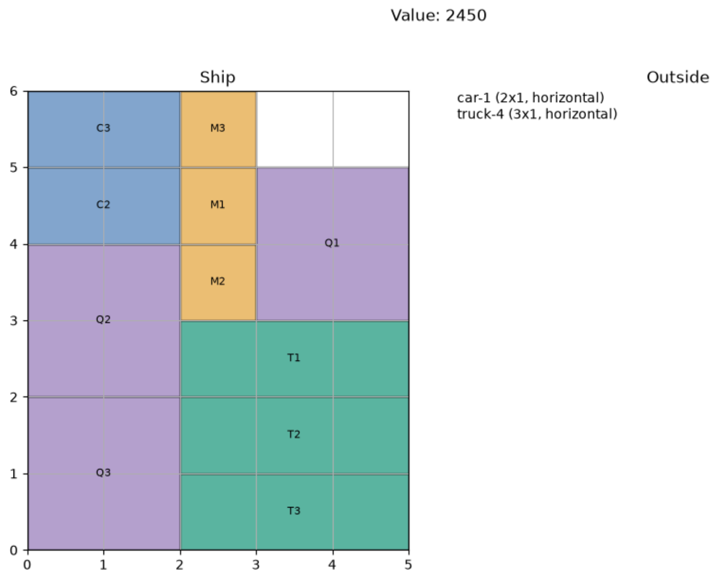
Berdasarkan tiga run yang telah dijalankan, Simulated Annealing menghasilkan nilai objektif akhir rata-rata sebesar 1733,33. Jumlah iterasi sama pada setiap run, tetapi nilai objektif akhirnya berbeda. Run 3 memperoleh hasil terbaik, sedangkan Run 1 dan Run 2 memperoleh nilai yang sama.

2.3.6. Algoritma *Genetic*

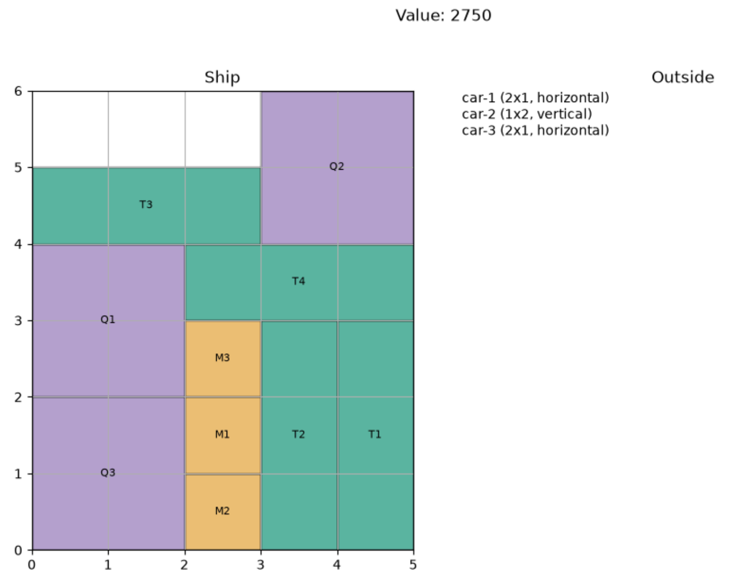
Berikut merupakan hasil eksperimen yang dilakukan sebanyak tiga kali menggunakan algoritma *steepest ascent hill-climbing.*

Eksperimen 1	
Visualisasi State Awal	 Value: 2450 Ship Outside car-2 (1x2, vertical) truck-3 (3x1, horizontal) Q1 T4 Q2 C3 M1 C1 M2 T1 T2 Q3 M3 0 1 2 3 4 5 0 1 2 3 4 5 6
Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (2x1, horizontal) car-2 (1x2, vertical) car-3 (1x2, vertical) Q3 Q2 T2 M1 T3 T1 T4 Q1 M2 M3 0 1 2 3 4 5 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***1.6940s***
Banyak Iterasi	***50***
Jumlah Populasi	***50***
Plot nilai objective function terhadap banyak iterasi yang dilewati	

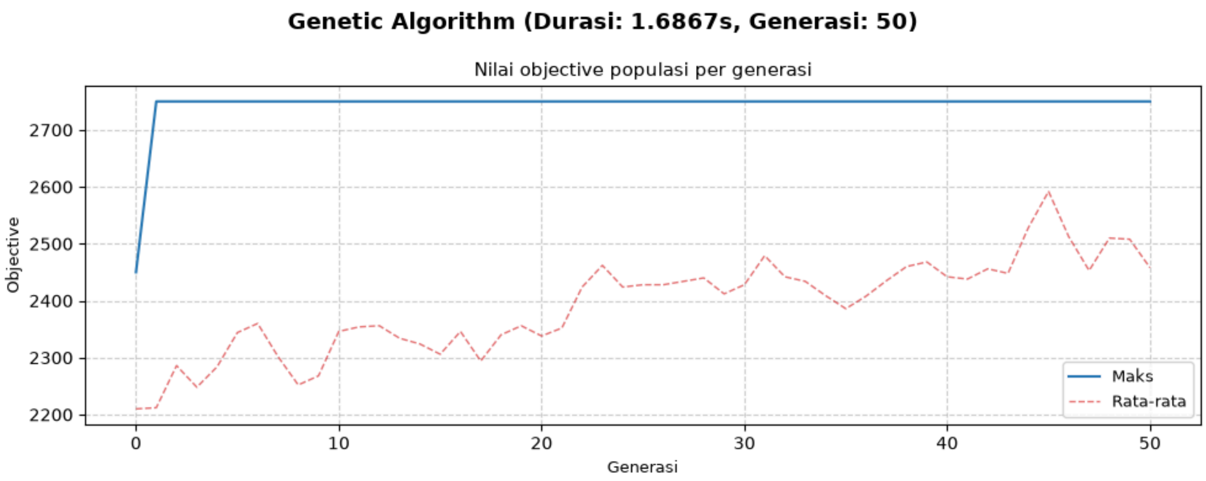


<table>
<tr><th colspan="2">Eksperimen 2</th></tr>
<tr><td>Visualisasi State Awal</td><td>
Value: 2450
Ship
Outside
car-1 (2x1, horizontal)
truck-4 (3x1, horizontal)
C3
M3
C2
M1
Q1
M2
Q2
T1
T2
Q3
T3
0
1
2
3
4
5
6

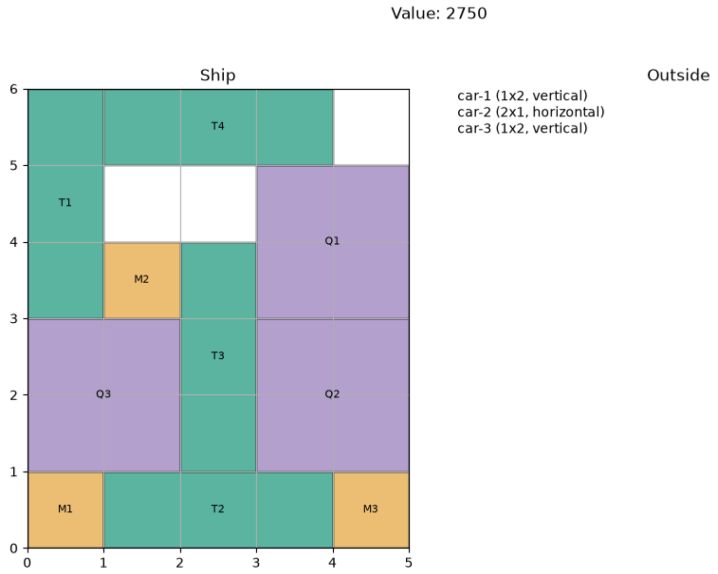
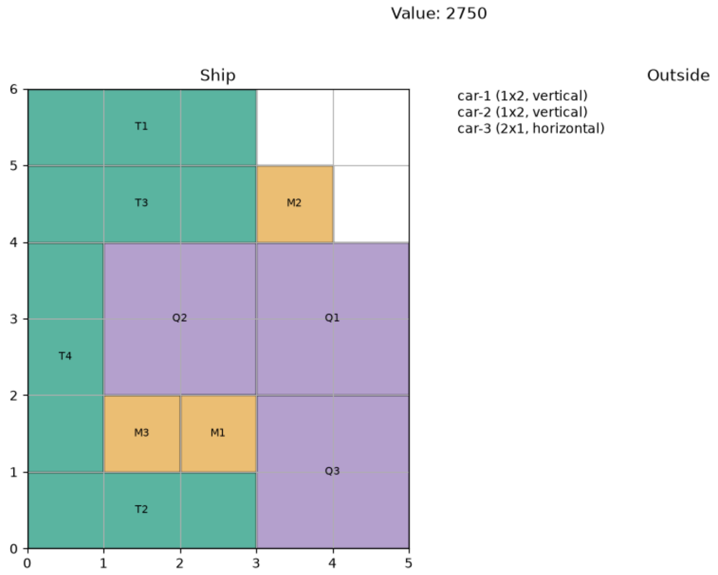
</td></tr>
</table>

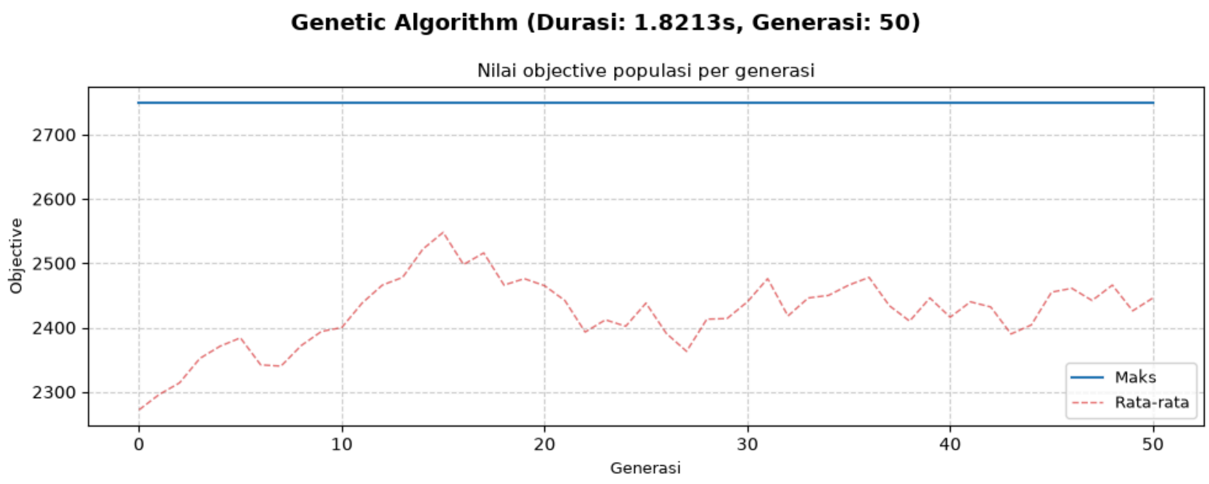
Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (2x1, horizontal) car-2 (1x2, vertical) car-3 (2x1, horizontal) Q2 T3 Q1 T4 M3 M1 T2 T1 Q3 M2 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***1.6867s***
Banyak Iterasi	***50***
Jumlah Populasi	***50***

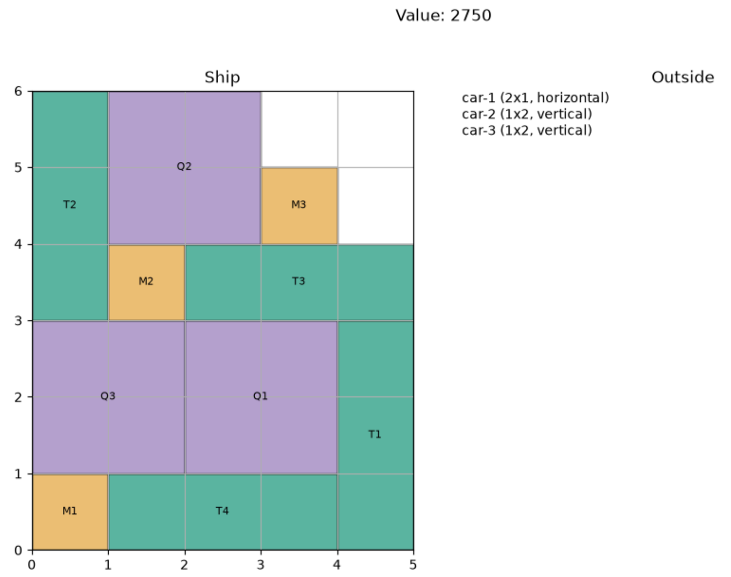
Plot nilai objective function terhadap banyak iterasi yang dilewati



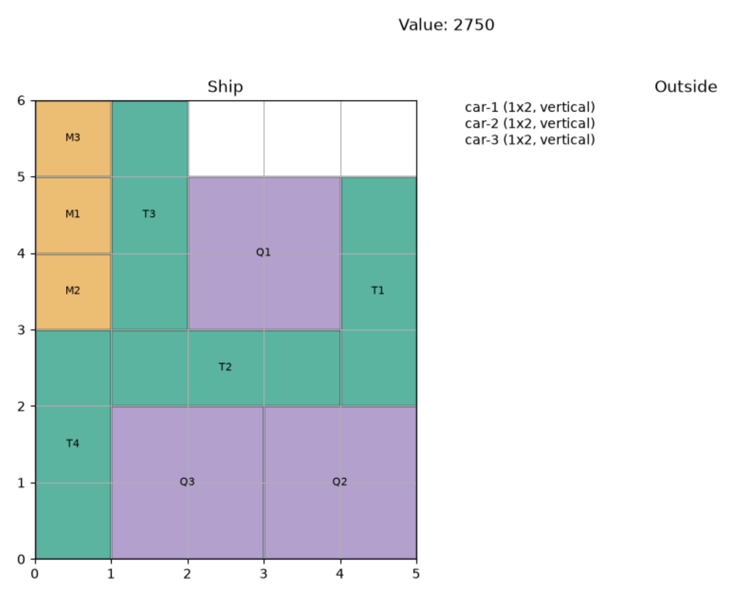
Eksperimen 3

Visualisasi State Awal	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (2x1, horizontal) car-3 (1x2, vertical) T4 T1 Q1 M2 T3 Q3 Q2 M1 T2 M3 0 1 2 3 4 5 6
Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (1x2, vertical) car-3 (2x1, horizontal) T1 T3 M2 Q2 Q1 T4 M3 M1 Q3 T2 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***1.8213s***
Banyak Iterasi	***50***
Jumlah Populasi	***50***
Plot nilai objective function terhadap banyak iterasi yang dilewati	



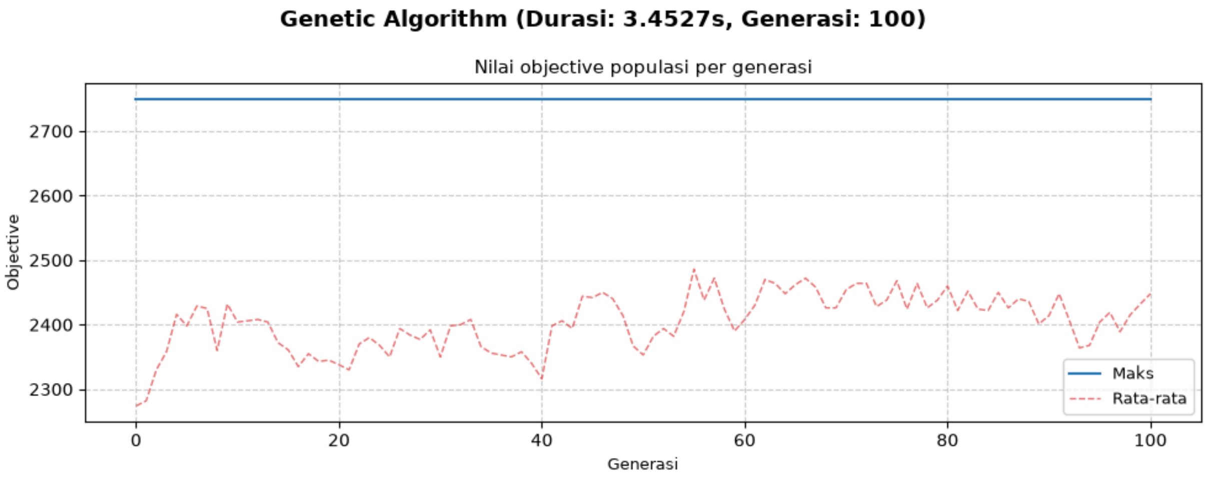
Eksperimen 4	
Visualisasi State Awal	 Value: 2750 Ship Outside car-1 (2x1, horizontal) car-2 (1x2, vertical) car-3 (1x2, vertical) T2 Q2 M3 M2 T3 Q3 Q1 T1 M1 T4 0 1 2 3 4 5 6

Visualisasi State Akhir

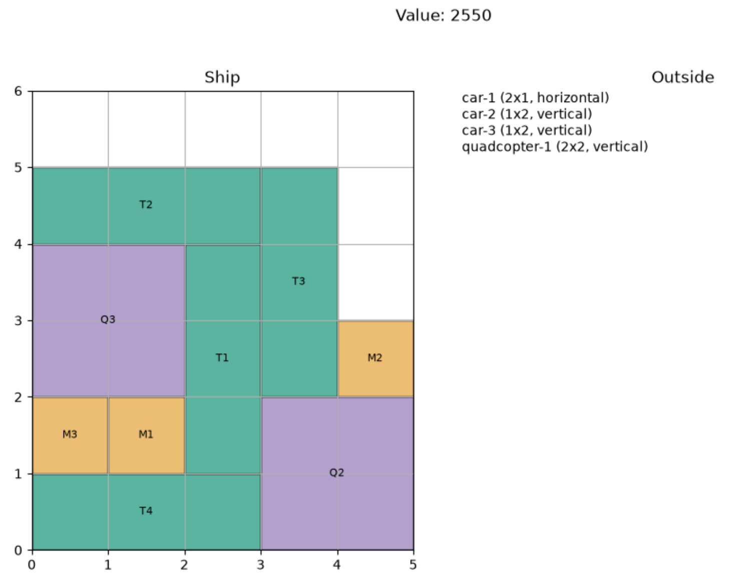
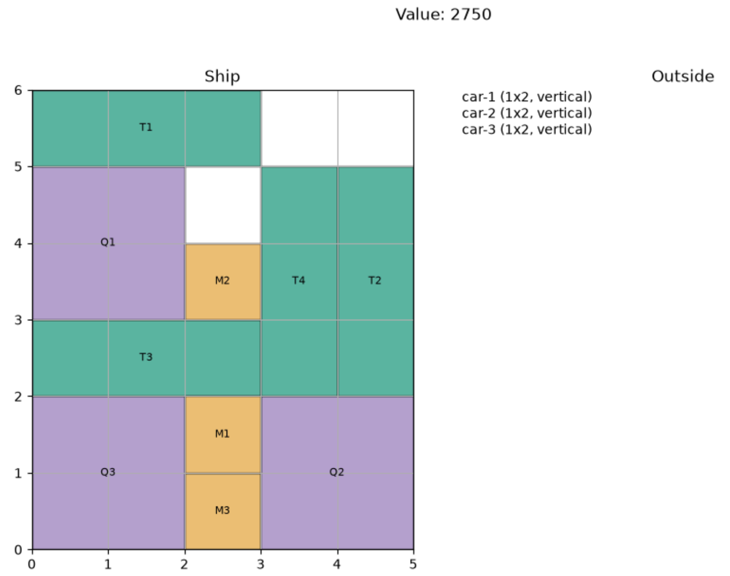

Value: 2750
Ship
Outside
car-1 (1x2, vertical)
car-2 (1x2, vertical)
car-3 (1x2, vertical)
M3
M1
M2
T3
Q1
T1
T2
T4
Q3
Q2
0
1
2
3
4
5
6
0
1
2
3
4
5

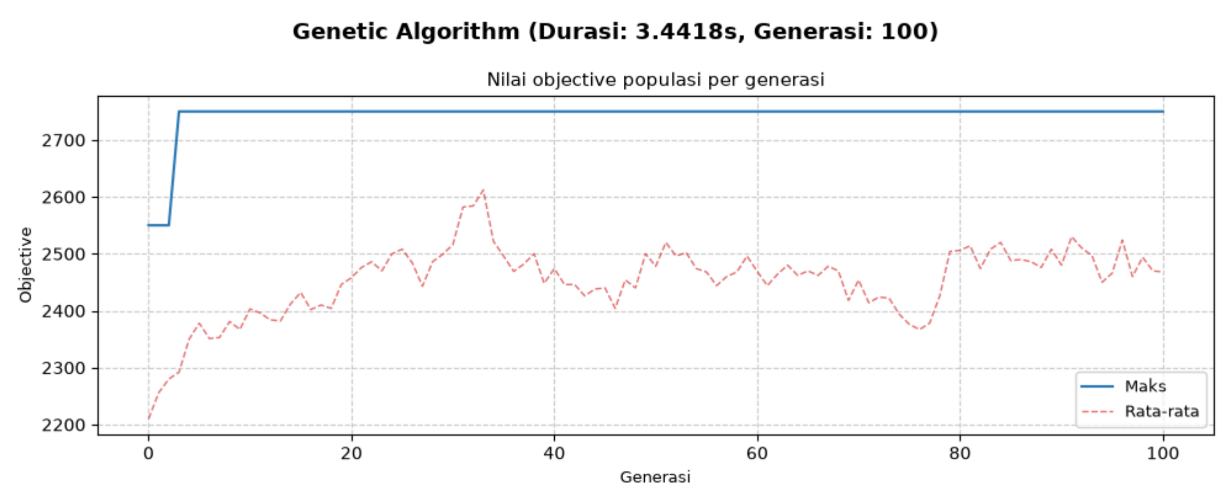
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***3.4527s***
Banyak Iterasi	***100***
Jumlah Populasi	***50***

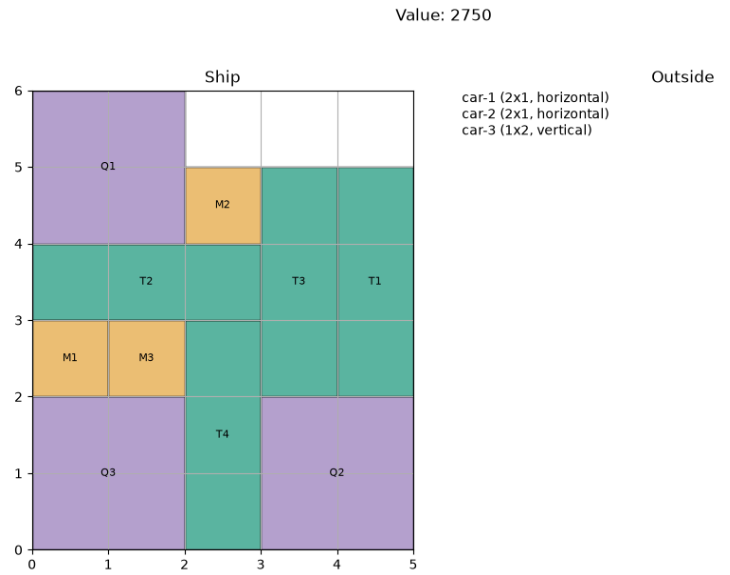
Plot nilai objective function terhadap banyak iterasi yang dilewati

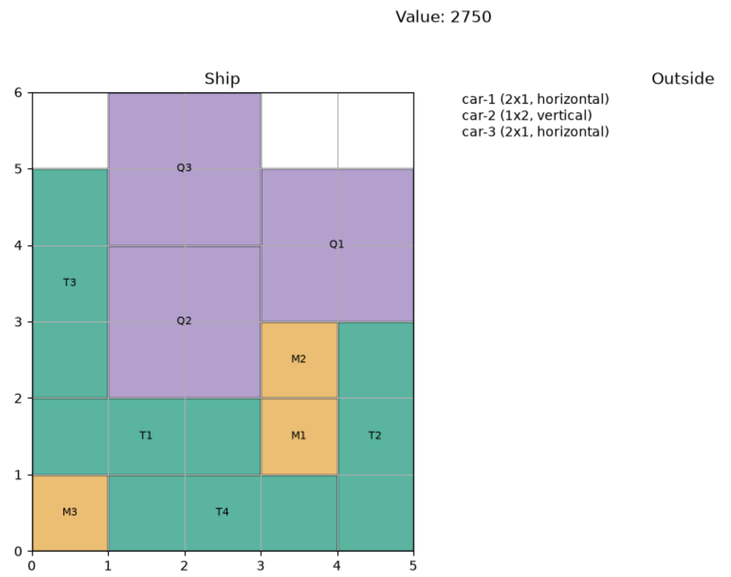
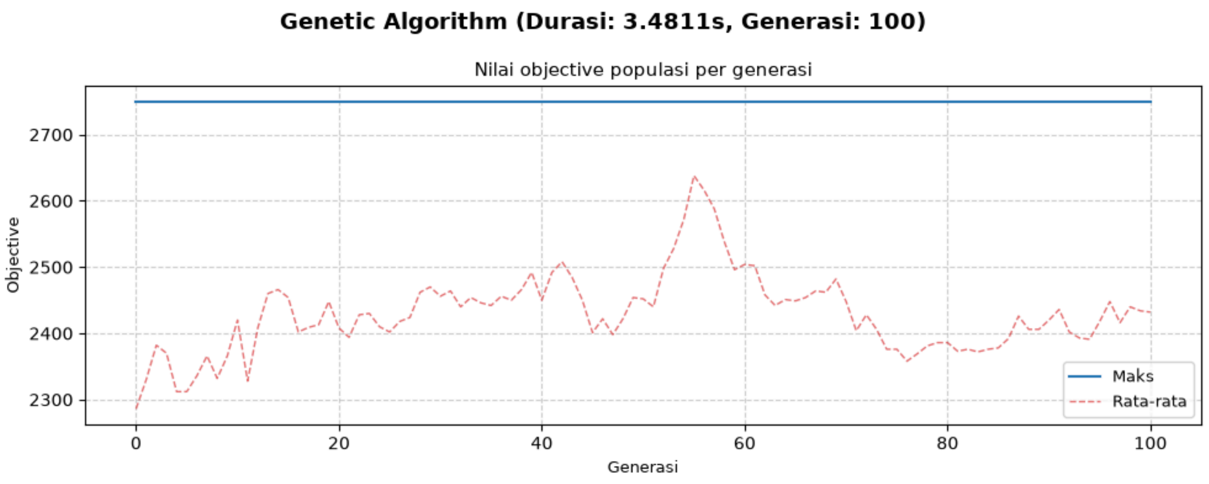


Eksperimen 5

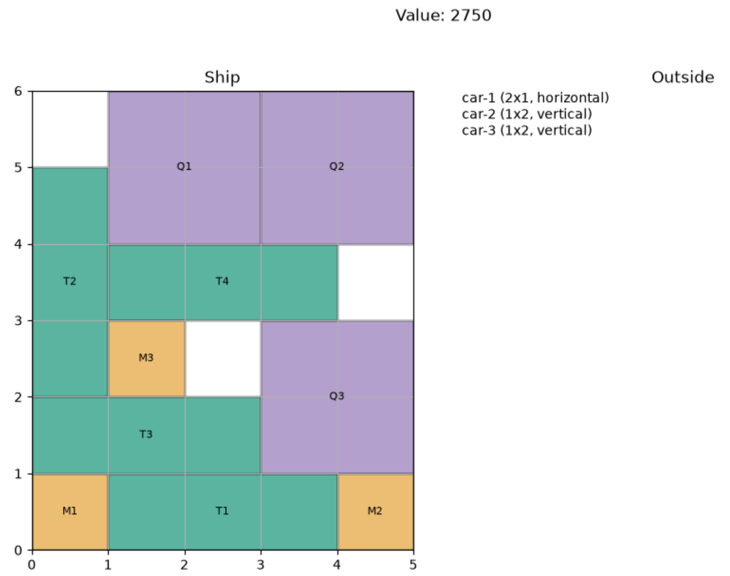
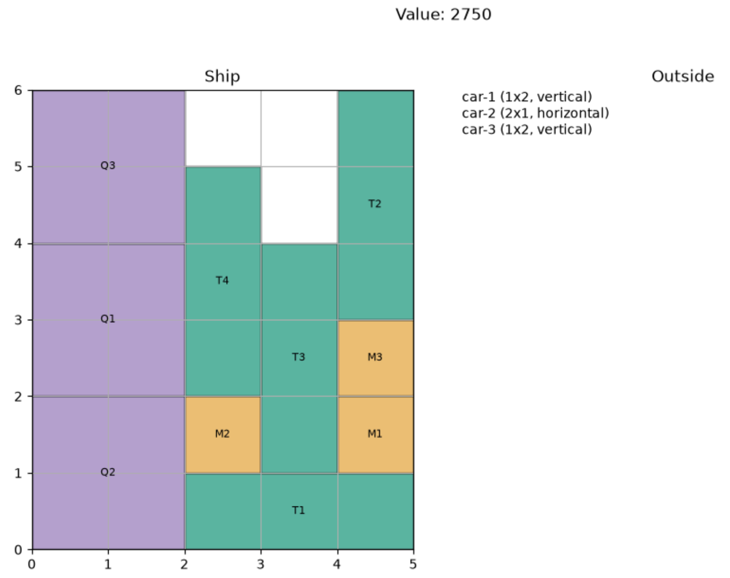
Visualisasi State Awal	 Value: 2550 Ship Outside T2 T3 Q3 T1 M2 M3 M1 Q2 T4 0 1 2 3 4 5 6 car-1 (2x1, horizontal) car-2 (1x2, vertical) car-3 (1x2, vertical) quadcopter-1 (2x2, vertical)
Visualisasi State Akhir	 Value: 2750 Ship Outside T1 Q1 M2 T4 T2 T3 M1 Q3 Q2 M3 0 1 2 3 4 5 6 car-1 (1x2, vertical) car-2 (1x2, vertical) car-3 (1x2, vertical)
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***3.4418s***
Banyak Iterasi	***100***
Jumlah Populasi	***50***
Plot nilai objective function terhadap banyak iterasi yang dilewati	

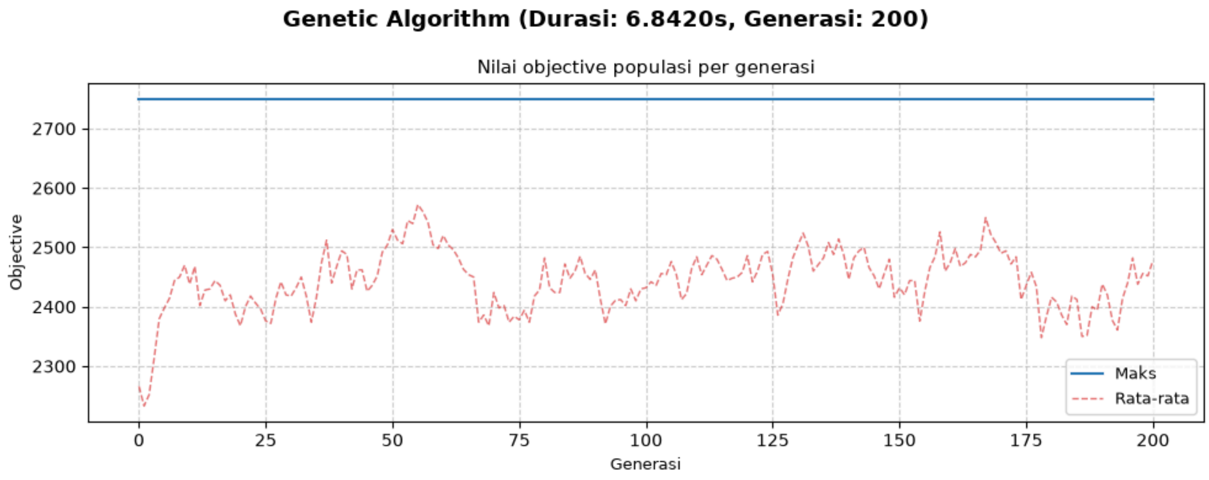


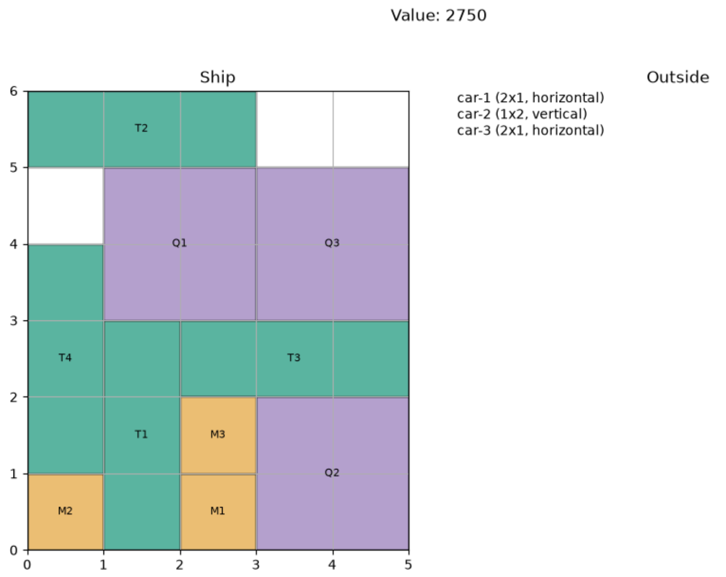
Eksperimen 6	
Visualisasi State Awal	 Value: 2750 Ship Outside car-1 (2x1, horizontal) car-2 (2x1, horizontal) car-3 (1x2, vertical) Q1 M2 T2 T3 T1 M1 M3 T4 Q3 Q2 0 1 2 3 4 5 6

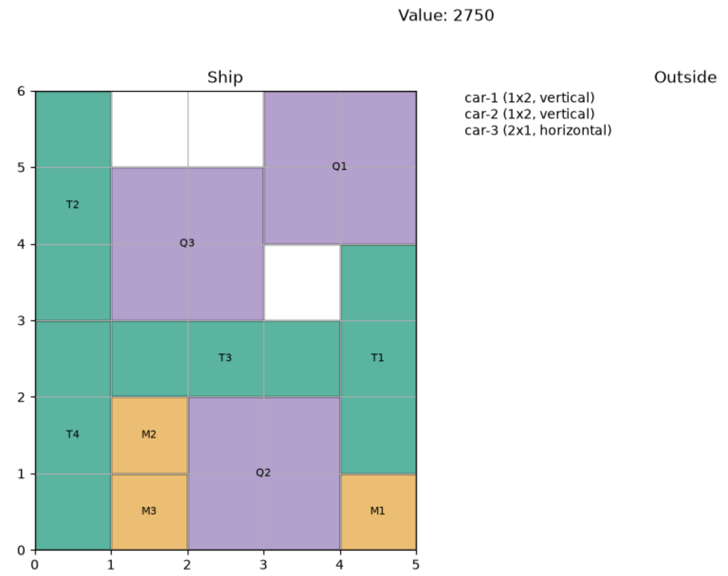
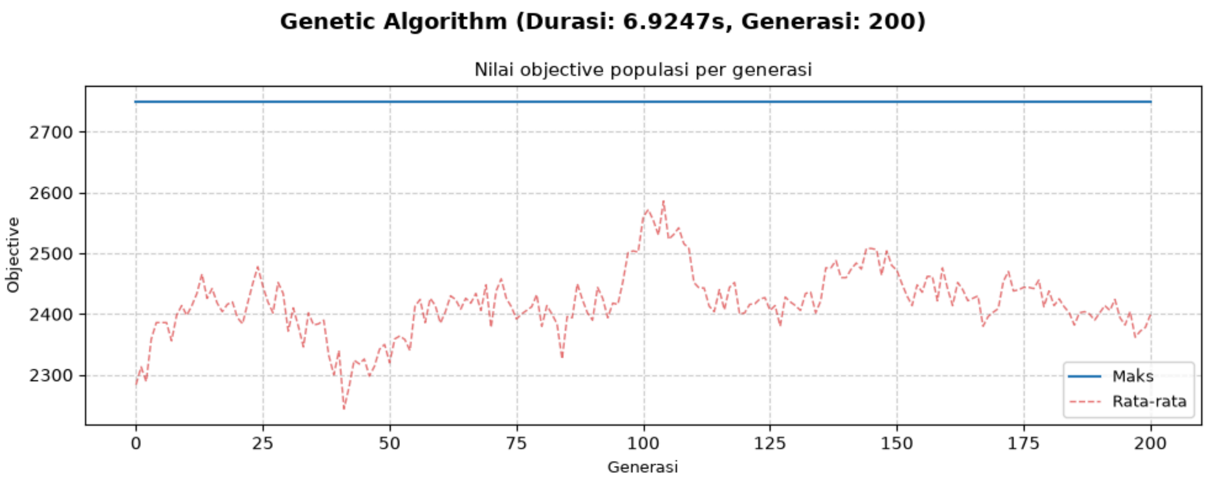
Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (2x1, horizontal) car-2 (1x2, vertical) car-3 (2x1, horizontal) Q3 Q1 T3 Q2 M2 T1 M1 T2 M3 T4 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***3.4811s***
Banyak Iterasi	***100***
Jumlah Populasi	***50***
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

Eksperimen 7

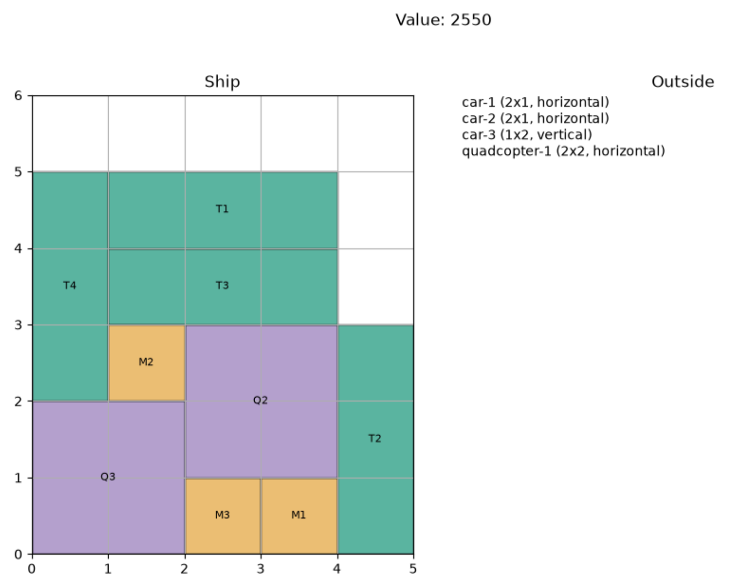
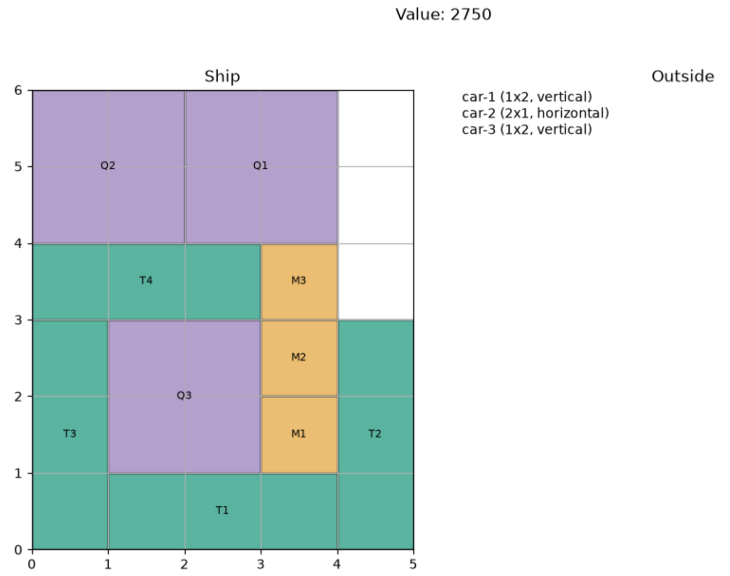
Visualisasi State Awal	 Value: 2750 Ship Outside car-1 (2x1, horizontal) car-2 (1x2, vertical) car-3 (1x2, vertical) Q1 Q2 T2 T4 M3 Q3 T3 M1 T1 M2 0 1 2 3 4 5 6
Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (2x1, horizontal) car-3 (1x2, vertical) Q3 T2 T4 Q1 T3 M3 M2 M1 Q2 T1 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***6.8420s***
Banyak Iterasi	***200***
Jumlah Populasi	***50***
Plot nilai objective function terhadap banyak iterasi yang dilewati	

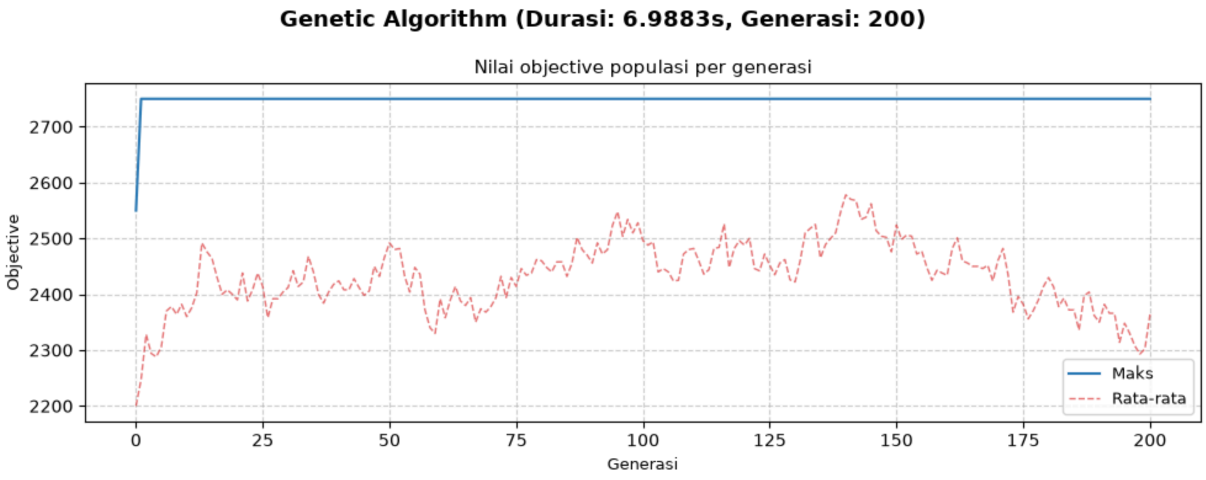


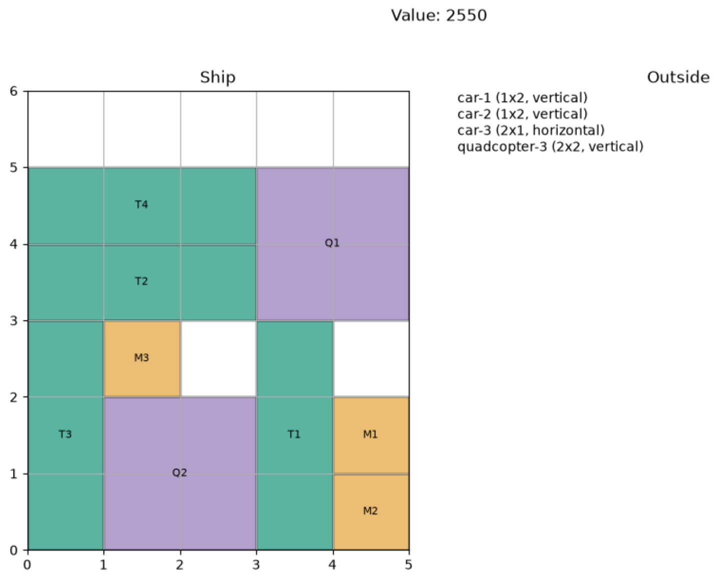
Eksperimen 8	
Visualisasi State Awal	 Value: 2750 Ship Outside car-1 (2x1, horizontal) car-2 (1x2, vertical) car-3 (2x1, horizontal) T2 Q1 Q3 T4 T3 T1 M3 Q2 M2 M1 0 1 2 3 4 5 6

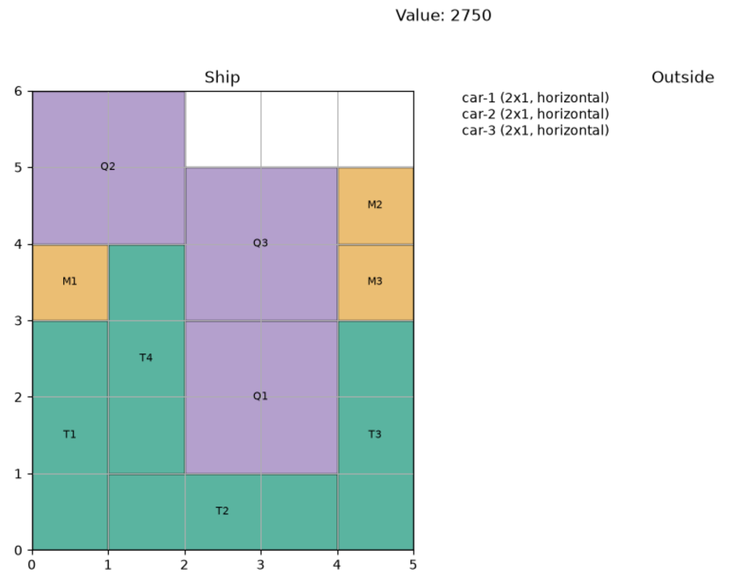
Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (1x2, vertical) car-3 (2x1, horizontal) T2 Q1 Q3 T3 T1 T4 M2 Q2 M3 M1 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***6.9247s***
Banyak Iterasi	***200***
Jumlah Populasi	***50***
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

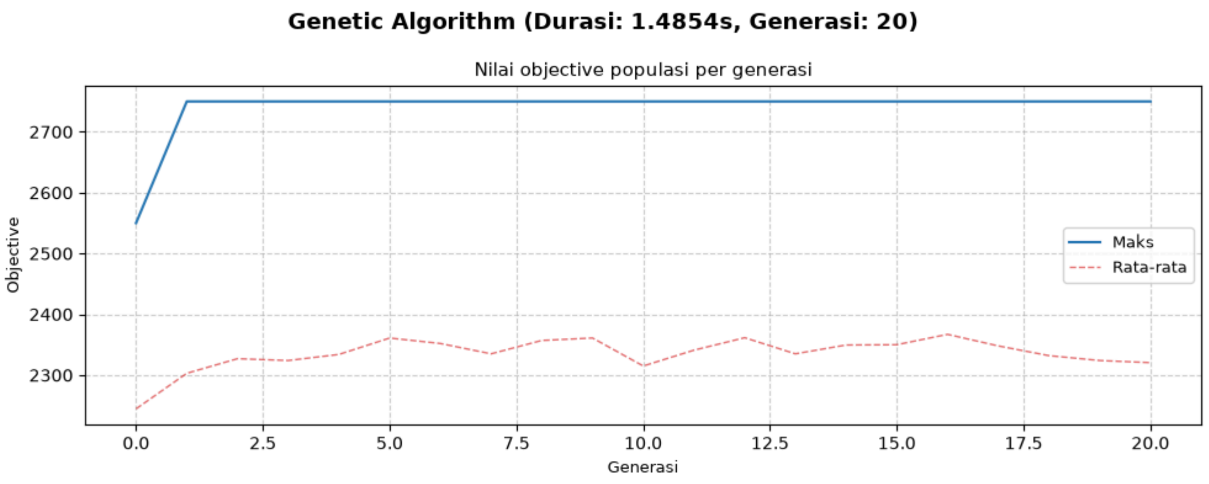
Eksperimen 9

Visualisasi State Awal	
Visualisasi State Akhir	
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***6.9883s***
Banyak Iterasi	***200***
Jumlah Populasi	***50***
Plot nilai objective function terhadap banyak iterasi yang dilewati	

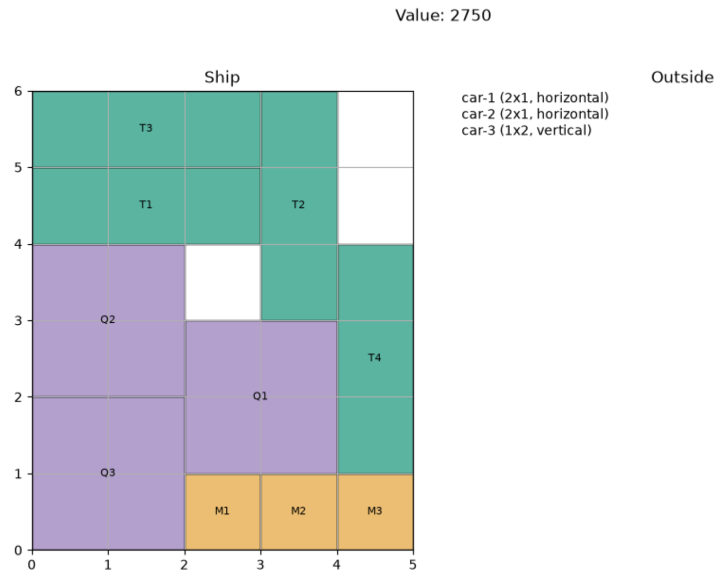
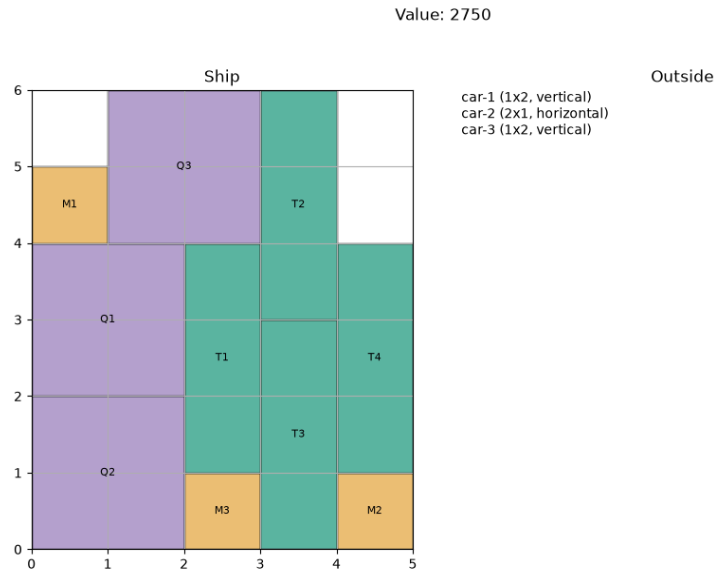


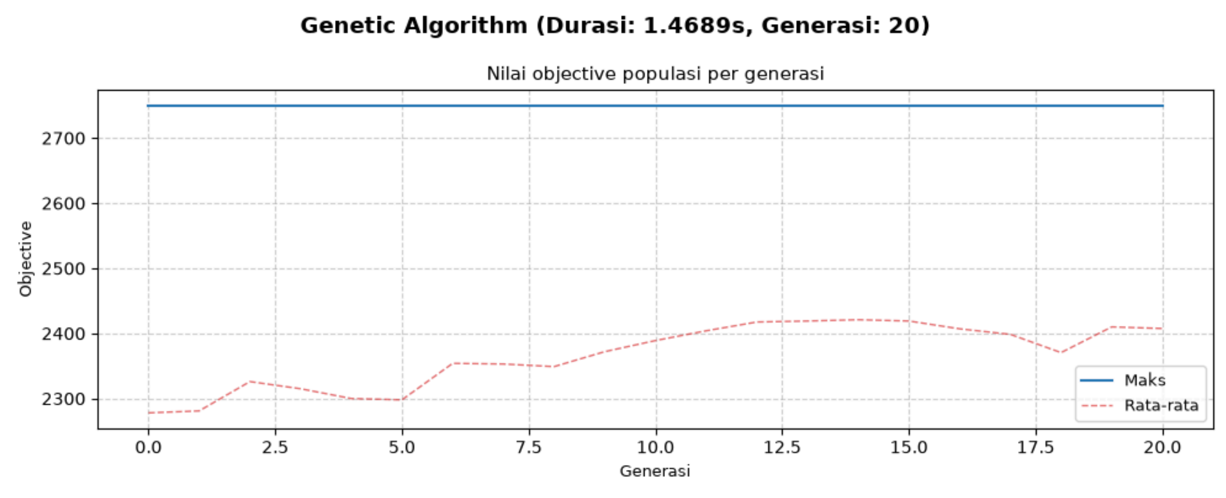
Eksperimen 10	
Visualisasi State Awal	 Value: 2550 Ship Outside car-1 (1x2, vertical) car-2 (1x2, vertical) car-3 (2x1, horizontal) quadcopter-3 (2x2, vertical) T4 T2 Q1 M3 T3 Q2 T1 M1 M2 0 1 2 3 4 5 6

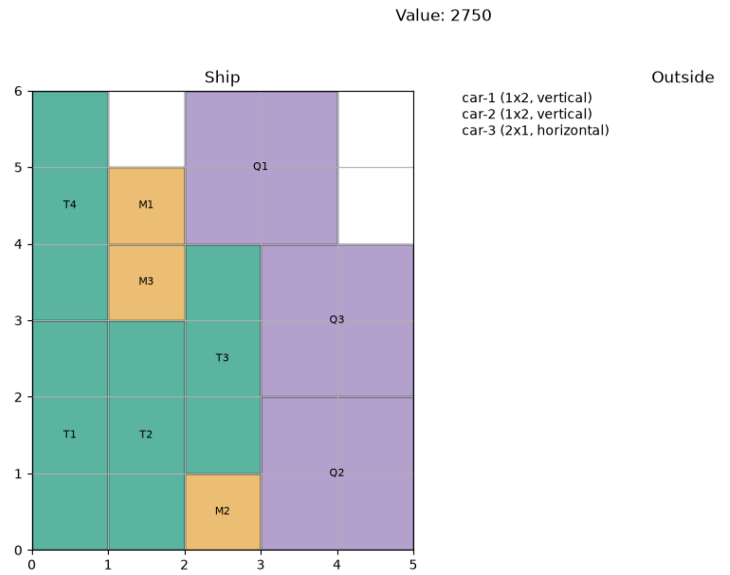
<table>
<tr><td>Visualisasi State Akhir</td><td colspan="2">
Value: 2750
Ship
Outside
car-1 (2x1, horizontal)
car-2 (2x1, horizontal)
car-3 (2x1, horizontal)
Q2
Q3
M2
M1
T4
M3
Q1
T1
T3
T2
0
1
2
3
4
5
6

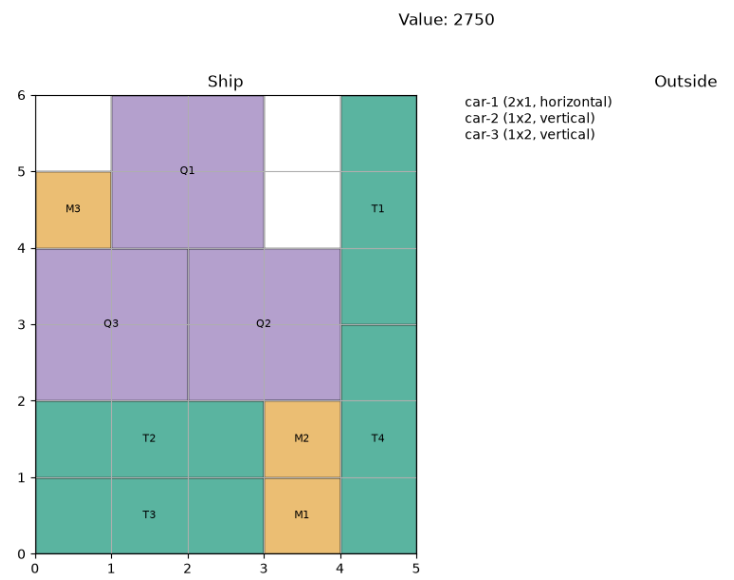
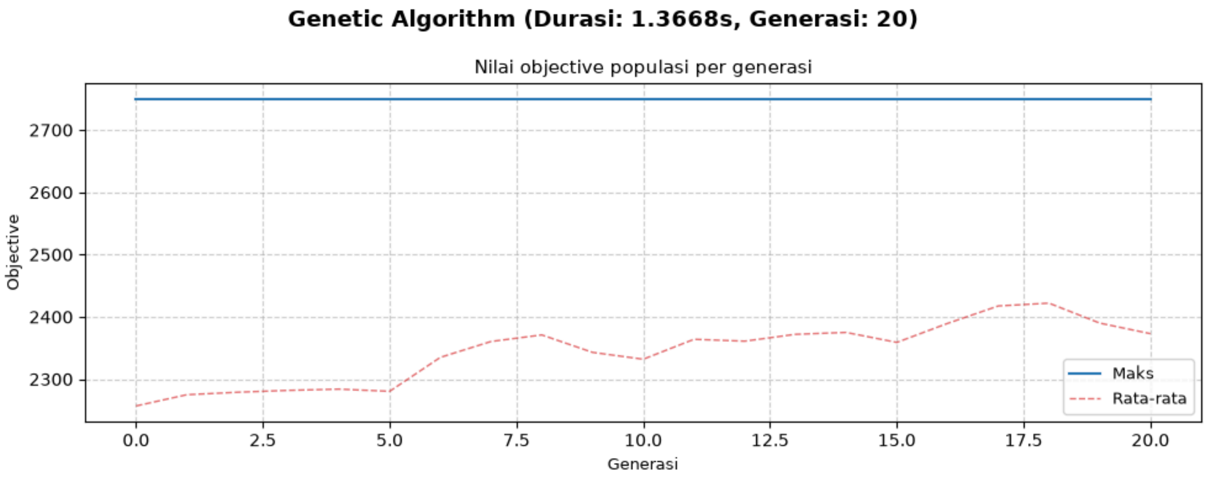
</td></tr>
<tr><td colspan="2">Nilai <i>objective function</i> yang dicapai</td><td>2750</td></tr>
<tr><td colspan="2">Durasi Proses Pencarian</td><td><i>1.4854s</i></td></tr>
<tr><td colspan="2">Banyak Iterasi</td><td><i>20</i></td></tr>
<tr><td colspan="2">Jumlah Populasi</td><td><i>100</i></td></tr>
<tr><td colspan="3">Plot nilai objective function terhadap banyak iterasi yang dilewati</td></tr>
<tr><td colspan="3">
</td></tr>
</table>

Eksperimen 11

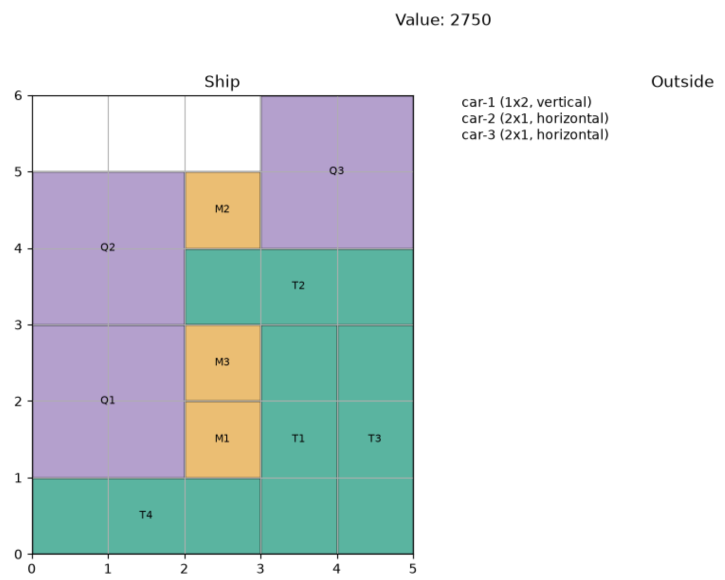
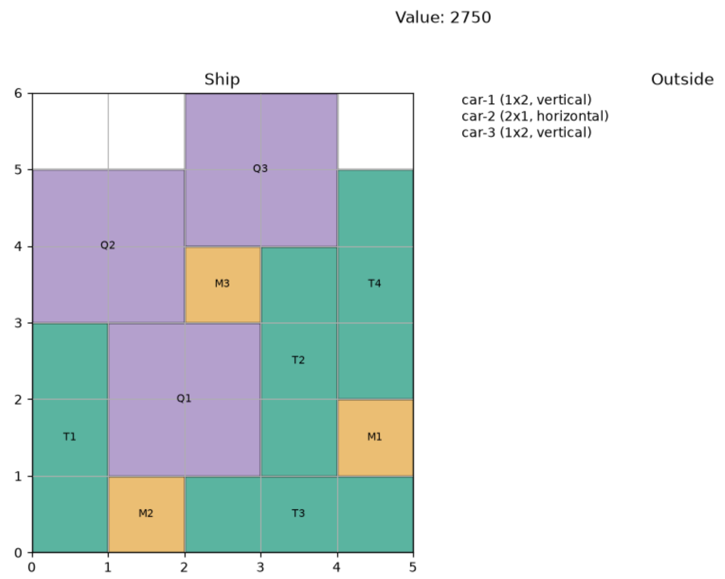
Visualisasi State Awal	
Visualisasi State Akhir	
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***1.4689s***
Banyak Iterasi	***20***
Jumlah Populasi	***100***
Plot nilai objective function terhadap banyak iterasi yang dilewati	

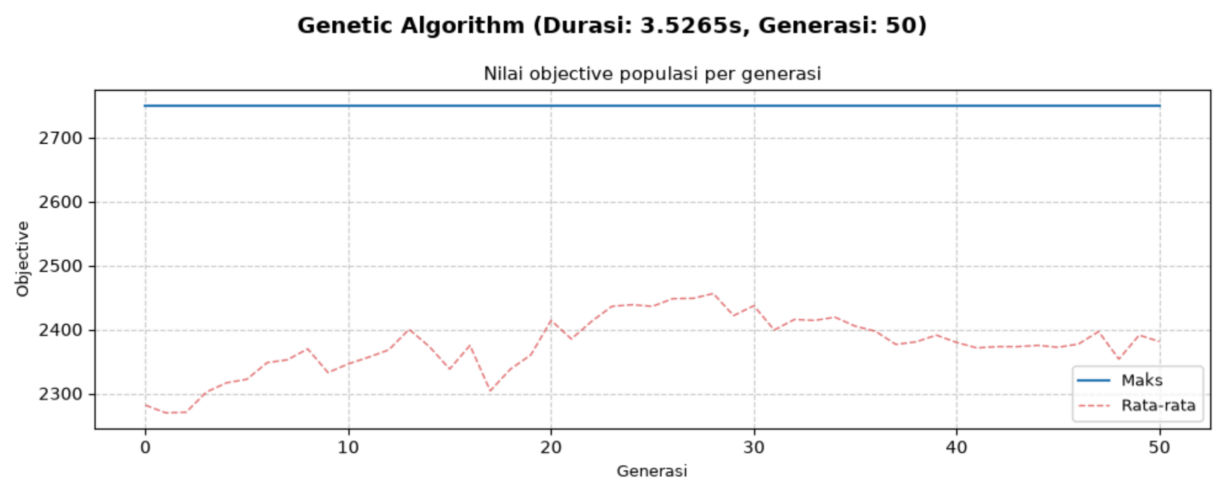


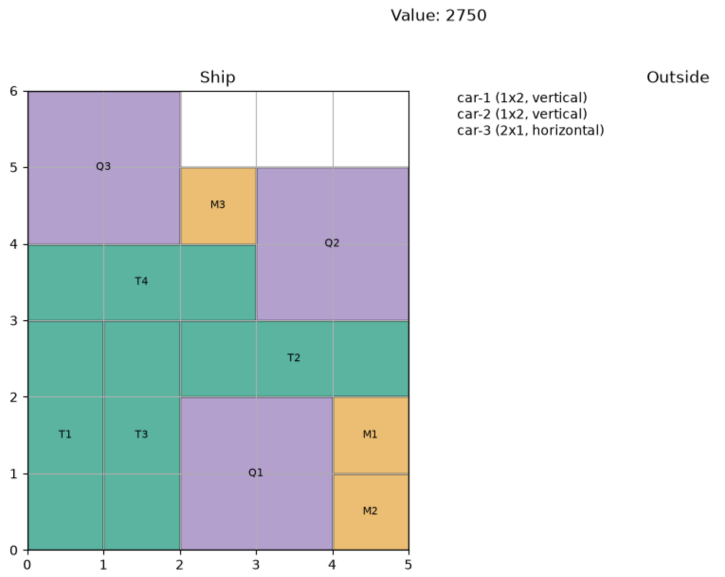
Eksperimen 12	
Visualisasi State Awal	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (1x2, vertical) car-3 (2x1, horizontal) T4 M1 Q1 M3 T3 Q3 T1 T2 Q2 M2 0 1 2 3 4 5 6

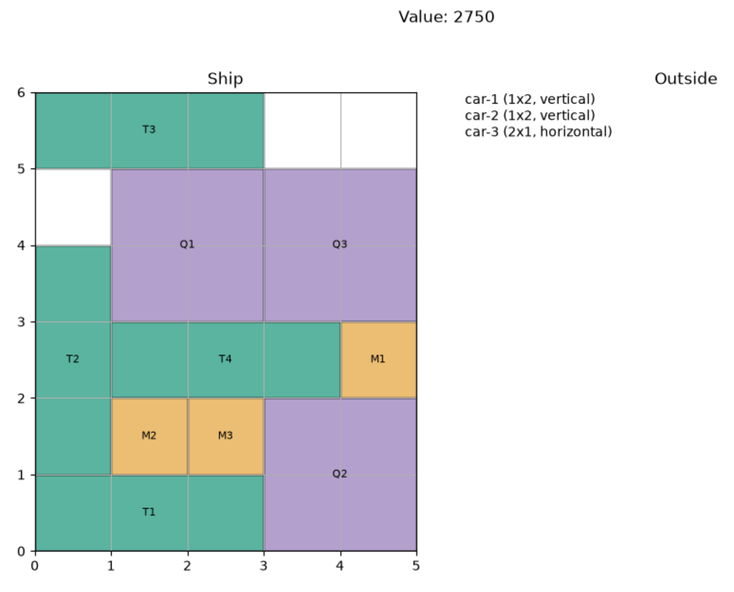
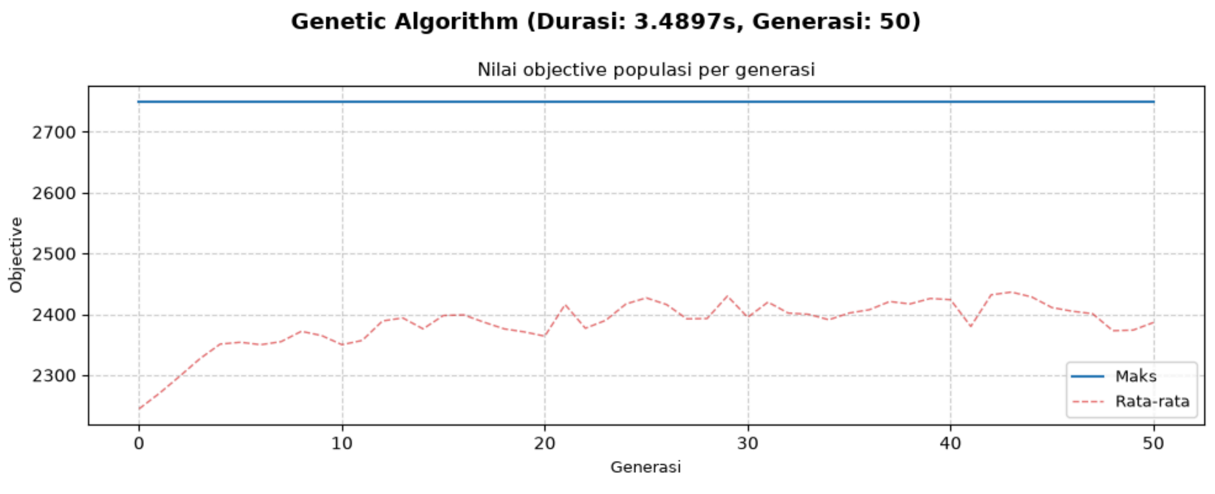
Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (2x1, horizontal) car-2 (1x2, vertical) car-3 (1x2, vertical) Q1 M3 T1 Q3 Q2 T2 M2 T4 T3 M1 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***1.3668s***
Banyak Iterasi	***20***
Jumlah Populasi	***100***
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

Eksperimen 13

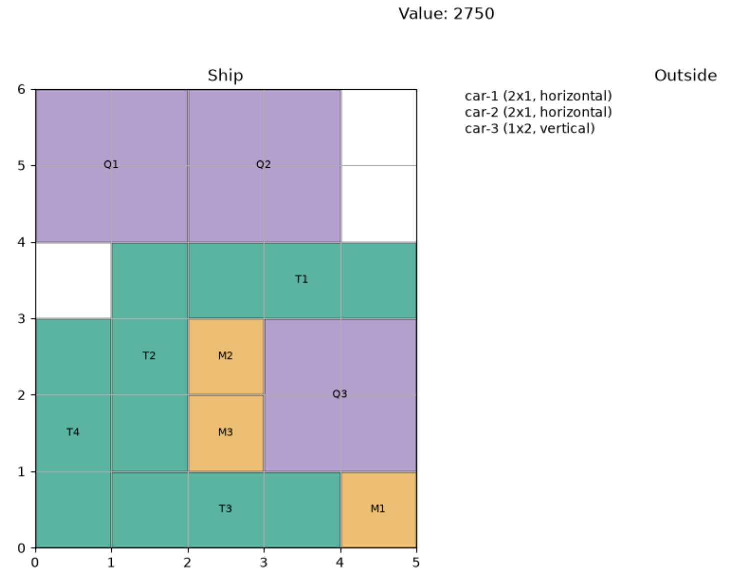
<table>
<tr><td>Visualisasi State Awal</td><td>

Value: 2750
Ship
Outside
car-1 (1x2, vertical)
car-2 (2x1, horizontal)
car-3 (2x1, horizontal)
Q3
M2
Q2
T2
M3
Q1
M1
T1
T3
T4
0
1
2
3
4
5
6
0
1
2
3
4
5
</td></tr>
<tr><td>Visualisasi State Akhir</td><td>

Value: 2750
Ship
Outside
car-1 (1x2, vertical)
car-2 (2x1, horizontal)
car-3 (1x2, vertical)
Q3
Q2
M3
T4
T2
Q1
T1
M1
M2
T3
0
1
2
3
4
5
6
0
1
2
3
4
5
</td></tr>
<tr><td>Nilai <i>objective function</i> yang dicapai</td><td>2750</td></tr>
<tr><td>Durasi Proses Pencarian</td><td><i>3.5265s</i></td></tr>
<tr><td>Banyak Iterasi</td><td><i>50</i></td></tr>
<tr><td>Jumlah Populasi</td><td><i>100</i></td></tr>
<tr><td colspan="2">Plot nilai objective function terhadap banyak iterasi yang dilewati</td></tr>
</table>

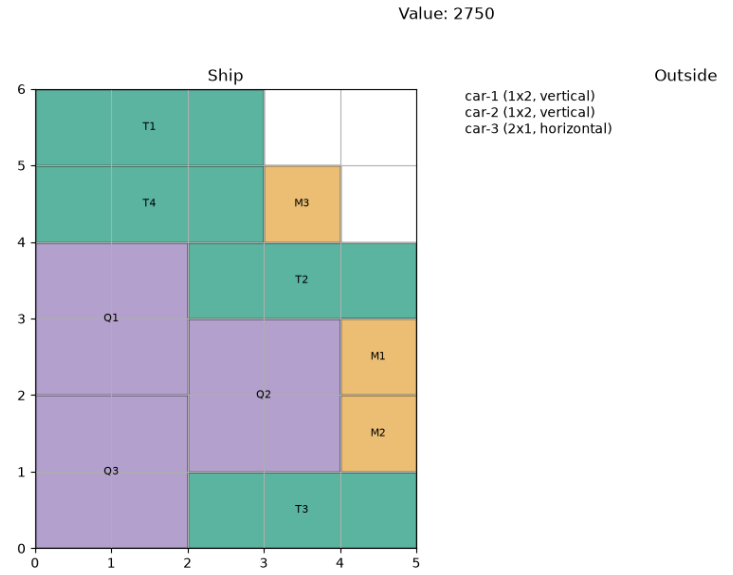


Eksperimen 14	
Visualisasi State Awal	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (1x2, vertical) car-3 (2x1, horizontal) Q3 M3 Q2 T4 T2 T1 T3 M1 Q1 M2 0 1 2 3 4 5 6

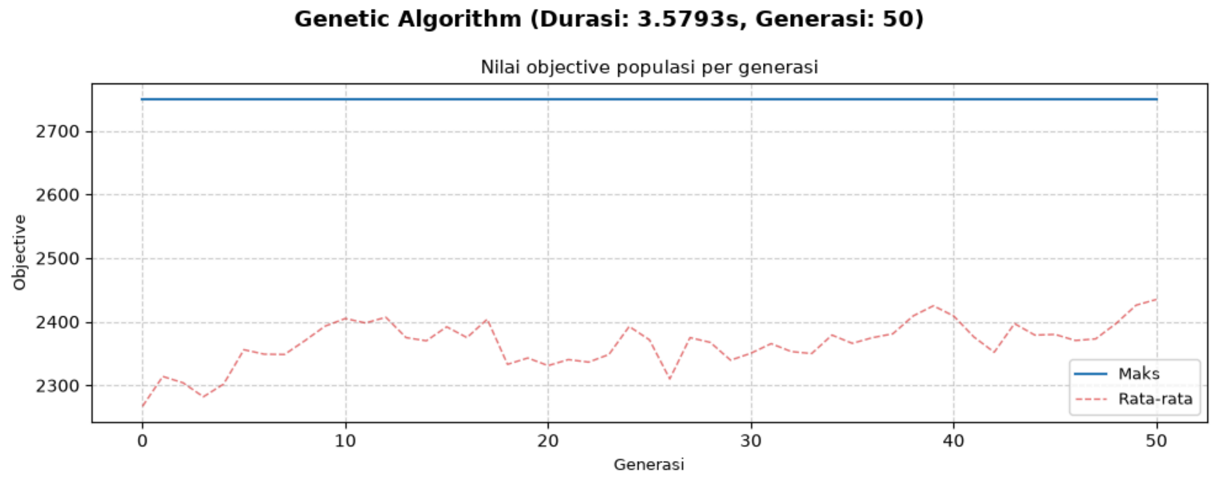
Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (1x2, vertical) car-3 (2x1, horizontal) T3 Q1 Q3 T2 T4 M1 M2 M3 Q2 T1 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***3.4897s***
Banyak Iterasi	***50***
Jumlah Populasi	***100***
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

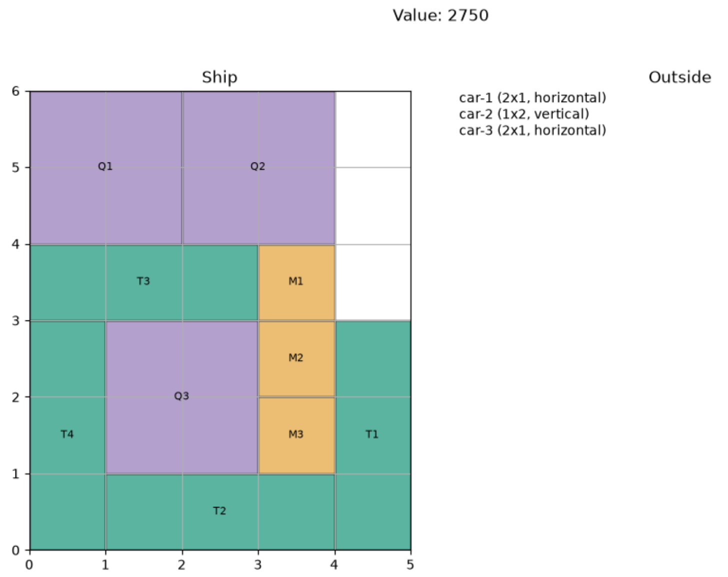
Eksperimen 15

<table>
<tr><td>Visualisasi State Awal</td><td>
Value: 2750
Ship
Outside
car-1 (2x1, horizontal)
car-2 (2x1, horizontal)
car-3 (1x2, vertical)
Q1
Q2
T1
T2
M2
Q3
T4
M3
T3
M1
0
1
2
3
4
5
6

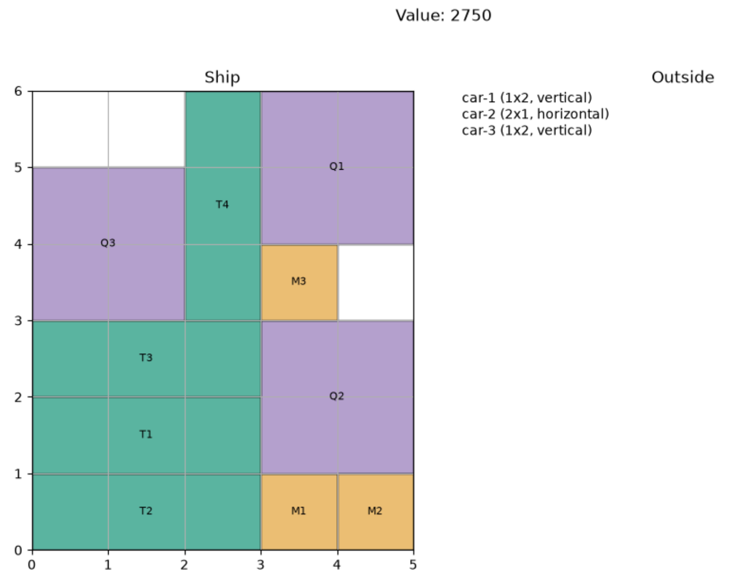
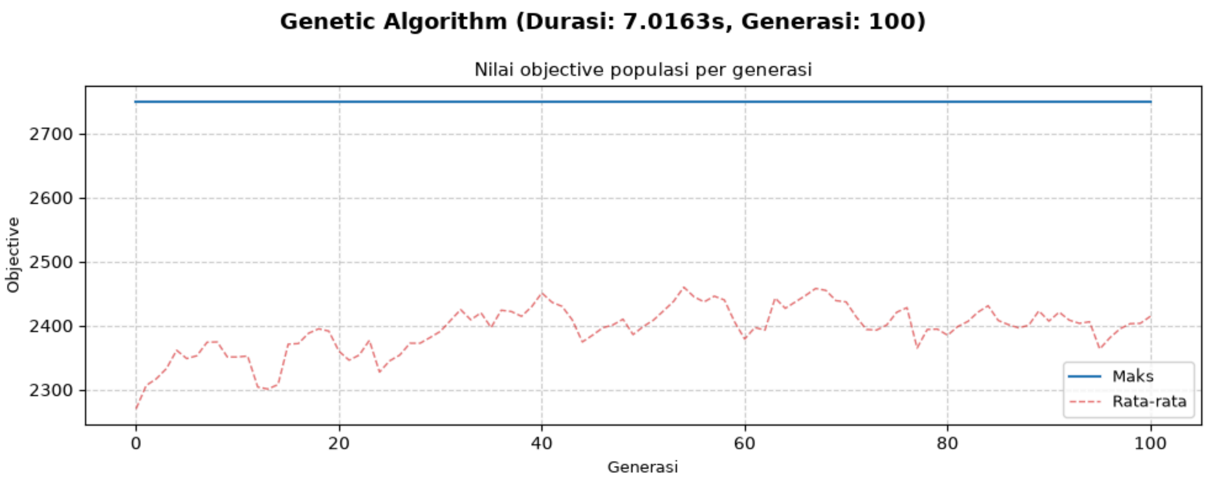
</td></tr>
<tr><td>Visualisasi State Akhir</td><td>
Value: 2750
Ship
Outside
car-1 (1x2, vertical)
car-2 (1x2, vertical)
car-3 (2x1, horizontal)
T1
T4
M3
T2
Q1
M1
Q2
M2
Q3
T3
0
1
2
3
4
5
6

</td></tr>
<tr><td>Nilai <i>objective function</i> yang dicapai</td><td>2750</td></tr>
<tr><td>Durasi Proses Pencarian</td><td><i>3.5793s</i></td></tr>
<tr><td>Banyak Iterasi</td><td><i>50</i></td></tr>
<tr><td>Jumlah Populasi</td><td><i>100</i></td></tr>
<tr><td colspan="2">Plot nilai objective function terhadap banyak iterasi yang dilewati</td></tr>
</table>

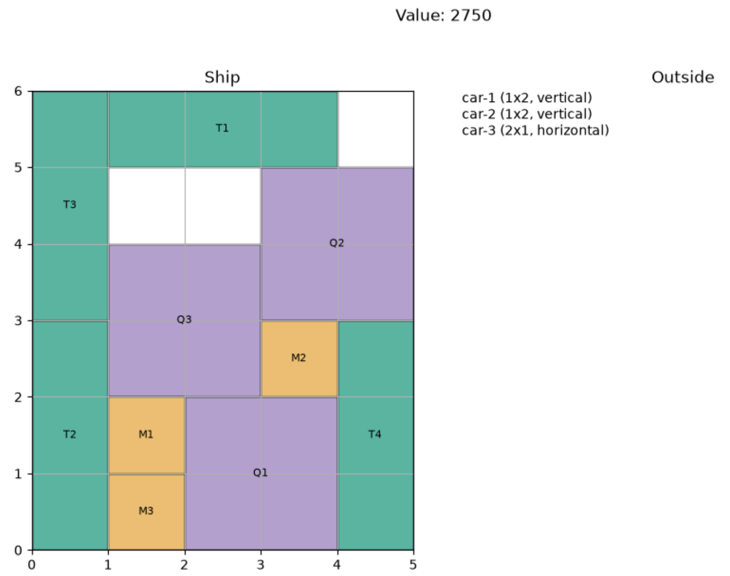


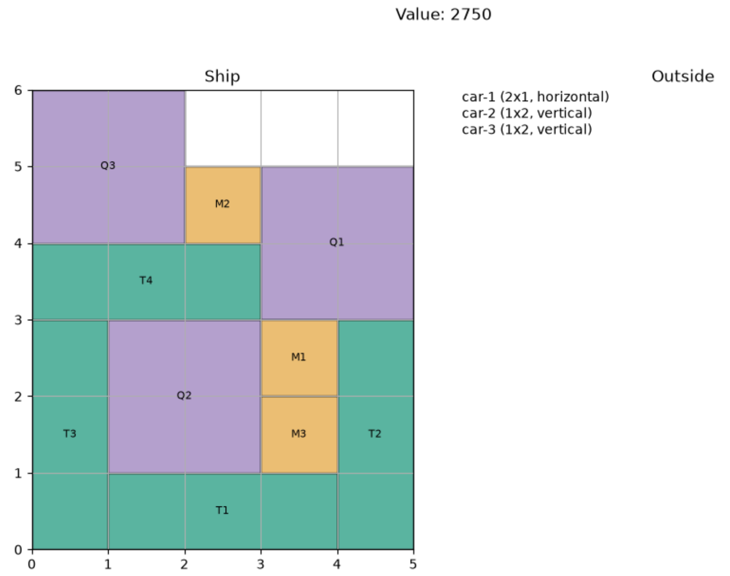
Eksperimen 16	
Visualisasi State Awal	

Value: 2750
Ship
Outside
car-1 (2x1, horizontal)
car-2 (1x2, vertical)
car-3 (2x1, horizontal)
Q1
Q2
T3
M1
M2
Q3
T4
M3
T1
T2
0
1
2
3
4
5
6

Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (2x1, horizontal) car-3 (1x2, vertical) Q1 T4 Q3 M3 T3 Q2 T1 T2 M1 M2 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***7.0163s***
Banyak Iterasi	***100***
Jumlah Populasi	***100***
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

Eksperimen 17

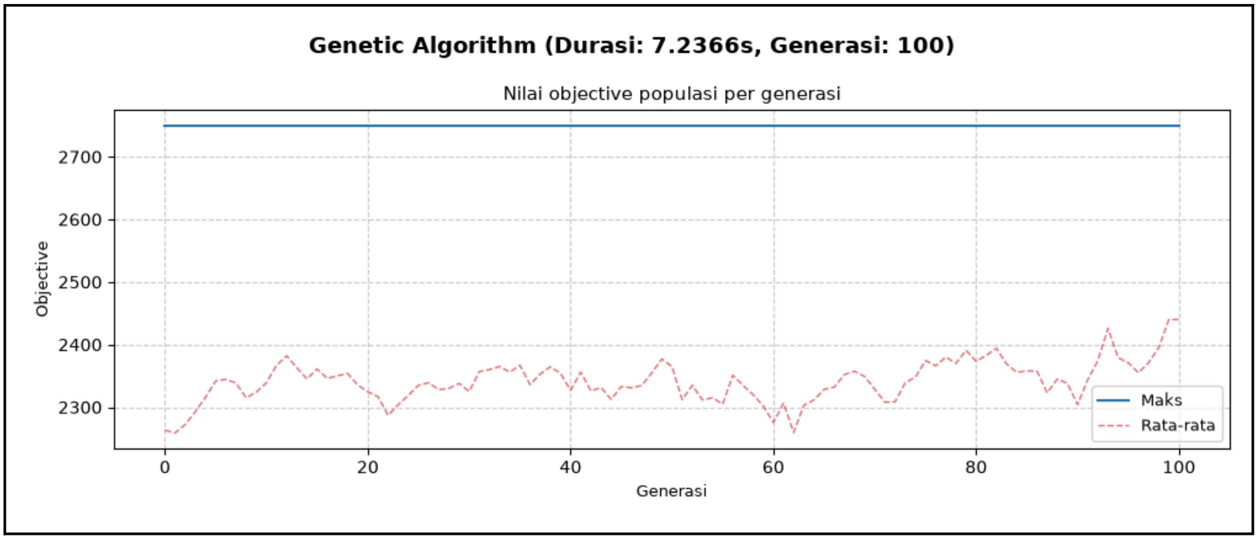
<table>
<tr><td>Visualisasi State Awal</td><td>
Value: 2750
Ship
Outside
car-1 (1x2, vertical)
car-2 (1x2, vertical)
car-3 (2x1, horizontal)
T1
T3
Q2
Q3
M2
T2
M1
T4
Q1
M3
0
1
2
3
4
5
6

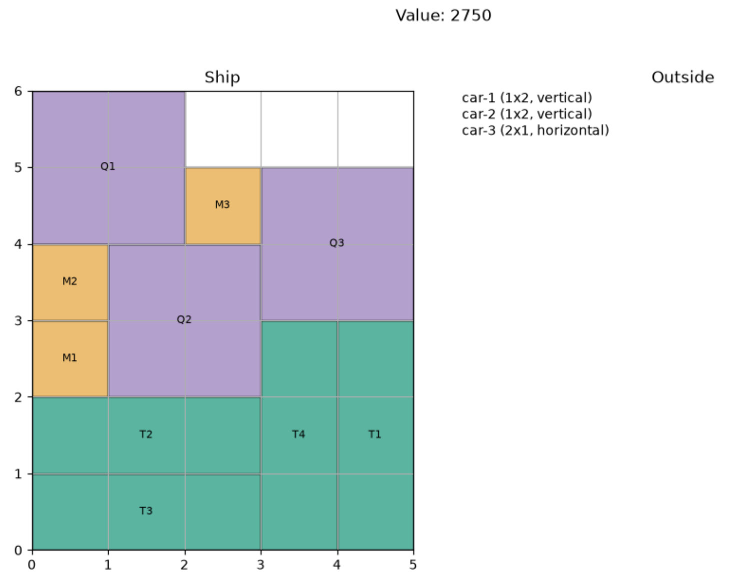
</td></tr>
<tr><td>Visualisasi State Akhir</td><td>
Value: 2750
Ship
Outside
car-1 (2x1, horizontal)
car-2 (1x2, vertical)
car-3 (1x2, vertical)
Q3
M2
Q1
T4
M1
Q2
T3
M3
T2
T1
0
1
2
3
4
5
6

</td></tr>
</table>

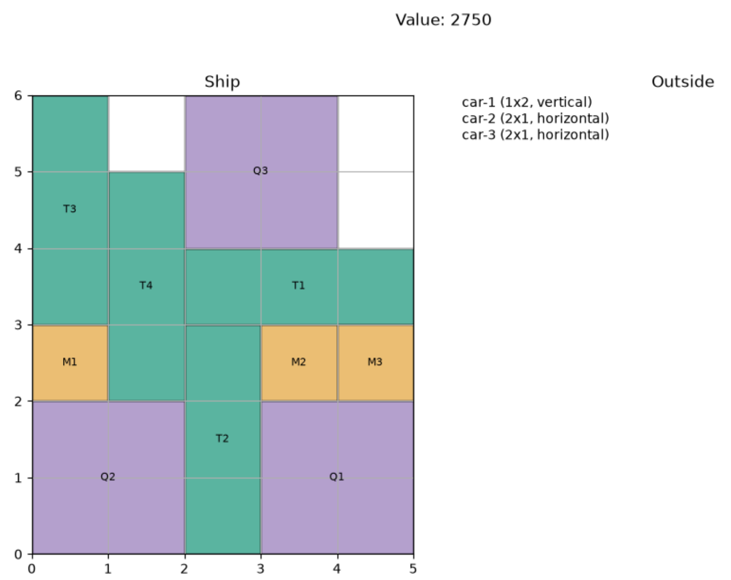
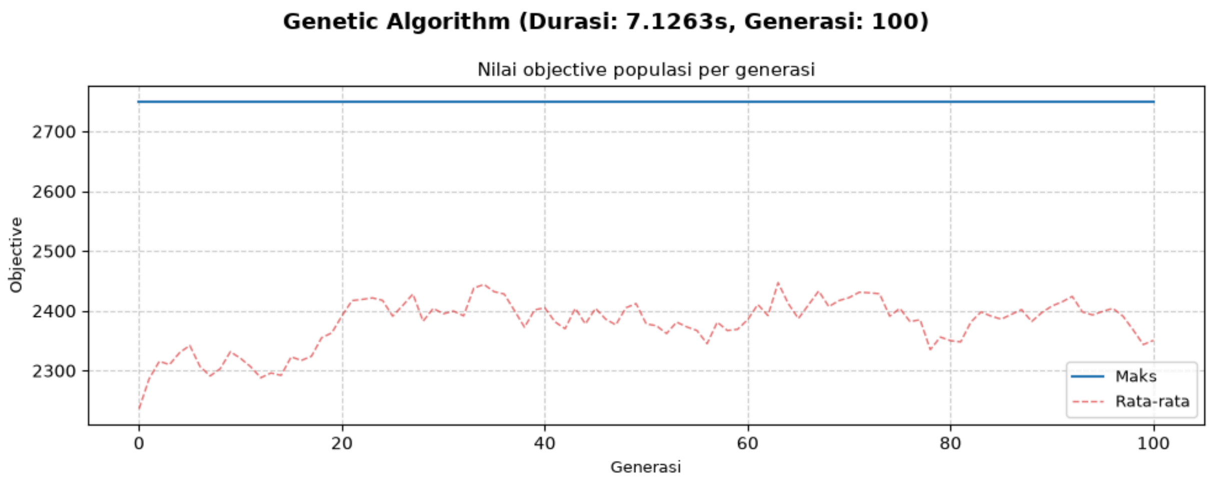
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***7.2366s***
Banyak Iterasi	***100***
Jumlah Populasi	***100***

Plot nilai objective function terhadap banyak iterasi yang dilewati



Eksperimen 18	
Visualisasi State Awal	

Value: 2750
Ship
Outside
car-1 (1x2, vertical)
car-2 (1x2, vertical)
car-3 (2x1, horizontal)
Q1
M3
Q3
M2
Q2
M1
T2
T4
T1
T3
0
1
2
3
4
5
6

Visualisasi State Akhir	 Value: 2750 Ship Outside car-1 (1x2, vertical) car-2 (2x1, horizontal) car-3 (2x1, horizontal) Q3 T3 T4 T1 M1 M2 M3 T2 Q2 Q1 0 1 2 3 4 5 6
Nilai *objective function* yang dicapai	**2750**
Durasi Proses Pencarian	***7.1263s***
Banyak Iterasi	***100***
Jumlah Populasi	***100***
Plot nilai objective function terhadap banyak iterasi yang dilewati	
	

Dari kedelapanbelas eksperimen tersebut, didapatkan

1. Run 1 menghasilkan nilai objektif akhir sebesar 2750, durasi 1,694 detik, dan sebanyak 50 generasi.

2. Run 2 menghasilkan nilai objektif akhir sebesar 2750, durasi 1,6867 detik, dan sebanyak 50 generasi.
3. Run 3 menghasilkan nilai objektif akhir sebesar 2750, durasi 1,8213 detik, dan sebanyak 50 generasi.
4. Run 4 menghasilkan nilai objektif akhir sebesar 2750, durasi 3,4527 detik, dan sebanyak 100 generasi.
5. Run 5 menghasilkan nilai objektif akhir sebesar 2750, durasi 3,4418 detik, dan sebanyak 100 generasi.
6. Run 6 menghasilkan nilai objektif akhir sebesar 2750, durasi 3,4811 detik, dan sebanyak 100 generasi.
7. Run 7 menghasilkan nilai objektif akhir sebesar 2750, durasi 6,842 detik, dan sebanyak 200 generasi.
8. Run 8 menghasilkan nilai objektif akhir sebesar 2750, durasi 6,9247 detik, dan sebanyak 200 generasi.
9. Run 9 menghasilkan nilai objektif akhir sebesar 2750, durasi 6,9883 detik, dan sebanyak 200 generasi.
10. Run 10 menghasilkan nilai objektif akhir sebesar 2750, durasi 1,4854 detik, dan sebanyak 20 generasi.
11. Run 11 menghasilkan nilai objektif akhir sebesar 2750, durasi 1,4689 detik, dan sebanyak 20 generasi.
12. Run 12 menghasilkan nilai objektif akhir sebesar 2750, durasi 1,3668 detik, dan sebanyak 20 generasi.
13. Run 13 menghasilkan nilai objektif akhir sebesar 2750, durasi 3,5265 detik, dan sebanyak 50 generasi.
14. Run 14 menghasilkan nilai objektif akhir sebesar 2750, durasi 3,4897 detik, dan sebanyak 50 generasi.
15. Run 15 menghasilkan nilai objektif akhir sebesar 2750, durasi 3,5793 detik, dan sebanyak 50 generasi.
16. Run 16 menghasilkan nilai objektif akhir sebesar 2750, durasi 7,0163 detik, dan sebanyak 100 generasi.
17. Run 17 menghasilkan nilai objektif akhir sebesar 2750, durasi 7,2366 detik, dan sebanyak 100 generasi.
18. Run 18 menghasilkan nilai objektif akhir sebesar 2750, durasi 7,1263 detik, dan sebanyak 100 generasi.

Berdasarkan seluruh 18 run, Genetic Algorithm secara konsisten menghasilkan nilai objektif akhir sebesar 2750. Jumlah generasi yang digunakan berbeda-beda, yaitu 20, 50, 100, dan 200 generasi. Durasi run berada pada rentang 1,3668–7,2366 detik. Dengan demikian, nilai akhir yang sama tercapai pada berbagai konfigurasi generasi, meskipun durasi komputasinya berbeda.

2.4. Analisis Hasil Eksperimen

1. Tingkat Keberhasilan Mendekati Global Optima

Berdasarkan batas atas kapasitas bobot (4500 kg) dan luas geladak kapal (30 sel), estimasi nilai Global Optima pada permasalahan ini berada pada angka 2750. Kedekatan masing-masing algoritma terhadap nilai tersebut adalah sebagai berikut:

- Genetic Algorithm (GA): Sangat Unggul. Mengukir tingkat keberhasilan 100% konsisten mencapai nilai optimum global (2750) pada seluruh eksperimen. Hal ini didorong oleh pendekatan berbasis populasi (*population-based*) yang melakukan eksplorasi paralel, dikombinasikan dengan mekanisme *greedy placement decoding* yang mampu memaksimalkan pemadatan ruang geladak secara presisi.
- Random Restart Hill Climbing: Sangat Baik. Mampu mencapai *global optima* 2750 pada run tertentu (dengan rata-rata 2616.7 dan rentang 2550–2750). Mekanisme pembentukan ulang *state* awal secara acak terbukti efektif membebaskan algoritma dari jebakan satu puncak lokal (*local optima*).
- Steepest Ascent & Sideways Move Hill Climbing: Cukup Baik. Mampu menyentuh angka 2750 apabila *initial state* yang dibangkitkan menguntungkan. Namun, keduanya rentan terhenti pada nilai sub-optimal (rata-rata 2370–2388; rentang 2050–2750) karena sifat evaluasi tetangganya yang deterministik/semi-deterministik membatasi pergerakan saat mencapai puncak lokal.
- Simulated Annealing (SA): Sedang. Menghasilkan nilai akhir pada rentang 1500 hingga 2200 (rata-rata 1733.3). Pada domain diskrit *bin packing* dengan *constraint* geometris yang ketat, pergerakan acak (*swap/move/orient*) sering menghasilkan posisi tidak valid (*invalid placement*), sehingga eksplorasi pada suhu rendah menjadi kurang optimal.
- Stochastic Hill Climbing: Kurang. Memperoleh hasil terendah (rata-rata 1616.7; rentang 1450–1850). Algoritma ini hanya memilih tetangga membaik secara acak tanpa meninjau seluruh *neighborhood*, sehingga sering menaiki bukit lokal yang landai dan terhenti sebelum mencapai konfigurasi berkerataan tinggi.

2. Perbandingan Hasil Pencarian Antar-Algoritma

Urutan efektivitas kualitas solusi akhir dari yang terbaik adalah:

1. Genetic Algorithm (GA)
2. Random Restart Hill Climbing
3. Steepest Ascent Hill Climbing ≈ Sideways Move Hill Climbing *(setara)*
4. Simulated Annealing
5. Stochastic Hill Climbing

- Genetic Algorithm vs. Local Search: GA jauh mengungguli seluruh varian *local search* berbasis individu tunggal. Pencarian lokal murni hanya mengolah satu *state* yang rawan memicu fragmentasi ruang geladak, sedangkan GA memadukan urutan penataan terbaik dari berbagai individu untuk menghasilkan pemadatan kapal yang efisien.
- Komparasi Varian Hill Climbing: *Random Restart HC* menjadi varian HC terbaik berkat cakupan eksplorasi di beberapa area *state space*. *Sideways Move HC* memiliki keunggulan berupa toleransi langkah horizontal pada *plateau* dibanding *Steepest Ascent HC*, walaupun kualitas akhir keduanya relatif setara. *Stochastic HC* berada di posisi terbawah karena pergerakannya yang spekulatif tanpa jaminan langkah terbaik (*steepest step*).
- Karakteristik Simulated Annealing: Walaupun SA memiliki fitur penerimaan langkah memburuk pada suhu tinggi untuk lolos dari *local optima*, kondisi *fitness landscape* yang terputus-putus (*rugged*) pada masalah ini membuat kinerjanya berada di bawah varian *Hill Climbing* utama.

3. Perbandingan Durasi Waktu Eksekusi (Komputasi)

Urutan algoritma dari yang paling cepat hingga paling lambat adalah:

1. Steepest Ascent Hill Climbing ($\sim 0.061s$)
2. Stochastic Hill Climbing ($\sim 0.079s$)
3. Sideways Move Hill Climbing ($\sim 0.219s$)
4. Random Restart Hill Climbing ($\sim 0.364s$)
5. Simulated Annealing ($\sim 0.427s$)
6. Genetic Algorithm ($1.36s - 7.24s$)

- Steepest Ascent & Stochastic HC (Paling Cepat): Membutuhkan waktu di bawah 0.1 detik karena proses konvergensi berlangsung singkat (hanya 4–8 iterasi).
- Sideways Move HC: Membutuhkan waktu lebih lama (~ 0.22 detik) karena melakukan penelusuran tambahan pada area *plateau* (bisa mencapai > 100 iterasi).

- Random Restart HC: Memiliki durasi komputasi yang berbanding lurus dengan jumlah *restart* yang dilakukan (~ 0.36 detik).
- Simulated Annealing: Berjalan sangat stabil (~ 0.43 detik) karena durasinya terikat langsung oleh skenario penurunan suhu (*cooling schedule*) sebanyak 1838 iterasi.
- Genetic Algorithm (Paling Lambat): Memerlukan waktu paling tinggi (1.36 hingga 7.24 detik) akibat kompleksitas komputasi yang tinggi, di mana proses dekode dan evaluasi *fitness* dilakukan untuk seluruh populasi (50–100 individu) pada setiap generasinya.

4. Tingkat Konsistensi dan Stabilitas Solusi

- Genetic Algorithm: Konsisten Sempurna (Variansi = 0). Dari 18 kali pengujian independen dengan konfigurasi populasi dan generasi berbeda, seluruh pengujian secara stabil menghasilkan nilai akhir tepat 2750.
- Simulated Annealing: Cukup Konsisten. Meskipun stabil, konvergensinya berada pada level sub-optimal (sebagian besar pengujian bernilai persis 1500, dan sebagian kecil mencapai 2200).
- Random Restart Hill Climbing: Konsisten Tinggi. Menghasilkan nilai akhir pada rentang atas yang sempit (2550–2750) dengan standar deviasi rendah.
- Steepest Ascent & Sideways Move Hill Climbing: Inkonsisten Moderat. Memiliki rentang hasil yang cukup lebar (2050–2750) karena performa akhirnya sangat bergantung (*path-dependent*) pada *state* awal yang dibangkitkan.
- Stochastic Hill Climbing: Paling Tidak Konsisten. Menunjukkan fluktuasi paling tinggi (1450–1850) akibat ketergantungan penuh pada keputusan acak di setiap iterasi langkah.

5. Pengaruh Ukuran Populasi dan Banyak Generasi pada Genetic Algorithm

Pengujian dilakukan dengan variasi populasi ($N \in \{50, 100\}$) dan jumlah generasi ($G \in \{20, 50, 100, 200\}$):

- Pengaruh Ukuran Populasi: Ukuran populasi yang lebih besar ($N = 100$) meningkatkan keberagaman genetik (*genetic diversity*) sejak awal. Pada $N = 100$, solusi optimal global (2750) sudah ditemukan hanya dalam 20 generasi. Sementara pada $N = 50$, dibutuhkan setidaknya 50 generasi untuk menyentuh nilai tersebut. Namun, populasi yang besar berkonsekuensi pada
- peningkatan beban komputasi per generasi.
- Pengaruh Jumlah Generasi (Iterasi): Menambah jumlah generasi memberikan kesempatan bagi proses *crossover* dan mutasi untuk memperbaiki struktur kromosom. Namun, setelah populasi mengalami konvergensi (nilai 2750 tercapai pada generasi ≤ 50), penambahan generasi lebih lanjut (100 hingga 200) tidak

lagi meningkatkan kualitas solusi, melainkan hanya menambah waktu eksekusi secara linier (durasi melonjak dari ~ 1.4 detik menjadi > 7.2 detik).

- Rekomendasi Parameter: Terjadi efek *diminishing return*. Kombinasi parameter paling efisien adalah populasi 50 dengan 50 generasi atau populasi 100 dengan 20–50 generasi, karena mampu mencapai nilai optimum global tanpa membuang waktu komputasi.

3. KESIMPULAN DAN SARAN

3.1. Kesimpulan

Berdasarkan hasil eksperimen, diperoleh beberapa kesimpulan berikut.

1. Fungsi objektif pada eksperimen ini memaksimalkan jumlah shipping fee kendaraan yang berhasil dimuat. Nilai 2750 digunakan sebagai acuan estimasi global optimum.
2. Dari seluruh algoritma dan run yang dianalisis, Genetic Algorithm memperoleh hasil paling konsisten: nilai objektif akhirnya mencapai 2750 pada seluruh 18 run. Durasi eksekusinya berada pada rentang 1,3668–7,2366 detik, bergantung pada ukuran populasi dan jumlah generasi yang digunakan.
3. Pada tiga run pertama, Random Restart Hill Climbing memiliki rata-rata nilai objektif tertinggi di antara algoritma non-genetik, yaitu 2616,67 dengan rentang 2550–2750. Steepest Ascent Hill Climbing memperoleh rata-rata 2483,33 dengan rentang 2250–2750, sedangkan Sideways Move Hill Climbing memperoleh rata-rata 2450 dengan rentang 2050–2750.

4. Simulated Annealing memperoleh rata-rata nilai objektif 1733,33 dengan rentang 1500–2200. Sementara itu, Stochastic Hill Climbing memperoleh rata-rata 1616,67 dengan rentang 1450–1850. Pada run yang dianalisis, kedua algoritma ini menghasilkan nilai objektif rata-rata yang lebih rendah daripada varian Hill Climbing lainnya.
5. Durasi rata-rata Steepest Ascent pada tiga run pertama adalah sekitar 0,0696 detik, sedangkan Stochastic Hill Climbing sekitar 0,079 detik. Keduanya lebih cepat daripada Random Restart Hill Climbing (0,364 detik) dan Simulated Annealing (0,427 detik). Genetic Algorithm menghasilkan nilai akhir paling konsisten, tetapi durasinya lebih tinggi.

Secara keseluruhan, hasil eksperimen menunjukkan pertukaran antara kualitas solusi, konsistensi, dan waktu komputasi. Genetic Algorithm memberikan hasil terbaik dan paling konsisten pada pengujian yang tersedia. Random Restart Hill Climbing menjadi alternatif pencarian lokal dengan rata-rata nilai objektif yang baik, sementara Steepest Ascent dan Stochastic Hill Climbing menawarkan waktu eksekusi singkat. Kesimpulan ini berlaku untuk run dan konfigurasi yang diuji; nilai 2750 merupakan estimasi optimum yang digunakan dalam eksperimen, bukan pembuktian matematis atas optimum global.

3.2. Saran

Untuk pengembangan dan eksperimen selanjutnya, beberapa saran yang dapat dipertimbangkan adalah sebagai berikut.

1. Gunakan Genetic Algorithm dengan populasi 100 dan 20 generasi sebagai konfigurasi awal yang efisien. Pada tiga run dengan konfigurasi tersebut, nilai objektif akhir mencapai 2750 dan durasi rata-ratanya sekitar 1,44 detik. Hasil ini tetap perlu diuji dengan lebih banyak run sebelum dijadikan kesimpulan umum.
2. Pilih algoritma sesuai kebutuhan. Jika prioritasnya kualitas dan konsistensi hasil, Genetic Algorithm dapat dipertimbangkan. Jika membutuhkan metode yang lebih cepat, Steepest Ascent atau Stochastic Hill Climbing dapat menjadi pilihan. Random Restart Hill Climbing dapat digunakan ketika kualitas solusi lokal yang baik lebih diutamakan daripada waktu eksekusi minimum.
3. Samakan kondisi perbandingan ant algoritma. Eksperimen berikutnya sebaiknya menggunakan seed atau keadaan awal yang terkontrol dan anggaran komputasi yang sebanding. Dengan demikian, pengaruh algoritma dapat dibedakan dari pengaruh konfigurasi dan kondisi awal.

4. Lakukan pengujian berulang dengan jumlah run yang seimbang. Sajikan rata-rata, rentang, dan ukuran variasi hasil untuk setiap algoritma. Hal ini akan membuat perbandingan konsistensi lebih kuat, terutama karena jumlah run Genetic Algorithm saat ini lebih banyak daripada algoritma lainnya.
5. Verifikasi kelayakan solusi dan pelaporan metrik. Laporkan secara terpisah nilai objektif yakni total shipping fee yang dimaksimalkan durasi, iterasi atau generasi, serta pelanggaran constraint jika metrik tersebut memang diperiksa. Sebutan “biaya” sebaiknya dihindari untuk nilai objektif agar tidak menimbulkan kesan bahwa nilainya diminimalkan.

4. PEMBAGIAN TUGAS

Nama	NIM	Kegiatan
Faizal Ali	18224007	1. Simulated Annealing 2. HC Random Restart 3. Dokumen
Muhammad Fariz Difa’urahman	18224028	1. Steepest Ascend HC, Sideways Move HC, Stochastic HC 2. Dokumen
Arkandhiya Ibrahim Dewantara	18224052	1. All Utilities 2. Genetic Algorithm

5. REFERENSI

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Sep. 16, 2026].

6. LAMPIRAN

6.1. LAMPIRAN FORM PENGGUNAAN GENERATIVE AI

FORM PENGGUNAAN GENERATIVE AI

6.2. LAMPIRAN LAIN